

COMPANY OPERATING SYSTEM

Web Development Department Handbook

From Zero to Professional: training, engineering standards, client delivery, costing and maintenance

Developed by **V. LOKESHWARAN**
VEL Software Solutions
velsoftwaresolutions.in@gmail.com

Document VEL-WD-001 · Version 1.0 (draft for management approval)
Tool versions, limits and vendor prices change. Always verify against official documentation and pricing calculators before building or quoting.

Table of Contents

PART A · FOUNDATIONS

How to Use This Handbook 5

1. Department and Service Portfolio 8

2. Employee Onboarding Programme 11

3. Learning Roadmap: Levels 1-5 14

Level 1 - Web Fundamentals (about 2 weeks) 15

L1.1 · Internet and Web Architecture 15

L1.2 · Browser and Server 16

L1.3 · HTTP and HTTPS 17

L1.4 · DNS and Domains 18

L1.5 · Hosting 19

L1.6 · Frontend and Backend 20

L1.7 · Databases (concepts) 21

L1.8 · APIs 22

L1.9 · Responsive Design (concepts) 23

Level 2 - Frontend Development (about 5 weeks) 24

L2.1 · HTML5 25

L2.2 · CSS3 26

L2.3 · JavaScript 27

L2.4 · TypeScript 28

L2.5 · Responsive Layouts - Flexbox and Grid 29

L2.6 · Accessibility 30

L2.7 · Browser Developer Tools 31

L2.8 · React 33

L2.9 · Next.js (and when to choose alternatives) 34

Level 3 - Backend Development (about 4 weeks) 35

L3.1 · Backend Framework and Your First API 36

L3.2 · REST API Design 38

L3.3 · Authentication 39

L3.4 · Authorisation 41

L3.5 · Business Logic and Service Layer 42

L3.6 · Input Validation 43

L3.7 · Error Handling and Logging Contracts 44

L3.8 · File Uploads 46

L3.9 · Email Integration 47

Level 4 - Database Engineering (about 3 weeks) 48

L4.1 · PostgreSQL and MySQL - Choosing and Installing 48

L4.2 · Database Design and Normalisation 50

L4.3 · Relationships and Keys 51

L4.4 · SQL 52

L4.5 · Indexes and Query Performance 53
L4.6 · Transactions and Concurrency 54
L4.7 · Migrations and Seed Data 55
L4.8 · Backup and Recovery 56
Level 5 - Professional Engineering (about 4 weeks) 57
L5.1 · Git 58
L5.2 · GitHub 59
L5.3 · VS Code 60
L5.4 · Code Reviews 61
L5.5 · Testing 62
L5.6 · Docker 64
L5.7 · CI/CD with GitHub Actions 65
L5.8 · Cloud Deployment 66
L5.9 · Logging 68
L5.10 · Monitoring and Alerting 69
L5.11 · Web Application Security 70
Level assessment summary 71
4. Technology Stack and Tool Selection 72
PART B · CLIENT DELIVERY
5. Client Project Lifecycle: Stages 01-27 76
Stage definitions 77
Stage 01 — Lead Qualification 77
Stage 02 — Client Discovery 78
Stage 03 — Business and Technical Requirement Collection 79
Stage 04 — Competitor and Market Research 79
Stage 05 — Scope and Statement of Work (SOW) Approval 80
Stage 06 — Input/Output and Feature Freeze 80
Stage 07 — Sitemap and Information Architecture 81
Stage 08 — UI/UX Design Approval 81
Stage 09 — Technical Architecture 82
Stage 10 — Database Design 83
Stage 11 — Development Environment Setup 83
Stage 12 — GitHub Repository Creation 84
Stage 13 — Frontend Development 84
Stage 14 — Backend Development 85
Stage 15 — API Integration 85
Stage 16 — Database Implementation 86
Stage 17 — Authentication and Permissions 86
Stage 18 — Payment and Third-Party Integrations 87
Stage 19 — Unit and Integration Testing 87
Stage 20 — Security and Performance Testing 88
Stage 21 — Staging Deployment 88

Stage 22 — Client Review and User Acceptance Testing (UAT) 89
Stage 23 — Production Deployment 89
Stage 24 — DNS and Domain Configuration 90
Stage 25 — SSL/TLS Verification 90
Stage 26 — Client Handover 91
Stage 27 — Maintenance and Support 92

6. Project Structure and Repository Standards 93

7. Professional Delivery Standards 100

PART C · BUSINESS AND PEOPLE

8. Commercial Costing and Pricing 105

9. Roles, Procedures and Escalation 113

PART D · PRACTICE AND SOURCES

10. Practical Employee Projects and Certification 117

11. Official Learning Sources 121

PART E · TOOLKIT

A1. Employee Onboarding Checklist 124

A2. 90-Day Training Plan 127

A3. Project Discovery Questionnaire 129

A4. Requirements Template 132

A5. Statement of Work Template 135

A6. QA Checklist 139

A7. Deployment Checklist 142

A8. Client Handover Checklist 144

A9. Project Costing Spreadsheet Specification 147

A10. Glossary 150

VEL SOFTWARE SOLUTIONS

PART A · FOUNDATIONS

How to Use This Handbook

This handbook is the single operating manual of the **Web Development Engineering Department**. It does two jobs: **(1)** it trains a beginner who knows basic computer operations into a professional who can join supervised client projects, and **(2)** it defines the mandatory 27-stage lifecycle every client project follows - with approval gates, deliverables and acceptance criteria - from the first enquiry to maintenance.

GOLDEN RULES

1. Do not skip stages. Do not start a stage until the acceptance criteria of the previous stage are met and its gate is signed.
2. "It runs on my laptop" is not "production-ready". Production-ready means secure, tested, monitored, backed up, restorable, documented, cost-controlled and handed over.
3. Never commit credentials, private keys, production `.env` files or confidential customer data to Git, chat, e-mail or documents.
4. Never run destructive commands (`rm -rf`, `DROP DATABASE`, `git push --force`) without understanding the effect and holding a verified backup.
5. Every production change needs a second person's review.
6. Never promise uptime, "unhackable" security, rankings or savings without evidence. We say what was tested - not that a system is "secure".
7. Never invent prices. Quote only from official vendor pricing pages and calculators, with the date checked.

Reading path for a new employee

Part A · Foundations → Part B · Client delivery → Part C · Business and people →
Part D · Practice and sources → Part E · Toolkit

Part	What you will do	Typical duration
A - Foundations	Understand the department, complete onboarding, study Levels 1-5, choose tools	Weeks 1-12
B - Client delivery	Learn and follow the 27-stage lifecycle, repository structure and delivery standards	Continuous, from week 4
C - Business and people	Understand costing, pricing, roles, daily procedures and escalation	Weeks 6-12, then reference
D - Practice and sources	Build seven practical projects; use official learning sources	Weeks 3-13
E - Toolkit	Use the checklists and templates on real work	Continuous

How every topic is taught (15-step methodology)

Each learning module in Section 3 follows the same fifteen steps so that nothing is skipped and every module can be assessed the same way.

#	Step	What it gives you
1	Definition	A plain-language meaning
2	Business purpose	Why a client would pay for it
3	Real-world example	A concrete case
4	Prerequisites	What you must know first
5	Tools and installation	What to install and whether it is free or paid
6	Step-by-step procedure	Exactly what to do
7	Complete example	Working code or configuration to copy and run
8	Expected output	What you should see
9	Verification	How to prove it works
10	Common errors	What usually goes wrong
11	Security considerations	How it can be attacked and how to defend
12	Documentation	What to write down
13	Practical assignment	Hands-on work
14	Assessment	How you are tested
15	Approval gate	Who signs you off

HOW TO USE THE HTML EDITION

Press `/` to search. Click a module title to collapse or expand it. Use **Copy** on any code block. Tick "Mark module complete" to track your own progress (saved in your browser only). Use **Print** for a clean printable version.

Conventions used in this handbook

Symbol / style	Meaning
`monospace`	A command, file name, code or configuration value
Gate box (amber)	A mandatory approval point
Forbidden / danger box (red)	An action that is never allowed, or a serious risk
Note / tip box (blue / green)	Helpful context or a recommended practice
CU	Generic currency unit used in costing examples (no real currency implied)
Level / Module ids	`L2.8` = Level 2, module 8

Document control

Item	Value
Document	VEL-WD-001 - Web Development Department Handbook
Version	1.0 (draft for management approval)
Owner	Technical Lead, Web Development Department
Review cycle	Every 6 months, or when a major tool/standard changes
Applies to	All web development employees, interns and approved contractors
Design reference	VEL Cloud Engineering and DevOps Playbook (VEL-CD-001) - page size, palette, typography and layout

VERIFY BEFORE YOU BUILD OR QUOTE

Tool versions, free-tier limits, licence terms and cloud prices change often. This handbook states **methods and formulas**, and it points to official sources. It deliberately does **not** state vendor prices. Always check the official documentation and pricing calculators on the day you build or quote, and record the source and date.

PART A · FOUNDATIONS

1. Department and Service Portfolio

The Web Development Engineering Department builds, launches and maintains web products for business clients. Fourteen commercially deliverable services are offered. Each can be sold alone or combined, and each follows the same 27-stage lifecycle (Section 5) - the lifecycle is scaled to the project, never skipped.

Commercial service portfolio

#	Service	What we deliver	Typical client trigger	Typical stack (chosen per project)
1	**Corporate website**	Multi-page brand site, CMS-editable content, SEO, analytics, contact/lead forms	"We need a professional company site."	Next.js or Astro + headless CMS, or WordPress
2	**Business website**	Small business site (3-8 pages), maps, WhatsApp/phone links, enquiry form	"We only have a Facebook page."	Static/Next.js + serverless form, or WordPress
3	**Landing page**	One-page campaign site, fast, tracked, A/B-test ready	"We run ads and need a page that converts."	Static HTML/Next.js on CDN
4	**E-commerce platform**	Catalogue, cart, checkout, payments, shipping, tax invoices, admin	"We want to sell online."	Shopify / WooCommerce, or custom Next.js + PostgreSQL
5	**Custom web application**	Workflow software built to the client's process	"Our spreadsheets cannot cope."	React/Next.js + FastAPI/Node + PostgreSQL
6	**SaaS web application**	Multi-tenant product with billing, roles, onboarding, analytics	"We want to launch a software product."	Next.js + API + PostgreSQL + queue + billing provider
7	**Customer portal**	Secure login area - orders, invoices, tickets, documents	"Customers keep e-mailing us for statements."	Next.js + API + managed identity
8	**Admin dashboard**	Internal control panel - users, content, reports, exports	"We need to manage the data without a developer."	React admin + API, or Django admin
9	**CMS development**	Content model, editor experience, preview, roles, workflows	"Staff must update the site without us."	Sanity/Strapi/Directus/WordPress/Django CMS
10	**Booking and appointment system**	Services, availability, calendar sync, reminders, payment, cancellations	"Phone bookings waste our time."	Next.js + API + PostgreSQL + e-mail/SMS provider

#	Service	What we deliver	Typical client trigger	Typical stack (chosen per project)
11	**Payment gateway integration**	Hosted checkout, webhooks, refunds, reconciliation	"Add online payments to our site."	Gateway SDK + server-side verification
12	**Website redesign and modernisation**	Audit, new design, migration of content/URLs, redirects, SEO protection	"Our site looks old and is slow."	Depends on audit
13	**Performance optimisation**	Core Web Vitals audit and fixes, caching, CDN, image pipeline, database tuning	"The site is slow and ranks badly."	Lighthouse, WebPageTest, CDN, profiling
14	**Maintenance and managed support**	Monitoring, patching, backups, content changes, SLA support, monthly report	"Nobody looks after our site."	Uptime monitor, Dependabot, ticket system

Service packaging

Package	Typical contents	Pricing approach
Starter	Landing page or small business site, 1 revision round, 30-day warranty	Fixed price
Professional	Corporate site or portal, 2 revision rounds, staging review, 60-day warranty, 3-month support option	Fixed price + monthly service
Advanced	Custom app / e-commerce / SaaS with phased milestones, security review, 90-day warranty	Milestone-based fixed price or time-and-materials with cap
Managed	Maintenance, monitoring, hosting management, SLA	Monthly retainer (MRR)

SELECT TECHNOLOGY BY REQUIREMENT

We never force one stack on every client. A five-page brochure site does not need a Kubernetes cluster; a payment-processing SaaS does not belong on a 30-page WordPress theme. The Technical Lead records every stack decision in an Architecture Decision Record (Stage 09).

Delivery models

Model	Best for	Control	Risk to us
Fixed price	Clearly defined scope	Change-request procedure	Under-estimation - mitigated by contingency and a frozen scope
Time and materials (capped)	Evolving requirements	Weekly timesheet report; cap needs written approval to raise	Low
Milestone-based	Large custom products	Payment tied to accepted deliverables	Medium

Model	Best for	Control	Risk to us
Retainer / managed service	Ongoing support and improvement	SLA, monthly report, hour bank	Un-priced "unlimited" support - never offer it

Department structure

Role	Reports to	Core responsibility
Technical Lead	Director	Architecture, standards, code quality, security sign-off
Project Manager	Director	Scope, schedule, budget, client communication
UI/UX Designer	Technical Lead / PM	Research, wireframes, design system, accessibility
Frontend and backend developers (junior to senior)	Technical Lead	Build and test features
Full-stack developer	Technical Lead	Features end to end
QA engineer	Technical Lead	Test planning, automation, release readiness
DevOps engineer	Technical Lead	Environments, CI/CD, hosting, monitoring, backups

Detailed role descriptions are in Section 9.

WHAT WE DO NOT DO

We do not copy a competitor's text, images or code. We do not build deceptive, illegal or privacy-violating systems. We do not store card numbers ourselves. We do not offer legal, tax or regulatory advice; we offer **compliance-readiness support** and say that certification and legal compliance are determined by qualified auditors and counsel. We never claim a system is "secure" or "guaranteed" - we state what was tested.

PART A · FOUNDATIONS

2. Employee Onboarding Programme

The onboarding programme turns a new joiner into a safe, productive team member in the first 13 weeks. It combines **access and policy setup (days 1-5)**, **foundation learning (weeks 1-4)** and **supervised delivery (weeks 5-13)**. The detailed tick-list is **Appendix A1**; the week-by-week plan is **Appendix A2**.

Who you are on day one

You know basic computer operations - files, folders, browser, e-mail. You are **not** expected to know programming. The programme assumes zero professional web experience and builds from there. You will always work under a named mentor until you pass Gate 5 (week 13).

Onboarding timeline

When	Focus	You will finish with
Before day 1	HR paperwork, laptop prepared, accounts requested	Laptop with disk encryption, accounts ready
Day 1	Welcome, company overview, security and confidentiality briefing, policies signed	Signed NDA and acceptable-use policy; password manager and MFA active
Day 2	Tooling - Git, GitHub, VS Code, terminal; clone the handbook repo	First commit and pull request
Day 3	Department tour - lifecycle, folder structure, branching, review rules	You can explain the 27 stages in your own words
Day 4	Level 1 start; mentor pairing; first practice task	Level 1 module L1.1 complete
Day 5	Week-1 review with mentor; set 30/60/90-day goals	Signed learning plan
Weeks 2-4	Levels 1 and 2 (HTML, CSS, JavaScript)	Projects 1 and 2 started
Weeks 5-8	Levels 3 and 4 (backend, databases)	Projects 3 and 4
Weeks 9-11	Level 5 (professional engineering)	Projects 5 and 6
Weeks 12-13	Simulated client project; final assessment	Project 7; certification decision

Accounts, devices and access (day 1-2)

Item	Rule
Laptop	Company-approved, full-disk encryption, screen lock at 5 minutes, OS and browser auto-update
Identity	One individual account per tool (e-mail, GitHub, project tracker, cloud console). **No shared logins**
MFA	Mandatory on e-mail, GitHub, cloud, password manager, registrar

Item	Rule
Password manager	Company-approved; unique random password per site; no passwords in notes, chat or files
SSH key	`ssh-keygen -t ed25519 -C "you@company.com"` with a passphrase; add the **public** key to GitHub; never share the private key
Access level	Least privilege: read-only first, write after Gate 2, production access only after Gate 5 and only for assigned projects
Personal devices	No client data on personal devices unless the written policy permits it

FORBIDDEN

Shared root or admin accounts. Client passwords in chat or e-mail. Client source code on personal GitHub accounts. Production credentials on laptops. Copying client data to USB drives or personal cloud storage.

Policies every employee signs

Policy	Summary
Confidentiality / NDA	Client information, source code and credentials are confidential during and after employment
Acceptable use	Company devices and accounts for company work; no pirated software
Information security	Report suspected incidents within one hour; follow Section 9 escalation
Data protection	Process personal data only as the project requires; use synthetic data for testing
Intellectual property	Work product belongs to the company or the client as per contract; open-source licences are checked before use
Code of conduct	Respectful, honest, collaborative; no plagiarism; credit sources
Leave and handover	Document work-in-progress before leave (Section 9.6)

Mentor model

Item	Rule
Ratio	One mentor per two trainees at most
Meetings	15-minute daily check-in; 45-minute weekly review
Stuck rule	Try for 30 minutes alone, 2 hours with documentation and notes, then ask - with a written summary of what you tried
Evidence	Every module ends in an artefact in the training repository (code, notes, screenshots)
Sign-off	The mentor signs module gates; the Technical Lead signs level gates

The five onboarding gates

Gate	When	Criteria
Gate 1 - Access and security	End of week 1	All accounts and MFA active; policies signed; passes the security basics quiz (80%)
Gate 2 - Foundations	End of week 4	Levels 1-2 modules signed; Projects 1-2 accepted; may now commit to client repositories via pull request
Gate 3 - Backend and data	End of week 8	Levels 3-4 modules signed; Projects 3-4 accepted
Gate 4 - Professional practice	End of week 11	Level 5 modules signed; Projects 5-6 accepted; first code review given
Gate 5 - Delivery readiness	End of week 13	Project 7 (simulated client project) accepted; oral assessment passed; may join supervised client work

> **A LEVEL IS PASSED ONLY WHEN EVERY EXERCISE HAS BEEN DEMONSTRATED**
>
> Reading is not enough. For each module you must show the working result to your mentor, answer the assessment questions, and have the gate signed in your learning record.

First-week checklist (summary)

- [ ] Laptop encrypted, locked and updated
- [ ] Company e-mail, GitHub, project tracker, password manager - all with MFA
- [ ] NDA and policies signed
- [ ] Git, VS Code, Node.js, Python, Docker installed per the README
- [ ] SSH key created and added to GitHub
- [ ] First pull request merged into the training repository
- [ ] Mentor assigned; 30/60/90-day goals written and signed

The complete checklist with owners and dates is **Appendix A1**.

PART A · FOUNDATIONS

3. Learning Roadmap: Levels 1-5

For **every** topic the trainee receives the same 15-step pack (definition to approval gate) - see Section "How every topic is taught". Modules are expandable; each contains working code, expected output and an assessment. A level is passed **only when every module's exercise has been demonstrated** to the mentor and the Technical Lead has signed the level gate.

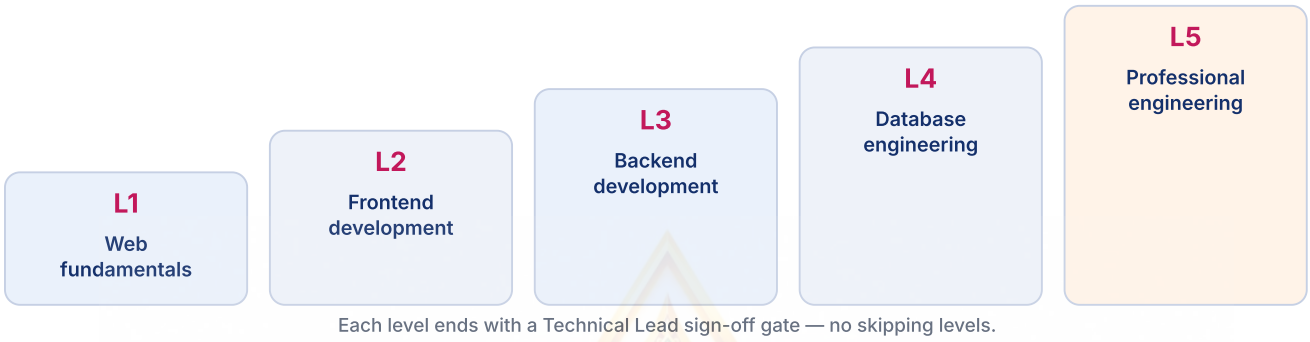


Figure 3.1 - The five-level learning roadmap

Level	Theme	Modules	Approx. time	Capstone / gate
1	Web fundamentals	9	2 weeks	Quiz (80%) + whiteboard of the request path
2	Frontend development	9	5 weeks	Projects 1 and 2 accepted
3	Backend development	9	4 weeks	Project 3 (authentication app) accepted
4	Database engineering	8	3 weeks	Project 4 (CRUD + DB) with restore evidence
5	Professional engineering	11	4 weeks	Projects 5-7, security review, delivery readiness

STUDY RHYTHM

Daily: 2 hours reading + 4 hours hands-on + 30 minutes writing notes in the training repository.
Weekly: mentor review and one demonstration. Never copy-paste code you cannot explain line by line.

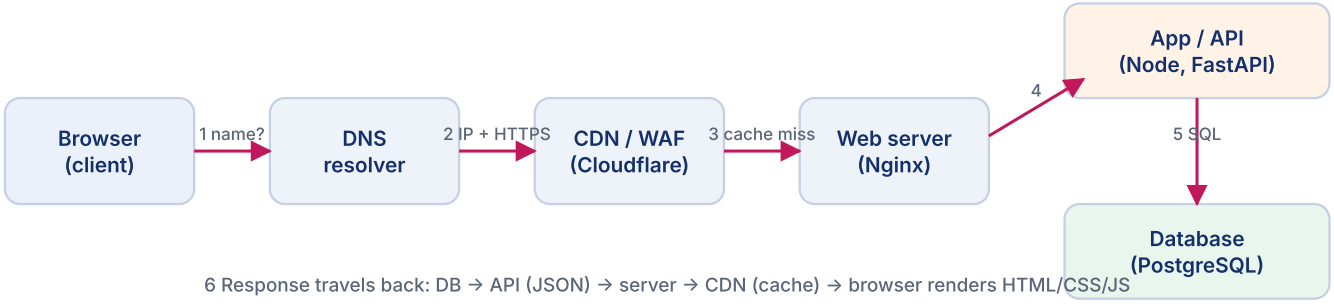


Figure 3.2 - Level 1 big picture: the path of one web request

Level 1 - Web Fundamentals (about 2 weeks)

Goal: explain how the web works end to end and name the part where any given problem lives.

▼ L1.1 · Internet and Web Architecture

Essential | Free | 3 h

1 · Definition	The **internet** is the global network of connected computers. The **web** is one service that runs on it - pages and apps delivered over HTTP. A **web architecture** is the set of parts (browser, DNS, CDN, server, application, database) that cooperate to answer one request.
2 · Business purpose	Every client problem ("the site is slow", "the form does not save") lives in one of these parts. An employee who can name the parts can find the fault in minutes instead of guessing for days.
3 · Real-world example	A customer opens a bakery website. The browser asks DNS for the address, the CDN returns cached images, the web server passes the order form to the application, and the application saves the order in the database.
4 · Prerequisites	Basic computer use; ability to open a browser and a terminal.
5 · Tools & installation	Chrome or Firefox (free), a terminal (Windows Terminal/WSL, macOS Terminal, Linux shell), draw.io or Excalidraw (free) for diagrams.

6 · STEP-BY-STEP PROCEDURE

1. Open any public website and open DevTools (F12), tab **Network**.
2. Reload the page and list every request - note the first one (the HTML document) and the later ones (CSS, JS, images, API calls).
3. Draw the path on paper - browser, DNS, CDN, server, app, database - and write one sentence per hop.
4. Label where each request could fail (DNS error, TLS error, 404, 500, database timeout).

7 · COMPLETE EXAMPLE

```text
Browser --DNS lookup--> DNS resolver  (name -> IP)
Browser --HTTPS-------> CDN / WAF     (cache, DDoS filter)
CDN     --HTTP--------> Web server    (Nginx)
Web server ----------> Application    (Node.js / FastAPI)
Application ---SQL---> Database       (PostgreSQL)
```

8 · EXPECTED OUTPUT

```output
A hand-drawn diagram with 6 boxes and arrows, plus a table:
Hop | What can fail | Symptom the client sees
DNS | wrong record | "Server not found"
TLS | expired certificate | browser security warning
App | unhandled exception | HTTP 500
```

9 · Verification	The Technical Lead can pick any hop and the employee explains what it does and one way it fails.
10 · Common errors	Treating "the website" as one thing; confusing the web (HTTP) with the internet (all traffic); forgetting that a CDN may serve a stale copy.

11 · Security considerations	Every hop is an attack surface. Only the web server/CDN should be public; the database must never be reachable from the internet.
12 · Documentation	Save the diagram as `docs/architecture.md` in the training repository.
13 · Practical assignment	Choose three real websites. For each, record in a table which hops you can identify from DevTools (CDN header, server header, API calls).
14 · Assessment	10-question quiz (pass 80%) plus a 5-minute whiteboard of the request path.
15 · Approval gate	Technical Lead signs Level 1 checklist item 1.

Read: MDN - How the web works

▼ L1.2 · Browser and Server

Essential | Free | 3 h

1 · Definition	A **browser** is a client program that requests, downloads and renders web resources. A **server** is a computer or program that listens for requests and returns responses.
2 · Business purpose	Knowing which side runs which code decides where bugs are fixed, where secrets may live and what the client can see.
3 · Real-world example	HTML/CSS/JS download to every visitor and run in the browser (public). Python or Node code runs only on our server (private) and may use secret keys.
4 · Prerequisites	L1.1.
5 · Tools & installation	Browser DevTools; Node.js LTS (free) to run a tiny server; VS Code.

6 · STEP-BY-STEP PROCEDURE

1. Install Node.js LTS and verify with `node -v`.
2. Create `server.js` with the example below.
3. Run `node server.js` and open `http://localhost:3000`.
4. In DevTools > Network, click the request and read the request and response headers.

7 · COMPLETE EXAMPLE

```js
// server.js - a server is just a program that listens on a port
const http = require('node:http');
http.createServer((req, res) => {
  console.log(req.method, req.url);
  res.writeHead(200, { 'Content-Type': 'text/html; charset=utf-8' });
  res.end('<h1>Hello from the server</h1>');
}).listen(3000, () => console.log('Listening on http://localhost:3000'));
```

8 · EXPECTED OUTPUT

```output
Terminal:  Listening on http://localhost:3000
Terminal:  GET /
Browser:   a page showing the heading "Hello from the server"
```

9 · Verification	Change the text, restart the server, reload the page; the new text appears. Stop the server and reload - the browser shows "connection refused".

10 · Common errors	Port already in use (EADDRINUSE); opening the file with `file://` instead of `http://`; forgetting to restart after editing.
11 · Security considerations	Never put API keys in browser-delivered code - anyone can read it via View Source or DevTools.
12 · Documentation	README section "Run locally" with the exact commands.
13 · Practical assignment	Extend the server to return different text for `/about` and a 404 for unknown paths.
14 · Assessment	Explain in 3 sentences why a payment secret key cannot live in JavaScript sent to the browser.
15 · Approval gate	Demo to a mentor; mentor signs the log.

▼ L1.3 · HTTP and HTTPS

Essential | Free | 4 h

1 · Definition	**HTTP** is the request/response protocol of the web: a method (GET, POST…), a path, headers and an optional body go out; a status code, headers and a body come back. **HTTPS** is HTTP inside a TLS-encrypted channel that also proves the server's identity with a certificate.
2 · Business purpose	Clients expect the padlock; search engines and browsers penalise plain HTTP; payment and login pages must be encrypted. Status codes are also the common language for debugging.
3 · Real-world example	A login form sends `POST /api/login`. The server answers `200` with a cookie, or `401` if the password is wrong.
4 · Prerequisites	L1.2.
5 · Tools & installation	`curl` (preinstalled on most systems), browser DevTools, Postman (free tier).

6 · STEP-BY-STEP PROCEDURE

1. Run `curl -I https://example.com` and read the status line and headers.
2. Run `curl -v https://example.com 2>&1 | head -30` and find the TLS handshake lines.
3. Request a path that does not exist and record the status code.
4. Memorise the status families - 2xx success, 3xx redirect, 4xx client error, 5xx server error.

7 · COMPLETE EXAMPLE

BASH

```
curl -I https://example.com                # headers only
curl -i https://example.com/missing-page   # headers + body of a 404
curl -X POST https://httpbin.org/post -H "Content-Type: application/json" -d
'{"name":"Asha"}'
```

8 · EXPECTED OUTPUT

OUTPUT

```
HTTP/2 200
content-type: text/html
cache-control: max-age=...
(exact headers vary by site and date)
```

9 · Verification	For five different URLs, state the status code before you run curl, then confirm.
10 · Common errors	Confusing 401 (not logged in) with 403 (logged in but not allowed); using GET to change data; ignoring redirect chains (301 > 302 > 200).
11 · Security considerations	Redirect all HTTP to HTTPS; enable HSTS only after HTTPS works everywhere; never send passwords in a URL query string.
12 · Documentation	Table of the status codes our APIs return, kept in `docs/api-specification.md`.
13 · Practical assignment	Make a one-page cheat sheet - 10 status codes, when we use each, example response body.
14 · Assessment	Quiz - choose the correct status for 12 scenarios (e.g. validation failed = 422/400, duplicate email = 409).
15 · Approval gate	Quiz pass (80%) recorded by the Technical Lead.

Read: MDN - HTTP

▼ L1.4 · DNS and Domains

Essential | Paid | 3 h

1 · Definition	**DNS** (Domain Name System) translates names such as `example.com` into IP addresses. A **domain** is a name registered through a registrar. Common records: **A** (IPv4), **AAAA** (IPv6), **CNAME** (alias), **MX** (mail), **TXT** (verification, SPF/DKIM).
2 · Business purpose	Wrong DNS is the most common cause of "the new site is not showing" and "emails go to spam". Domain ownership must also be in the client's name.
3 · Real-world example	We deploy the client site to `203.0.113.10`. We add an **A** record `www -> 203.0.113.10` at the registrar; within the TTL, visitors reach the site.
4 · Prerequisites	L1.3.
5 · Tools & installation	`dig` or `nslookup`; a registrar account (paid, usually yearly); Cloudflare DNS (free plan available - verify current plan terms).

6 · STEP-BY-STEP PROCEDURE

1. Run `dig example.com A +short` and `dig example.com MX +short`.
2. Find the nameservers with `dig example.com NS +short`.
3. In a sandbox domain (or Cloudflare test zone), add an A record and a CNAME and watch propagation with `dig`.
4. Lower the TTL to 300 seconds 24 hours before any planned cut-over.

7 · COMPLETE EXAMPLE

```bash
dig yourdomain.com A +short
dig www.yourdomain.com CNAME +short
dig yourdomain.com MX +short
nslookup yourdomain.com 1.1.1.1
```

8 · EXPECTED OUTPUT

```
203.0.113.10            <- A record (documentation IP - yours will differ)
yourdomain.com.         <- CNAME target
10 mail.provider.net.   <- MX priority + host
```
OUTPUT

9 · Verification	The record returned by two different public resolvers (1.1.1.1 and 8.8.8.8) matches what you configured.
10 · Common errors	Editing records at the wrong place (registrar vs. DNS host); CNAME on the root domain; forgetting `www`; expecting instant propagation.
11 · Security considerations	Enable registrar lock and 2FA; domain registered to the **client**, never an employee's personal account; keep a record of renewal dates.
12 · Documentation	DNS record table in `docs/deployment.md` - type, name, value, TTL, owner, date.
13 · Practical assignment	Write the DNS plan for a client migrating from shared hosting to a new server with zero email downtime.
14 · Assessment	Given five broken-symptom scenarios, name the record type to inspect.
15 · Approval gate	Mentor review of the DNS plan.

Read: MDN - What is a domain name

▼ L1.5 · Hosting

Essential | Paid | 4 h

1 · Definition	**Hosting** is the computer and network capacity that runs a website. Models: static/CDN hosting, shared hosting, VPS, managed platform (PaaS), containers, serverless, and dedicated cloud.
2 · Business purpose	The hosting choice sets cost, speed, security and maintenance effort for years. It must match the project, not the developer's habit.
3 · Real-world example	A 5-page brochure site goes on static hosting + CDN. A booking system with a database goes on a managed platform or a small cloud VM with automated backups.
4 · Prerequisites	L1.4.
5 · Tools & installation	Cloudflare Pages, Vercel, Netlify, AWS, Azure, GCP, DigitalOcean, a cPanel host. Free tiers exist but **limits and prices change - check the vendor's current pricing page**.

6 · STEP-BY-STEP PROCEDURE

1. Classify the project - static, dynamic, or stateful with a database.
2. List non-functional needs - traffic, uptime target, data location, budget, who maintains it.
3. Compare three hosts in a table (cost, backup, scaling, region, support) using each vendor's official pricing page and record the date.
4. Pick one and write the reason in an Architecture Decision Record.

7 · COMPLETE EXAMPLE

```text
ADR-001: Hosting choice
Context : 8-page site + contact form, ~2,000 visits/month, owner non-technical
Options : A static host + serverless form   B shared hosting   C VPS
Decision: A - lowest maintenance, automatic TLS, global CDN
Consequences: form handled by serverless function; no server patching by us
Verified pricing on: <date> from <vendor pricing URL>
```

8 · EXPECTED OUTPUT

```output
A one-page comparison table and a signed ADR stored in `docs/adr/`.
```

9 · Verification	Another employee can re-derive the same cost figure from the linked vendor page.
10 · Common errors	Choosing by price alone; ignoring bandwidth/egress charges; no backup on a VPS; quoting prices from memory.
11 · Security considerations	Least-privilege access; separate accounts per environment; automatic OS and runtime patching decided before go-live.
12 · Documentation	ADR + cost table in `docs/architecture.md`.
13 · Practical assignment	Produce hosting recommendations for three client types - dentist brochure site, online shop, SaaS dashboard.
14 · Assessment	Tech Lead questions the ADR for 10 minutes; employee defends the decision.
15 · Approval gate	ADR approved by Technical Lead.

▼ L1.6 · Frontend and Backend

Essential | Free | 3 h

1 · Definition	The **frontend** is everything the user sees and interacts with in the browser (HTML, CSS, JavaScript/TypeScript, React). The **backend** is the server-side code and services that apply business rules, talk to the database and protect secrets.
2 · Business purpose	Clear separation lets frontend and backend developers work in parallel, lets us change one without breaking the other, and keeps secrets safe.
3 · Real-world example	Frontend: a booking form validates the date for convenience. Backend: re-validates the date, checks the slot is free, charges the card, saves the booking.
4 · Prerequisites	L1.2, L1.3.
5 · Tools & installation	VS Code, Node.js LTS, a browser.

6 · STEP-BY-STEP PROCEDURE

1. Take a feature such as "add to cart".
2. List what the frontend does (button, UI state, call API).
3. List what the backend does (check stock, price, save, respond).
4. Mark every rule that must be enforced on the server even if the browser also checks it.

7 · COMPLETE EXAMPLE

```text
FEATURE: Apply discount code
Frontend : input box, "Apply" button, show new total, show error text
Backend  : find code, check expiry + usage limit + minimum order,
           recalculate total from DB prices (never trust browser total),
           store discount on the order, return updated total
Rule     : price and discount are ALWAYS computed on the server
```

TEXT

8 · EXPECTED OUTPUT

```
A two-column responsibility table for three features of your choice.
```

OUTPUT

9 · Verification	Mentor picks one rule; employee shows where it is enforced on the server.
10 · Common errors	Trusting browser-supplied prices or roles; duplicating business rules in both places and letting them drift.
11 · Security considerations	All validation is repeated on the server; frontend validation is only for user comfort.
12 · Documentation	Responsibility table in the feature's section of `docs/requirements.md`.
13 · Practical assignment	Split "user registration with email verification" into frontend/backend tasks with estimates.
14 · Assessment	Three scenario questions - where does this logic belong and why.
15 · Approval gate	Mentor sign-off.

▼ L1.7 · Databases (concepts)

Essential | Free | 3 h

1 · Definition	A **database** is organised, durable storage for application data. A **relational** database (PostgreSQL, MySQL) stores rows in tables linked by keys and is queried with SQL. Other types (document, key-value, search) exist for specific needs.
2 · Business purpose	Business data - customers, orders, bookings - is the client's most valuable asset. Wrong design is expensive to fix later; lost data is a business-ending event.
3 · Real-world example	A shop has `customers`, `orders` and `order_items` tables. One customer can have many orders; each order has many items.
4 · Prerequisites	L1.6.
5 · Tools & installation	SQLite (built in to Python, free) for first exercises; later PostgreSQL.

6 · STEP-BY-STEP PROCEDURE

1. Choose a business (clinic, school, shop) and list the "things" it tracks (entities).
2. For each entity list its attributes and a unique identifier.
3. Draw relationships (one-to-many, many-to-many).
4. Create the tables in SQLite and insert 5 sample rows each.

7 · COMPLETE EXAMPLE

```python
import sqlite3
db = sqlite3.connect(":memory:")
db.executescript("""
CREATE TABLE customers(id INTEGER PRIMARY KEY, name TEXT NOT NULL, email TEXT UNIQUE NOT
NULL);
CREATE TABLE orders(id INTEGER PRIMARY KEY, customer_id INTEGER NOT NULL REFERENCES
customers(id), total_minor INTEGER NOT NULL);
INSERT INTO customers(name,email) VALUES ('Asha','asha@example.com');
INSERT INTO orders(customer_id,total_minor) VALUES (1, 129900);
""")
print(db.execute("SELECT c.name, o.total_minor FROM orders o JOIN customers c ON
c.id=o.customer_id").fetchall())
```

PYTHON

8 · EXPECTED OUTPUT

```
[('Asha', 129900)]
```

OUTPUT

9 · Verification	Try inserting an order with `customer_id = 99` - the database rejects it (foreign key). Insert a duplicate email - rejected (UNIQUE).
10 · Common errors	Storing money as floating point (use integer minor units or DECIMAL); storing comma-separated lists in one column; no primary key.
11 · Security considerations	Never expose a database port to the internet; use a dedicated low-privilege application user; never string-concatenate SQL.
12 · Documentation	ER diagram in `docs/database-design.md` .
13 · Practical assignment	Model a clinic appointment system with at least five tables.
14 · Assessment	Whiteboard review - find three weaknesses in a deliberately flawed schema.
15 · Approval gate	Mentor approves the ER diagram.

▼ L1.8 · APIs

Essential Free 3 h

1 · Definition	An **API** (Application Programming Interface) is a defined way for one program to talk to another. A **REST API** exposes resources at URLs (`/api/products/12`) and uses HTTP methods - GET (read), POST (create), PUT/PATCH (update), DELETE (remove) - usually exchanging **JSON**.
2 · Business purpose	The frontend, mobile apps, payment providers and the client's other systems all connect through APIs. A clear contract lets teams build in parallel.
3 · Real-world example	The shop frontend calls `GET /api/products?category=shoes` and receives a JSON list it renders as cards.
4 · Prerequisites	L1.3, L1.6.
5 · Tools & installation	Postman (free tier) or `curl` ; the public JSONPlaceholder test API; browser DevTools.

6 · STEP-BY-STEP PROCEDURE

1. Call a public API with GET and read the JSON.
2. Call it with POST and a JSON body.
3. Record method, URL, headers, body, status and response in a Postman collection.
4. Write the contract for one new endpoint before any code exists.

7 · COMPLETE EXAMPLE

BASH

```
curl -s https://jsonplaceholder.typicode.com/posts/1
curl -s -X POST https://jsonplaceholder.typicode.com/posts \
  -H "Content-Type: application/json" \
  -d '{"title":"Hello","body":"First post","userId":1}'
```

8 · EXPECTED OUTPUT

OUTPUT

```
{
  "userId": 1,
  "id": 1,
  "title": "sunt aut facere ...",
  "body": "quia et suscipit ..."
}
(second call returns the created object with "id": 101)
```

9 · Verification	Your Postman collection runs end to end with no manual edits.
10 · Common errors	Missing `Content-Type` header; sending invalid JSON (trailing comma); treating a 200 with an error message as success.
11 · Security considerations	APIs need authentication, authorisation, rate limits and validation (OWASP API Security Top 10).
12 · Documentation	Endpoint table - method, path, auth, request, response, errors - in `docs/api-specification.md`.
13 · Practical assignment	Design (on paper) a REST API for a library - books, members, loans - with request/response examples.
14 · Assessment	Review a flawed API (verbs in URLs, wrong status codes) and list corrections.
15 · Approval gate	Tech Lead reviews API design.

Read: OWASP API Security Top 10

▼ L1.9 · Responsive Design (concepts)

Essential · Free · 3 h

1 · Definition	**Responsive design** makes one site work well on every screen size - phone, tablet, laptop, large monitor - using flexible layouts, flexible media and CSS media queries. The strategy is **mobile-first**: design for the smallest screen, then enhance.
2 · Business purpose	Most client traffic is on phones. A site that needs pinch-zoom loses customers and search ranking.
3 · Real-world example	A three-column feature grid collapses to one column on a 360 px phone, two on a tablet and three on a desktop.

4 · Prerequisites	L1.6.
5 · Tools & installation	Browser DevTools device toolbar, a real phone, Lighthouse (built into Chrome DevTools).

6 · STEP-BY-STEP PROCEDURE

1. Add `<meta name="viewport" content="width=device-width, initial-scale=1">` to every page.
2. Use relative units (`rem`, `%`, `fr`) and `max-width`, not fixed pixel widths.
3. Write base CSS for 360 px, then add `@media (min-width: 768px)` and `(min-width: 1024px)` enhancements.
4. Test at 360, 768, 1024 and 1440 px, then on a real phone.

7 · COMPLETE EXAMPLE

```css
.grid { display: grid; gap: 1rem; grid-template-columns: 1fr; }
@media (min-width: 768px)  { .grid { grid-template-columns: repeat(2, 1fr); } }
@media (min-width: 1024px) { .grid { grid-template-columns: repeat(3, 1fr); } }
img { max-width: 100%; height: auto; }
```

8 · EXPECTED OUTPUT

```output
1 column at 360 px, 2 columns at 800 px, 3 columns at 1200 px; no horizontal scrollbar at any width.
```

9 · Verification	DevTools responsive mode at the four widths plus Lighthouse "Best practices/Accessibility" with no layout errors.
10 · Common errors	Missing viewport meta tag; fixed-width elements causing horizontal scroll; tap targets smaller than about 44 px; text under 16 px on phones.
11 · Security considerations	Not directly security-related - but hidden mobile menus must remain keyboard accessible.
12 · Documentation	List the breakpoints chosen in `docs/architecture.md` (frontend section).
13 · Practical assignment	Convert a fixed 960 px desktop layout to responsive using only CSS.
14 · Assessment	Practical test - given a broken page, fix it at all four widths in 30 minutes.
15 · Approval gate	Level 1 completion review - all nine modules signed, quiz average at least 80%.

Read: web.dev - Learn Responsive Design

Level 2 - Frontend Development (about 5 weeks)

Goal: build responsive, accessible, fast user interfaces with HTML, CSS, JavaScript, TypeScript, React and Next.js.

L2.1 · HTML5

Essential | Free | 6 h

1 · Definition	**HTML** (HyperText Markup Language) describes the structure and meaning of a page using elements such as `<header>`, `<nav>`, `<main>`, `<h1>`, `<p>`, `<img>`, `<a>` and `<form>`. HTML5 added semantic elements, native media and form controls.
2 · Business purpose	Correct semantic HTML gives accessibility, SEO and maintainability for free. Clients lose search traffic when pages are built from anonymous `<div>` soup.
3 · Real-world example	A clinic homepage uses one `<h1>`, a `<nav>` with a list of links, `<main>` with sections, and a `<footer>` with address markup - screen readers and Google both understand it.
4 · Prerequisites	Level 1 complete.
5 · Tools & installation	VS Code with Live Server and Prettier extensions; Chrome; the W3C Nu HTML Checker (free, online).

6 · STEP-BY-STEP PROCEDURE

1. Create `index.html` with `<!doctype html>`, `lang`, charset and viewport meta tags.
2. Build the layout with `<header>`, `<nav>`, `<main>`, `<section>`, `<article>`, `<footer>`.
3. Add one `<h1>` and a logical heading order (h2 under h1, h3 under h2).
4. Add images with meaningful `alt`, a contact form with `<label for>` on every input.
5. Validate with the W3C checker and fix every error.

7 · COMPLETE EXAMPLE

HTML

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Sunrise Dental Clinic - Chennai</title>
  <meta name="description" content="Family dentistry with online booking.">
</head>
<body>
  <header><nav aria-label="Main"><a href="/">Home</a> <a href="/services">Services</a></nav>
</header>
  <main>
    <h1>Gentle dentistry for the whole family</h1>
    <form action="/api/contact" method="post">
      <label for="email">Email</label>
      <input id="email" name="email" type="email" required autocomplete="email">
      <button type="submit">Request a call</button>
    </form>
  </main>
  <footer><p>&copy; 2026 Sunrise Dental</p></footer>
</body>
</html>
```

8 · EXPECTED OUTPUT

OUTPUT

```
A page with a heading, a labelled email field and a button; zero errors in the W3C validator.
```

9 · Verification	Tab through the page with the keyboard - every link and control gets a visible focus; Lighthouse accessibility score at least 90.
10 · Common errors	Skipping heading levels for styling; `<div>` instead of `<button>`; missing `alt`; placeholder used instead of label; duplicate ids.
11 · Security considerations	Never build HTML by concatenating untrusted text (XSS). Add `rel="noopener noreferrer"` on `target="_blank"` links.
12 · Documentation	Page templates and component notes in `frontend/README.md`.
13 · Practical assignment	Build a one-page business site (header, hero, services list, contact form, footer) using only semantic HTML.
14 · Assessment	Mentor opens the page with CSS disabled - the content must still read in a sensible order.
15 · Approval gate	Zero validator errors; mentor sign-off.

Read: MDN - HTML · HTML Living Standard

▼ L2.2 · CSS3

Essential | Free | 8 h

1 · Definition	**CSS** (Cascading Style Sheets) controls presentation - colour, type, spacing, layout, animation. Key ideas are the **cascade** and **specificity**, the **box model** (content, padding, border, margin), **custom properties** (`--brand`) and **units** (`rem` , `%` , `vw`).
2 · Business purpose	Consistent, maintainable CSS makes a client's brand look right on every page and keeps later redesigns cheap.
3 · Real-world example	One `--brand` colour variable changes the whole site's buttons, links and focus rings in a single edit.
4 · Prerequisites	L2.1.
5 · Tools & installation	VS Code, DevTools Elements/Styles panel, Stylelint (optional).

6 · STEP-BY-STEP PROCEDURE

1. Add a reset (`box-sizing:border-box`) and define design tokens as CSS variables.
2. Style type scale and spacing with `rem` units.
3. Style components with simple class selectors (avoid deep nesting and `!important`).
4. Add states - `:hover` , `:focus-visible` , `:disabled` .
5. Inspect an element and read which rule wins and why.

7 · COMPLETE EXAMPLE

```css
:root { --brand:#14306b; --accent:#c2185b; --space:1rem; --radius:.5rem; }
*,*::before,*::after { box-sizing: border-box; }
body { margin:0; font:1rem/1.5 system-ui, sans-serif; color:#171c28; }
.btn { background:var(--brand); color:#fff; padding:.6rem 1.1rem; border:0; border-radius:var(--radius); }
.btn:hover { background:var(--accent); }
.btn:focus-visible { outline:3px solid var(--accent); outline-offset:2px; }
```

8 · EXPECTED OUTPUT

```
A navy button that turns magenta on hover and shows a visible outline on keyboard focus.
```
OUTPUT

9 · Verification	Change `--brand` once and confirm every component updates; contrast checker shows at least 4.5:1 for body text.
10 · Common errors	Over-specific selectors; magic pixel numbers; removing the focus outline; relying on `!important`.
11 · Security considerations	Avoid loading CSS from untrusted origins; do not inline user-supplied styles.
12 · Documentation	List the design tokens (colours, spacing, type) in `frontend/README.md`.
13 · Practical assignment	Style the L2.1 page to match a provided mock-up within a 2 px tolerance.
14 · Assessment	Timed task - make a given unstyled form look like a design and fix three CSS bugs.
15 · Approval gate	Mentor sign-off against the mock-up.

Read: MDN - CSS

▼ L2.3 · JavaScript

Essential Free 14 h

1 · Definition	**JavaScript** is the programming language of the browser (and, with Node.js, of servers). Core ideas are variables (`const` , `let`), functions, arrays and objects, the DOM, events, `fetch` and **asynchronous code** (`Promise` , `async/await`).
2 · Business purpose	JavaScript makes pages interactive - menus, forms, filters, carts - and is the foundation of React, Next.js and Node.
3 · Real-world example	A booking form fetches free slots from the API when the date changes and shows them without reloading the page.
4 · Prerequisites	L2.1, L2.2.
5 · Tools & installation	VS Code, Node.js LTS, browser console, ESLint + Prettier.

6 · STEP-BY-STEP PROCEDURE

1. Learn values, functions, arrays (`map` , `filter` , `reduce`) and objects.
2. Select elements and respond to events.
3. Fetch JSON from an API with `async/await` and handle errors.
4. Update the page safely using `textContent` (not `innerHTML` with user data).

7 · COMPLETE EXAMPLE

```JS
const list = document.querySelector('#posts');
async function load() {
  try {
    const res = await fetch('https://jsonplaceholder.typicode.com/posts?_limit=3');
    if (!res.ok) throw new Error(`HTTP ${res.status}`);
    const posts = await res.json();
    list.replaceChildren(...posts.map(p => {
      const li = document.createElement('li');
      li.textContent = p.title;          // textContent = safe against XSS
      return li;
    }));
  } catch (err) {
    list.textContent = 'Could not load posts. Please retry.';
    console.error(err);
  }
}
document.querySelector('#reload').addEventListener('click', load);
load();
```

8 · EXPECTED OUTPUT

```OUTPUT
Three post titles appear as list items; with the network disabled the friendly error message appears instead.
```

9 · Verification	Run ESLint with no errors; test the success and failure paths in DevTools (offline mode).
10 · Common errors	Forgetting `await`; mutating shared state; using `==` instead of `===`; not handling a failed fetch; blocking the page with a long loop.
11 · Security considerations	XSS - never insert untrusted data with `innerHTML`; never store tokens in `localStorage` for sensitive apps if httpOnly cookies are possible.
12 · Documentation	JSDoc comments on shared helpers; a short "how it works" note per feature.
13 · Practical assignment	Build a task list - add, complete, delete, filter, persist to `localStorage`.
14 · Assessment	Code review of a 60-line script with seeded bugs; find at least 5.
15 · Approval gate	Mentor code review passed; 20-question quiz at least 80%.

Read: MDN - JavaScript

▼ L2.4 · TypeScript

Essential Free 8 h

1 · Definition	**TypeScript** is JavaScript plus a static type system. Types are checked at build time and erased before the code runs, so the browser still executes plain JavaScript.
2 · Business purpose	Types catch whole classes of bugs before the client sees them, make refactoring safe and document the shape of API data. Our standard for all new projects.
3 · Real-world example	A function `price(order: Order)` refuses to compile if someone passes `{ total: '12' }` where a number is required.

4 · Prerequisites	L2.3.
5 · Tools & installation	Node.js LTS, `npm i -D typescript`, VS Code (built-in TS support), `tsc --noEmit`.

6 · STEP-BY-STEP PROCEDURE

1. Run `npx tsc --init`, then enable `"strict": true`.
2. Model API data with `interface` / `type`, unions and optional properties.
3. Avoid `any`; use `unknown` and narrow with checks.
4. Share types between frontend and backend or generate them from the API schema.

7 · COMPLETE EXAMPLE

```ts
type Role = 'admin' | 'staff' | 'customer';
interface User { id: number; email: string; role: Role; phone?: string }

function canRefund(user: User): boolean {
  return user.role === 'admin' || user.role === 'staff';
}

const u: User = { id: 1, email: 'asha@example.com', role: 'customer' };
console.log(canRefund(u));          // false
// canRefund({ id: 2, email: 'x', role: 'owner' })  -> compile error: 'owner' is not a Role
```

8 · EXPECTED OUTPUT

```output
`false` at runtime; the commented line fails with TS2322 when uncommented.
```

9 · Verification	`npx tsc --noEmit` reports zero errors in CI.
10 · Common errors	Silencing errors with `any` or `as`; disabling strict mode; types that do not match the real API response.
11 · Security considerations	Types do not validate runtime data - validate untrusted input at the boundary (e.g. with Zod/Pydantic) before trusting the types.
12 · Documentation	Document shared types in `docs/api-specification.md`.
13 · Practical assignment	Convert the L2.3 task list to TypeScript with strict mode and zero `any`.
14 · Assessment	Fix 10 compiler errors in a supplied project without using `any`.
15 · Approval gate	CI type-check passes; mentor review.

Read: TypeScript Handbook

▼ L2.5 · Responsive Layouts - Flexbox and Grid

Essential | Free | 8 h

1 · Definition	**Flexbox** lays items out along one axis (a row or a column) and distributes spare space. **CSS Grid** lays out in two dimensions - rows and columns - and is ideal for page and card layouts.
2 · Business purpose	These two systems replace floats and tables, and produce layouts that adapt to every screen with very little CSS.

3 · Real-world example	A header uses Flexbox (logo left, menu right). A product catalogue uses Grid with `auto-fit` so the number of columns adapts to screen width automatically.
4 · Prerequisites	L2.2, L1.9.
5 · Tools & installation	DevTools Flexbox/Grid overlays; Flexbox Froggy and Grid Garden (free games).

6 · STEP-BY-STEP PROCEDURE

1. Build a header with `display:flex; justify-content:space-between; align-items:center`.
2. Build a card grid with `repeat(auto-fit, minmax(240px, 1fr))`.
3. Add `gap` instead of margins between items.
4. Test at 360, 768, 1024 and 1440 px.

7 · COMPLETE EXAMPLE

```css
.site-header { display:flex; justify-content:space-between; align-items:center; gap:1rem; flex-wrap:wrap; }
.cards { display:grid; gap:1rem; grid-template-columns:repeat(auto-fit, minmax(240px, 1fr)); }
.card { padding:1rem; border:1px solid #d5dcea; border-radius:.75rem; }
```

8 · EXPECTED OUTPUT

```output
1 card per row at 360 px, 2-3 at tablet widths, 4 on wide screens, with no media query.
```

9 · Verification	No horizontal scroll at any width from 320 px to 1600 px.
10 · Common errors	Fixed heights that clip text; forgetting `flex-wrap`; nested grids too deep; using Grid for everything or Flexbox for everything.
11 · Security considerations	None specific.
12 · Documentation	Layout rules recorded in the frontend README (which pattern for which component).
13 · Practical assignment	Rebuild a 3-section landing page (hero, features, pricing, footer) that is fully responsive.
14 · Assessment	Reproduce a given responsive mock-up in 45 minutes.
15 · Approval gate	Mentor sign-off at four widths.

Read: MDN - CSS layout

▼ L2.6 · Accessibility

Essential Free 6 h

1 · Definition	**Accessibility (a11y)** means people with disabilities can perceive, operate and understand the site - keyboard-only users, screen-reader users, low-vision and colour-blind users. The standard is **WCAG 2.2** (target level AA).
2 · Business purpose	Accessibility is a quality requirement, widens the client's audience and, in many markets, a legal expectation. Fixing it late costs far more than building it in.

3 · Real-world example	A customer who cannot use a mouse can still open the menu, fill the booking form and pay, using only Tab, Enter and Space.
4 · Prerequisites	L2.1, L2.2.
5 · Tools & installation	Lighthouse, axe DevTools extension (free), NVDA (Windows, free) or VoiceOver (macOS), WebAIM contrast checker.

6 · STEP-BY-STEP PROCEDURE

1. Use semantic elements first; add ARIA only when HTML cannot express the meaning.
2. Make every control keyboard reachable with a visible `:focus-visible` style.
3. Give each image a text alternative (empty `alt=""` for decorative images).
4. Ensure text contrast of at least 4.5:1 and never use colour alone to convey meaning.
5. Associate error messages with inputs using `aria-describedby`.

7 · COMPLETE EXAMPLE

```HTML
<label for="phone">Phone</label>
<input id="phone" name="phone" inputmode="tel" aria-describedby="phone-err" aria-invalid="true">
<p id="phone-err" role="alert">Enter a 10-digit mobile number.</p>
<button type="button" aria-expanded="false" aria-controls="menu">Menu</button>
```

8 · EXPECTED OUTPUT

```OUTPUT
A screen reader announces "Phone, edit text, invalid entry, Enter a 10-digit mobile number."
```

9 · Verification	axe reports zero critical/serious issues; the whole flow works with the keyboard only; NVDA/VoiceOver read the form logically.
10 · Common errors	Click handlers on `<div>`; removing outlines; vague link text such as "click here"; images of text; autoplaying media.
11 · Security considerations	CAPTCHAs must have accessible alternatives; do not rely on hover for critical actions.
12 · Documentation	Accessibility statement and known issues in `docs/user-guide.md`.
13 · Practical assignment	Audit a sample site with axe and manual keyboard test; write a prioritised fix list.
14 · Assessment	Fix the audit's top 10 issues in a given codebase.
15 · Approval gate	Accessibility checklist in QA (Appendix A6) fully passed on your Level 2 project.

Read: web.dev - Learn Accessibility · MDN - Accessibility

▼ L2.7 · Browser Developer Tools

Essential | Free | 4 h

1 · Definition	**DevTools** are the debugging suite built into the browser - Elements (DOM/CSS), Console, Network, Sources (breakpoints), Application (storage, cookies), Performance and Lighthouse.

2 · Business purpose	DevTools is the fastest way to reproduce and explain a bug to a client or colleague, and the main instrument for performance and accessibility checks.
3 · Real-world example	A client says "the page is slow". The Network tab shows a 4 MB uncompressed hero image; compressing it to WebP cuts load time by seconds.
4 · Prerequisites	L2.1 to L2.3.
5 · Tools & installation	Chrome or Edge DevTools (F12), Firefox DevTools.

6 · STEP-BY-STEP PROCEDURE

1. Elements - change CSS live and copy the working rule back to your code.
2. Console - read errors; run small experiments.
3. Network - throttle to "Fast 4G", disable cache, sort by size and time.
4. Sources - set a breakpoint, step through a function.
5. Application - inspect cookies and `localStorage`.
6. Lighthouse - run a mobile audit and read the opportunities.

7 · COMPLETE EXAMPLE

```js
// Paste into the Console to find images that are too large for their display size
[...document.images]
  .filter(i => i.naturalWidth > i.clientWidth * 2)
  .map(i => ({ src: i.currentSrc, natural: i.naturalWidth, shown: i.clientWidth }));
```

8 · EXPECTED OUTPUT

```
An array of image objects (empty if all images are sized well).
```

9 · Verification	You can reproduce one bug using only DevTools and record a screenshot of the failing request or console error.
10 · Common errors	Testing with cache enabled; testing only on a fast desktop connection; editing in DevTools and forgetting to copy the change back.
11 · Security considerations	Do not paste unknown scripts into the console (self-XSS). Never share screenshots or HAR files that contain tokens or cookies.
12 · Documentation	A "debugging notes" file in the training repo showing three bugs and how DevTools found them.
13 · Practical assignment	Take a slow public page you do not own (read-only analysis) and write a 1-page performance finding with screenshots.
14 · Assessment	Mentor plants three bugs - find them using only DevTools in 20 minutes.
15 · Approval gate	Mentor sign-off.

Read: Chrome DevTools docs · web.dev - Performance

L2.8 · React

Essential | Free | 24 h

1 · Definition	**React** is a JavaScript library for building user interfaces from reusable **components**. A component is a function that takes **props** and returns UI; **state** (`useState`) holds data that changes; **hooks** (`useEffect` , custom hooks) share behaviour.
2 · Business purpose	React lets a team build large interactive applications - dashboards, portals, shops - from small, tested, reusable parts, and has the largest ecosystem and hiring pool.
3 · Real-world example	A `<ProductCard>` component is reused in the catalogue, search results and the "recently viewed" strip; fix its layout once and every place updates.
4 · Prerequisites	L2.3, L2.4.
5 · Tools & installation	Node.js LTS, Vite (`npm create vite@latest`), React DevTools extension, Vitest or Jest, React Testing Library.

6 · STEP-BY-STEP PROCEDURE

1. Scaffold with Vite using the React + TypeScript template.
2. Create a typed component with props and local state.
3. Fetch data in an effect or a data library and render loading, error and empty states.
4. Lift state up instead of duplicating it; keep components small.
5. Write a test with React Testing Library.

7 · COMPLETE EXAMPLE

```tsx
import { useState } from 'react';

type Props = { name: string; price: number };

export function ProductCard({ name, price }: Props) {
  const [qty, setQty] = useState(1);
  return (
    <article className="card">
      <h2>{name}</h2>
      <p>Rs {(price * qty).toLocaleString('en-IN')}</p>
      <button onClick={() => setQty(q => q + 1)} aria-label={`Add one more ${name}`}>+
</button>
      <span aria-live="polite">Qty {qty}</span>
    </article>
  );
}
```

8 · EXPECTED OUTPUT

```output
A card showing the product name and a total that updates when "+" is pressed; the quantity
change is announced to screen readers.
```

9 · Verification	`npm run build` succeeds; component test passes; React DevTools shows the expected props and state.
10 · Common errors	Mutating state directly; missing `key` in lists; effects with missing dependencies; fetching in a loop; giant components.

11 · Security considerations	React escapes text by default; never use `dangerouslySetInnerHTML` with untrusted content; keep secrets out of the client bundle (only `VITE_` / `NEXT_PUBLIC_` variables are public).
12 · Documentation	Component catalogue (name, props, usage) in `frontend/README.md`; use Storybook where the project justifies it.
13 · Practical assignment	Build a product catalogue with search, filter, cart state and a responsive layout.
14 · Assessment	Code review against the delivery standard (Section 7); mentor adds three change requests to implement in 2 hours.
15 · Approval gate	Passed review; tests green; Lighthouse accessibility at least 90.

Read: React - Learn

▼ L2.9 · Next.js (and when to choose alternatives)

Essential | Free | 16 h

1 · Definition	**Next.js** is a React framework that adds file-based routing, server rendering (SSR), static generation (SSG), API/route handlers, image optimisation and deployment tooling. Alternatives - **Astro** (content sites), **Remix/React Router**, **Vite + React SPA**, **Nuxt/SvelteKit** (other ecosystems), **WordPress** (CMS-led sites).
2 · Business purpose	SEO-critical, fast, full-stack React apps need server rendering and good defaults. Choosing the framework per project (not by habit) saves cost and avoids over-engineering.
3 · Real-world example	A marketing site with a blog uses Astro or Next.js static export on a CDN. A logged-in customer portal uses Next.js with server components and an API.
4 · Prerequisites	L2.8.
5 · Tools & installation	Node.js LTS, `npx create-next-app@latest`, Vercel / Cloudflare / Docker for deployment.

6 · STEP-BY-STEP PROCEDURE

1. Create a project and read the generated folder structure (`app/`, `public/`).
2. Add a route (`app/services/page.tsx`) and a shared layout.
3. Fetch data on the server and render it; add `metadata` for SEO.
4. Add a route handler for a form endpoint and validate input.
5. Run `npm run build` and `npm start` to test the production build.

7 · COMPLETE EXAMPLE

```tsx
// app/services/page.tsx  (server component by default)
export const metadata = { title: 'Services - Sunrise Dental', description: 'Check-ups,
whitening, implants.' };

async function getServices() {
  const res = await fetch(`${process.env.API_URL}/services`, { next: { revalidate: 300 } });
  if (!res.ok) throw new Error('Failed to load services');
  return res.json() as Promise<{ id: number; name: string }[]>;
}

export default async function ServicesPage() {
  const services = await getServices();
  return <ul>{services.map(s => <li key={s.id}>{s.name}</li>)}</ul>;
}
```

8 · EXPECTED OUTPUT

```output
`/services` returns fully rendered HTML (View Source shows the list) and refreshes its cache
every 5 minutes.
```

9 · Verification	Lighthouse SEO at least 90; View Source contains the content; `npm run build` has no errors.
10 · Common errors	Putting secrets in `NEXT_PUBLIC_` variables; mixing server and client code; fetching the same data repeatedly; skipping `loading` / `error` states.
11 · Security considerations	Validate every route handler input; add security headers; keep server-only code out of client components; protect authenticated routes on the server.
12 · Documentation	Rendering strategy per page (static, server, client) in `docs/architecture.md`.
13 · Practical assignment	Build the multi-page corporate website (Project 2) with SEO metadata, sitemap and optimised images.
14 · Assessment	Explain, for five pages, which rendering mode you chose and why.
15 · Approval gate	Level 2 completion review - modules signed, Project 1 and 2 accepted.

Read: Next.js docs · Astro docs

Level 3 - Backend Development (about 4 weeks)

Goal: build secure, tested, documented APIs with authentication, authorisation, validation and business logic.

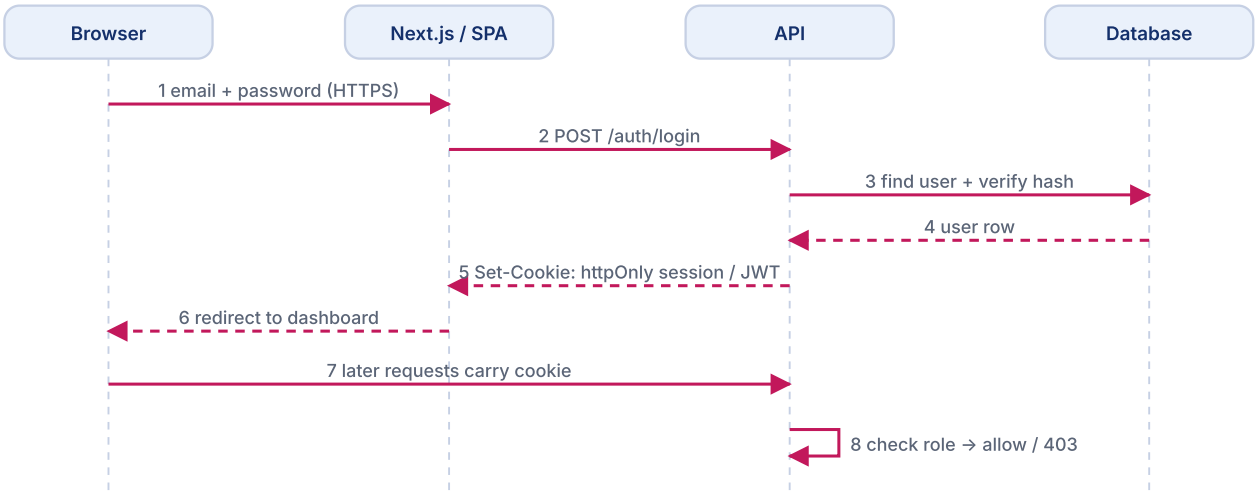


Figure 3.3 - Authentication sequence used throughout Level 3

▼ L3.1 · Backend Framework and Your First API

Essential | Free | 12 h

1 · Definition	A **backend framework** gives routing, request parsing, validation, dependency handling and error responses so we do not rebuild them. Our primary options are **FastAPI** (Python, typed, auto-documented), **Django** (Python, batteries-included with ORM and admin), and **Node.js** with **Express/Fastify/NestJS**.
2 · Business purpose	The framework sets how fast the team builds, how easy the code is to test and how well the API is documented. We pick per project - FastAPI for API-first products, Django when an admin panel and ORM save time, Node when the team is TypeScript end-to-end.
3 · Real-world example	A booking platform uses FastAPI for the API (automatic OpenAPI docs the frontend team reads) and Next.js for the UI.
4 · Prerequisites	Level 2 modules L2.3 and L2.4; Python 3.11+ or Node LTS installed.
5 · Tools & installation	Python 3.11+, `pip` / `venv` (or `uv`), `fastapi`, `uvicorn`, `pytest`, `httpx`; Postman. Django alternative - `django`, `djangorestframework`. Node alternative - Fastify or NestJS.

6 · STEP-BY-STEP PROCEDURE

1. Create a virtual environment and install `fastapi` and `uvicorn[standard]`.
2. Write `main.py` with a health endpoint and a create/read endpoint.
3. Run `uvicorn main:app --reload` and open `/docs` (Swagger UI).
4. Write a pytest using `TestClient`.

7 · COMPLETE EXAMPLE

PYTHON

```python
# main.py - run with: uvicorn main:app --reload
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel, Field

app = FastAPI(title="Clinic API", version="0.1.0")

class ServiceIn(BaseModel):
    name: str = Field(min_length=2, max_length=80)
    price_minor: int = Field(ge=0, description="Price in paise/cents")

class Service(ServiceIn):
    id: int

_db: dict[int, Service] = {}

@app.get("/health")
def health() -> dict[str, str]:
    return {"status": "ok"}

@app.post("/api/services", response_model=Service, status_code=201)
def create_service(payload: ServiceIn) -> Service:
    sid = len(_db) + 1
    _db[sid] = Service(id=sid, **payload.model_dump())
    return _db[sid]

@app.get("/api/services/{sid}", response_model=Service)
def get_service(sid: int) -> Service:
    if sid not in _db:
        raise HTTPException(status_code=404, detail="Service not found")
    return _db[sid]
```

8 · EXPECTED OUTPUT

OUTPUT

```
POST /api/services {"name":"Whitening","price_minor":450000}
  -> 201 {"name":"Whitening","price_minor":450000,"id":1}
GET  /api/services/9 -> 404 {"detail":"Service not found"}
POST with name "A"   -> 422 (validation error list)
```

9 · Verification	`pytest -q` passes; `/docs` lists every endpoint with examples.
10 · Common errors	Business logic inside route functions; global mutable state in production; blocking calls inside `async def`; no versioning of the API.
11 · Security considerations	Disable debug/reload in production; restrict CORS to known origins; never return stack traces to clients.
12 · Documentation	The generated OpenAPI file is exported into `docs/api-specification.md` at each release.
13 · Practical assignment	Re-implement the same two endpoints in a second framework (Django REST or Fastify) and write a one-page comparison.
14 · Assessment	Explain which framework you would pick for three client scenarios and why.
15 · Approval gate	Tech Lead reviews the code and comparison.

Read: FastAPI · Django · Node.js docs

L3.2 · REST API Design

Essential | Free | 8 h

1 · Definition	A **REST API** models business objects as **resources** with predictable URLs, standard HTTP methods and consistent JSON. Good design covers naming, status codes, pagination, filtering, versioning and idempotency.
2 · Business purpose	A predictable API cuts integration time for the frontend, mobile apps and the client's other systems, and keeps maintenance cost low.
3 · Real-world example	`GET /api/v1/orders?status=paid&page=2&page_size=20` lists orders; `POST /api/v1/orders` creates one; `PATCH /api/v1/orders/17` changes it; `DELETE` cancels it.
4 · Prerequisites	L1.8, L3.1.
5 · Tools & installation	Postman, OpenAPI/Swagger, FastAPI `/docs`.

6 · STEP-BY-STEP PROCEDURE

1. List resources as plural nouns (`/orders`, `/orders/{id}/items`).
2. Map operations to methods and status codes (201 create, 204 delete, 404 missing, 409 conflict, 422 validation).
3. Add pagination (`page`, `page_size` with a maximum) and filtering on every list endpoint.
4. Add an `Idempotency-Key` header for payment-like POST requests.
5. Version the path (`/api/v1`).

7 · COMPLETE EXAMPLE

```text
GET    /api/v1/orders                200  list (paginated)
POST   /api/v1/orders                201  create  (Idempotency-Key supported)
GET    /api/v1/orders/{id}           200 | 404
PATCH  /api/v1/orders/{id}           200 | 404 | 409 | 422
DELETE /api/v1/orders/{id}           204 | 404

List response:
{ "items": [ ... ], "page": 2, "page_size": 20, "total": 143 }
```

8 · EXPECTED OUTPUT

```output
A table of endpoints with request/response examples and error cases for one module of the sample project.
```

9 · Verification	A teammate can build the frontend screen from the written spec alone, without reading the backend code.
10 · Common errors	Verbs in URLs (`/getOrders`); returning 200 for errors; unbounded list endpoints; inconsistent field names (`userId` vs `user_id`).
11 · Security considerations	Rate-limit auth and expensive endpoints; check ownership on every `{id}` route (prevents IDOR); never expose internal ids/fields you do not need.

12 · Documentation	Complete the endpoint table in `docs/api-specification.md` before coding.
13 · Practical assignment	Write the API specification for the booking system (services, slots, bookings, payments).
14 · Assessment	Peer review another employee's spec using the standard checklist.
15 · Approval gate	Spec approved before implementation (this is a mandatory project gate).

Read: OWASP API Security Top 10

▼ L3.3 · Authentication

Essential | Free | 12 h

1 · Definition	**Authentication** proves who the user is. Common approaches are server-side **sessions** in an httpOnly cookie, **JWT** access tokens with short expiry plus refresh tokens, and delegated **OAuth 2.0/OpenID Connect** sign-in (Google, Microsoft). Passwords are stored only as salted slow hashes (**Argon2id** or bcrypt).
2 · Business purpose	Login is the front door of every portal, dashboard and SaaS product. A mistake here leads directly to account takeover and data breach.
3 · Real-world example	A customer portal logs users in with email and password, issues a 15-minute access token and a rotating refresh token in httpOnly cookies, and supports "Sign in with Google" for staff.
4 · Prerequisites	L3.1, L3.2, L1.3.
5 · Tools & installation	`argon2-cffi` , `PyJWT` (Python); `bcrypt` / `jose` (Node); or a managed provider (Auth0, Clerk, Firebase Auth, Cognito, Keycloak) - paid or free by plan, verify current terms.

6 · STEP-BY-STEP PROCEDURE

1. Hash passwords with Argon2id; never log or return them.
2. Verify the password in constant time and return a generic error on failure.
3. Issue a short-lived token/session; set cookie flags `HttpOnly; Secure; SameSite=Lax`.
4. Add rate limiting, account lockout/backoff and a password-reset flow with single-use expiring tokens.
5. Offer MFA for admin accounts.

7 · COMPLETE EXAMPLE

```python
import os, datetime as dt
import jwt                                  # PyJWT
from argon2 import PasswordHasher
from argon2.exceptions import VerifyMismatchError

ph = PasswordHasher()                       # Argon2id defaults
SECRET = os.environ["JWT_SECRET"]           # from environment, never in Git

stored_hash = ph.hash("S3cure-pass!")

def login(password: str, user_id: int, role: str) -> str | None:
    try:
        ph.verify(stored_hash, password)
    except VerifyMismatchError:
        return None                         # same message for wrong user or password
    now = dt.datetime.now(dt.timezone.utc)
    claims = {"sub": str(user_id), "role": role, "iat": now, "exp": now +
dt.timedelta(minutes=15)}
    return jwt.encode(claims, SECRET, algorithm="HS256")

token = login("S3cure-pass!", 42, "customer")
print(jwt.decode(token, SECRET, algorithms=["HS256"])["role"])   # customer
```

8 · EXPECTED OUTPUT

```output
customer
(login with a wrong password returns None; an expired token raises jwt.ExpiredSignatureError)
```

9 · Verification	Tests prove: wrong password rejected, expired token rejected, tampered token rejected, rate limit triggers after N attempts.
10 · Common errors	Storing plain or fast-hash (MD5/SHA-1) passwords; secrets hard-coded; tokens that never expire; different error messages that reveal whether an email exists; `alg` not pinned.
11 · Security considerations	OWASP ASVS authentication controls; rotate secrets; invalidate sessions on password change; use HTTPS everywhere; protect against CSRF when using cookies.
12 · Documentation	Auth flow diagram and token lifetimes in `docs/security.md`.
13 · Practical assignment	Build register, login, logout, refresh and password-reset endpoints with tests (Project 3).
14 · Assessment	Security review - the Tech Lead attempts five common attacks (credential stuffing, token tampering, brute force, replay, CSRF) against your API.
15 · Approval gate	All five attacks fail; Tech Lead signs the security checklist.

Read: OWASP Cheat Sheets - Authentication · OWASP ASVS

L3.4 · Authorisation

Essential Free 6 h

1 · Definition	**Authorisation** decides what an authenticated user may do. Models - **RBAC** (roles such as admin/staff/customer), **ownership checks** (a user may edit only their own records) and **permissions/policies** for finer rules.
2 · Business purpose	Broken access control is the number-one web risk (OWASP Top 10). One missing check can let a customer read another customer's invoices.
3 · Real-world example	A customer calls `GET /orders/17`. The API loads the order and returns 404 unless `order.customer_id` equals the caller's id; staff may read any order but not delete it; only admin may delete.
4 · Prerequisites	L3.3.
5 · Tools & installation	Framework dependency/middleware system (FastAPI `Depends`, Django permissions, Nest guards); optional policy engines (Casbin, OPA).

6 · STEP-BY-STEP PROCEDURE

1. Write an access matrix - role by action by resource.
2. Implement authentication once as a dependency; implement role and ownership checks as reusable functions.
3. Deny by default; require an explicit allow.
4. Test the matrix - every cell has an automated test.

7 · COMPLETE EXAMPLE

PYTHON

```python
from fastapi import Depends, HTTPException

def require_role(*allowed: str):
    def checker(user=Depends(current_user)):          # current_user validates the token
        if user.role not in allowed:
            raise HTTPException(status_code=403, detail="Forbidden")
        return user
    return checker

@app.delete("/api/v1/orders/{oid}", status_code=204)
def delete_order(oid: int, user=Depends(require_role("admin"))):
    ...

@app.get("/api/v1/orders/{oid}")
def get_order(oid: int, user=Depends(current_user)):
    order = repo.get_order(oid)
    if order is None or (user.role == "customer" and order.customer_id != user.id):
        raise HTTPException(status_code=404)          # do not reveal that it exists
    return order
```

8 · EXPECTED OUTPUT

OUTPUT

```
customer reads own order      -> 200
customer reads another's      -> 404
staff deletes order           -> 403
admin deletes order           -> 204
```

9 · Verification	The access-matrix test suite passes and is run in CI.
10 · Common errors	Checking permissions only in the frontend; trusting a `role` field sent by the client; forgetting list endpoints (leaking other users' rows through filters).
11 · Security considerations	Enforce on the server, deny by default, log access-denied events, re-check on every request.
12 · Documentation	Access matrix table in `docs/security.md`.
13 · Practical assignment	Add roles and ownership rules to the Project 3 application and write the matrix tests.
14 · Assessment	Given a codebase with three planted IDOR bugs, find and fix them.
15 · Approval gate	Tech Lead review of the access matrix and tests.

Read: OWASP Top 10

▼ L3.5 · Business Logic and Service Layer

Essential | Free | 8 h

1 · Definition	**Business logic** is the client's rules turned into code - pricing, discounts, stock, availability, approvals. The **service layer** keeps those rules in plain, testable functions separate from HTTP routes and database queries.
2 · Business purpose	Rules change often (offers, tax, policies). When logic is isolated and tested, changes are quick and safe, and the client's money is calculated correctly.
3 · Real-world example	`create_booking()` checks the slot is free, applies the member discount, creates the booking and the payment intent in one transaction, and returns the result - routes only translate HTTP to and from this function.
4 · Prerequisites	L3.2, L4.1 (concepts).
5 · Tools & installation	Pytest/Jest unit tests; a clear folder structure (`routes` , `services` , `repositories`).

6 · STEP-BY-STEP PROCEDURE

1. Write the rule in plain language with examples.
2. Implement it as a pure function where possible (inputs in, result out).
3. Keep money in integer minor units and use `Decimal` for rate maths.
4. Write unit tests from the examples first, including edge cases (zero, negative, boundaries).
5. Call the service from the route; keep the route thin.

7 · COMPLETE EXAMPLE

PYTHON

```
from decimal import Decimal, ROUND_HALF_UP

def price_with_gst(net_minor: int, gst_percent: Decimal = Decimal("18")) -> dict[str, int]:
    """Return net, tax and gross in minor units (paise). Rounds half up per line."""
    tax = (Decimal(net_minor) * gst_percent / Decimal(100)).quantize(Decimal("1"),
rounding=ROUND_HALF_UP)
    return {"net": net_minor, "tax": int(tax), "gross": net_minor + int(tax)}

assert price_with_gst(100_000) == {"net": 100000, "tax": 18000, "gross": 118000}
assert price_with_gst(10_001)["tax"] == 1800   # 1800.18 rounds to 1800
```

8 · EXPECTED OUTPUT

```
Both assertions pass silently; a failing rule raises `AssertionError` with the mismatch.
```
OUTPUT

9 · Verification	Unit tests cover normal, boundary and invalid inputs; mutation of a rule makes a test fail.
10 · Common errors	Rules duplicated in frontend and backend; floating-point money; rounding differences between invoice and ledger; business logic hidden in SQL triggers nobody documents.
11 · Security considerations	Never trust prices or totals from the client; recompute on the server; log who changed rule configuration.
12 · Documentation	Business rules glossary with examples in `docs/requirements.md`.
13 · Practical assignment	Implement discount, coupon and tax rules for the e-commerce prototype with at least 25 unit tests.
14 · Assessment	Client-style change request - "Add a 10% loyalty discount above Rs 5,000 but not with coupons". Implement and test in 90 minutes.
15 · Approval gate	Tests green; Tech Lead reviews rule coverage.

▼ L3.6 · Input Validation

Essential Free 5 h

1 · Definition	**Validation** ensures every value entering the system has the right type, length, range and format before it is used. Validate at the edge (request models), then enforce again in the database with constraints.
2 · Business purpose	Most injection, corruption and crash bugs start with unvalidated input. Validation also gives users clear, fast error messages.
3 · Real-world example	A registration endpoint rejects an email without `@`, a 3-character password, a 5,000-character name and an unknown `role` value - each with a precise message.
4 · Prerequisites	L3.1.
5 · Tools & installation	Pydantic (Python), Zod/Valibot (TypeScript), Django forms/serializers, database CHECK/UNIQUE/NOT NULL.

6 · STEP-BY-STEP PROCEDURE

1. Define a request model for every endpoint (no raw dicts).
2. Set min/max lengths, ranges, patterns and enums; use allow-lists, not block-lists.
3. Normalise (trim, lowercase email) before validating uniqueness.
4. Reject unknown fields where mass-assignment is a risk.
5. Mirror critical rules as database constraints.

7 · COMPLETE EXAMPLE

```python
from pydantic import BaseModel, EmailStr, Field, ConfigDict
from typing import Literal

class RegisterIn(BaseModel):
    model_config = ConfigDict(extra="forbid", str_strip_whitespace=True)
    email: EmailStr
    password: str = Field(min_length=10, max_length=128)
    name: str = Field(min_length=2, max_length=80)
    role: Literal["customer"] = "customer"        # clients can never choose admin
```

8 · EXPECTED OUTPUT

```
422 Unprocessable Entity
{"detail":[{"loc":["body","password"],"msg":"String should have at least 10 characters",
...}]}
```

9 · Verification	A table-driven test of valid and invalid payloads passes, including very long strings and unexpected fields.
10 · Common errors	Validating only in the browser; accepting any JSON object and saving it; trusting file names or MIME types; regexes that cause catastrophic backtracking.
11 · Security considerations	Parameterised queries for SQL; output-encode on rendering; validate file uploads (L3.8); apply size limits at the proxy and framework.
12 · Documentation	Validation rules are the single source in the request models and appear in the API spec.
13 · Practical assignment	Write validation tests with at least 30 cases for the registration and booking endpoints.
14 · Assessment	Fuzz your own API with 20 malicious payloads (long strings, SQL fragments, unicode, nulls) and record the results.
15 · Approval gate	No 500 errors from any fuzz case; mentor sign-off.

Read: OWASP Input Validation Cheat Sheet

▼ L3.7 · Error Handling and Logging Contracts

Essential | Free | 5 h

1 · Definition	**Error handling** means every failure produces a safe, consistent response for the user and a detailed log record for the team. Distinguish **client errors** (4xx - fixable by the caller) from **server errors** (5xx - our fault).
2 · Business purpose	Clients judge a product by how it fails. Clear messages reduce support calls; correlation ids let us find the cause in minutes.
3 · Real-world example	A payment timeout returns `502` with `{error:{code:'PAYMENT_UNAVAILABLE', message:'Please try again', request_id:'a1b2'}}` while the log holds the full exception and the gateway response.
4 · Prerequisites	L3.1, L3.6.

5 · Tools & installation	Framework exception handlers, structured logger (`structlog` , `pino`), Sentry (free tier - check current limits).

6 · STEP-BY-STEP PROCEDURE

1. Define one error format used by every endpoint.
2. Add global handlers for validation, not-found, domain and unexpected errors.
3. Generate a request id per call, return it in the response header and put it in every log line.
4. Never leak internals - SQL, file paths, stack traces - to the client.
5. Add retries with limits only for safe, idempotent calls.

7 · COMPLETE EXAMPLE

PYTHON

```python
import logging, uuid
from fastapi import Request
from fastapi.responses import JSONResponse

log = logging.getLogger("api")

@app.middleware("http")
async def request_id(request: Request, call_next):
    request.state.rid = request.headers.get("x-request-id", uuid.uuid4().hex[:12])
    response = await call_next(request)
    response.headers["x-request-id"] = request.state.rid
    return response

@app.exception_handler(Exception)
async def unhandled(request: Request, exc: Exception):
    log.exception("unhandled error rid=%s path=%s", request.state.rid, request.url.path)
    return JSONResponse(status_code=500, content={"error": {"code": "INTERNAL", "message":
"Something went wrong.", "request_id": request.state.rid}})
```

8 · EXPECTED OUTPUT

OUTPUT

```
HTTP/1.1 500
x-request-id: 3f9a2c1b7d04
{"error":{"code":"INTERNAL","message":"Something went wrong.","request_id":"3f9a2c1b7d04"}}
```

9 · Verification	Force an error in a test; the response is safe and the log line carries the same request id.
10 · Common errors	Swallowing exceptions (`except: pass`); returning 200 with an error; logging secrets or full card/personal data; inconsistent error shapes.
11 · Security considerations	Error responses must not reveal stack traces, queries or user existence; mask personal data in logs.
12 · Documentation	Error code catalogue in `docs/api-specification.md` .
13 · Practical assignment	Add global error handling, request ids and structured logs to the Project 3 API.
14 · Assessment	Given an incident description and a request id, find the root cause from logs alone.
15 · Approval gate	Tech Lead review.

L3.8 · File Uploads

Essential Free 6 h

1 · Definition	A **file upload** lets users send files (images, PDFs, CSVs) to the server. Safe handling needs size limits, type verification by content, random storage names, private storage and, for large files, direct-to-storage uploads with **pre-signed URLs**.
2 · Business purpose	Upload features (profile pictures, KYC documents, invoices) are a frequent route for malware, storage abuse and data leaks.
3 · Real-world example	A user uploads a PDF invoice. The API checks size and real file type, stores it in a private bucket under a random key, saves only the key in the database and serves it later through a short-lived signed link.
4 · Prerequisites	L3.6.
5 · Tools & installation	FastAPI `UploadFile` / `python-multipart` ; `python-magic` or `filetype` ; S3-compatible object storage (AWS S3, Cloudflare R2, Azure Blob - check current pricing); optional antivirus scanning (ClamAV).

6 · STEP-BY-STEP PROCEDURE

1. Set a maximum size at the proxy (e.g. Nginx `client_max_body_size`) and in the app.
2. Allow-list extensions and verify the real content type from the file bytes.
3. Generate a random name; never use the client's file name in a path.
4. Store outside the web root or in a private bucket; scan if the data is untrusted.
5. Serve through authenticated, expiring links.

7 · COMPLETE EXAMPLE

PYTHON

```python
import secrets, pathlib
from fastapi import UploadFile, HTTPException

ALLOWED = {"image/jpeg": ".jpg", "image/png": ".png", "application/pdf": ".pdf"}
MAX_BYTES = 5 * 1024 * 1024
STORE = pathlib.Path("/var/app/uploads")        # private folder, not served by Nginx

async def save_upload(file: UploadFile) -> str:
    data = await file.read(MAX_BYTES + 1)
    if len(data) > MAX_BYTES:
        raise HTTPException(413, "File too large (max 5 MB)")
    kind = sniff_mime(data)                     # inspect bytes, e.g. with python-magic
    if kind not in ALLOWED:
        raise HTTPException(415, "Unsupported file type")
    key = secrets.token_hex(16) + ALLOWED[kind]
    (STORE / key).write_bytes(data)
    return key
```

8 · EXPECTED OUTPUT

OUTPUT

```
5.2 MB file -> 413 File too large
renamed .exe -> 415 Unsupported file type
valid PDF   -> 201 {"file_key":"9c1e...b7.pdf"}
```

9 · Verification	Tests upload an oversized file, a disguised executable, a path-traversal name (`../../x`) and a valid file; only the last succeeds.

10 · Common errors	Trusting `Content-Type` or the extension; saving with the original name; keeping uploads inside the public folder; reading huge files fully into memory.
11 · Security considerations	OWASP file-upload guidance - allow-list, random names, private storage, scanning, no execution rights on the upload folder, SVG treated as active content.
12 · Documentation	Allowed types, limits and storage location in `docs/security.md`.
13 · Practical assignment	Add avatar upload and invoice PDF upload with signed download links to Project 4.
14 · Assessment	Attack your own upload endpoint with five malicious files and report the results.
15 · Approval gate	All malicious uploads rejected; Tech Lead sign-off.

▼ L3.9 · Email Integration

Essential | Paid | 6 h

1 · Definition	**Transactional email** is automated mail triggered by an action - verification, receipt, password reset. It is sent through SMTP or a provider API (Amazon SES, SendGrid, Mailgun, Postmark, Resend, Microsoft 365). Deliverability depends on **SPF**, **DKIM** and **DMARC** DNS records.
2 · Business purpose	Password resets, booking confirmations and invoices must reach the inbox. Misconfigured domains send client emails to spam or let attackers spoof the client's brand.
3 · Real-world example	After booking, the system queues a confirmation email with an `.ics` calendar attachment; a worker sends it and records the provider's message id.
4 · Prerequisites	L3.5, L1.4.
5 · Tools & installation	Provider SDK/SMTP, a template engine (Jinja2, React Email, MJML), a background queue (Celery, RQ, BullMQ) and a local test inbox (Mailpit, free).

6 · STEP-BY-STEP PROCEDURE

1. Choose a provider and read its current pricing and limits on the official site.
2. Verify the sending domain; add the SPF, DKIM and DMARC records it provides.
3. Build HTML + plain-text templates with variables; never put secrets in links without expiry.
4. Send through a queue with retries; record status and provider message id.
5. Test locally with Mailpit before using the real provider.

7 · COMPLETE EXAMPLE

PYTHON

```
import smtplib
from email.message import EmailMessage

def send_reset(to: str, link: str) -> None:
    msg = EmailMessage()
    msg["From"], msg["To"], msg["Subject"] = "no-reply@clientdomain.com", to, "Reset your
password"
    msg.set_content(f"Use this link within 30 minutes:\n{link}\nIf you did not ask, ignore
this email.")
    with smtplib.SMTP("localhost", 1025) as s:     # Mailpit in development
        s.send_message(msg)
```

8 · EXPECTED OUTPUT

```
OUTPUT
The message appears in the Mailpit web UI at `http://localhost:8025`; in production the same
call goes to the provider with credentials from environment variables.
```

9 · Verification	An email to a Gmail test account passes SPF, DKIM and DMARC (view message headers: `Authentication-Results`).
10 · Common errors	Sending from an unverified domain; no plain-text part; long-lived reset links; sending synchronously inside the request; no unsubscribe on marketing mail.
11 · Security considerations	Reset tokens are random, single-use and expire; do not reveal whether an email is registered; protect provider API keys; follow anti-spam law (consent, unsubscribe).
12 · Documentation	Email types, templates, sender domains and DNS records in `docs/deployment.md`.
13 · Practical assignment	Implement verification, reset and booking-confirmation emails with a queue and tests.
14 · Assessment	Diagnose three failure cases (SPF fail, DKIM missing, link expired) from headers and logs.
15 · Approval gate	Level 3 completion review - modules signed; Project 3 authentication app accepted.

Read: FastAPI · OWASP

Level 4 - Database Engineering (about 3 weeks)

Goal: design, build, query, tune, migrate and back up relational databases.

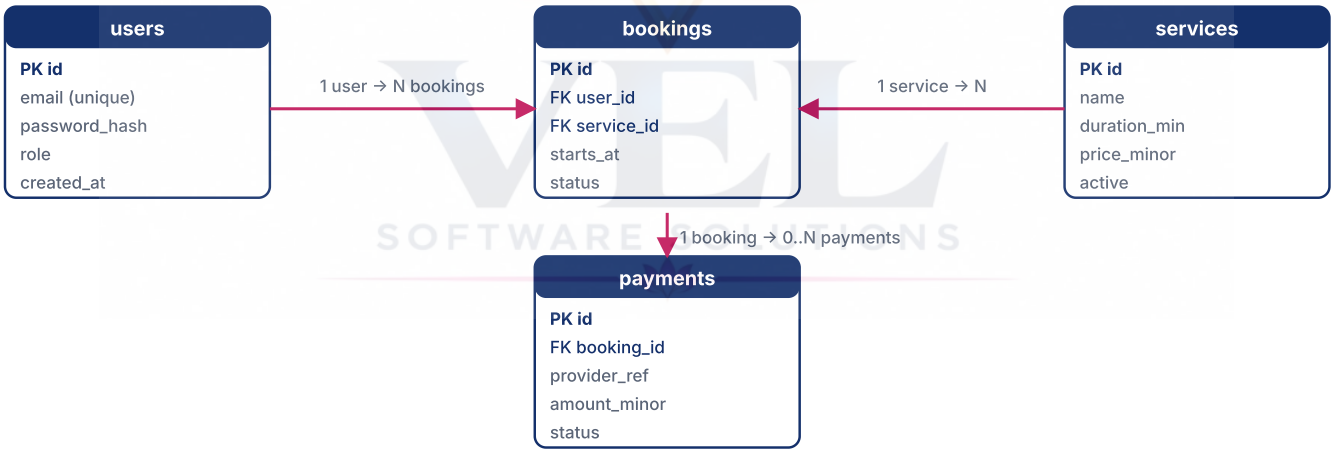


Figure 3.4 - Sample entity-relationship diagram for a booking system

▼ L4.1 · PostgreSQL and MySQL - Choosing and Installing

Essential | Free | 5 h

1 · Definition	**PostgreSQL** and **MySQL/MariaDB** are open-source relational database servers. PostgreSQL has the richest feature set (JSONB, advanced indexes, strong standards compliance, row-level security). MySQL/MariaDB is widely supported by hosting panels and WordPress/WooCommerce.

2 · Business purpose	The database engine is hard to change later. We default to PostgreSQL for custom applications and use MySQL/MariaDB when the platform requires it (WordPress, WooCommerce, some legacy systems).
3 · Real-world example	A custom SaaS platform uses PostgreSQL on a managed service. A WooCommerce shop keeps MySQL because the plugin ecosystem expects it.
4 · Prerequisites	L1.7.
5 · Tools & installation	PostgreSQL (free, open source), MySQL Community Edition / MariaDB (free), Docker, `psql`, DBeaver or pgAdmin (free). Managed services (AWS RDS, Azure Database, Cloud SQL, Supabase, Neon, PlanetScale) are paid or have limited free tiers - check each vendor's current pricing.

6 · STEP-BY-STEP PROCEDURE

1. Run PostgreSQL in Docker for development (command below).
2. Connect with `psql`, create a database and an application user with limited rights.
3. Repeat for MySQL/MariaDB and list three syntax differences you meet.
4. Write a one-page decision note - when PostgreSQL, when MySQL, when a managed service.

7 · COMPLETE EXAMPLE

```bash
docker run --name pg-dev -e POSTGRES_PASSWORD=devonly -p 5432:5432 -d postgres:16
psql "postgresql://postgres:devonly@localhost:5432/postgres" -c "CREATE DATABASE clinic;"
psql "postgresql://postgres:devonly@localhost:5432/clinic" <<'SQL'
CREATE ROLE app_user LOGIN PASSWORD 'change-me-dev';
GRANT CONNECT ON DATABASE clinic TO app_user;
SQL
```

8 · EXPECTED OUTPUT

```output
CREATE DATABASE
CREATE ROLE
GRANT
```

9 · Verification	`psql -c "SELECT version();"` prints the server version; `app_user` cannot create or drop databases.
10 · Common errors	Using the `postgres` superuser in the application; exposing port 5432 publicly; confusing `users` (application table) with database roles; default passwords left in place.
11 · Security considerations	Dedicated least-privilege application role; strong secrets from a secret store; TLS to the database; private network only; automated patching.
12 · Documentation	Database section of `docs/architecture.md` - engine, version, hosting, access method.
13 · Practical assignment	Install both engines locally and load the same 3-table schema into each.
14 · Assessment	Explain three real differences (types, auto-increment/identity, quoting, JSON handling).
15 · Approval gate	Mentor sign-off.

Read: PostgreSQL docs · MySQL reference

L4.2 · Database Design and Normalisation

Essential | Free | 8 h

1 · Definition	**Database design** turns business information into tables, columns, keys and constraints. **Normalisation** removes duplication - 1NF (atomic values), 2NF (no partial dependency), 3NF (no transitive dependency) - so each fact is stored once.
2 · Business purpose	A good design makes reports accurate, updates safe and growth cheap. A poor one produces duplicate customers, wrong totals and expensive rewrites.
3 · Real-world example	Instead of repeating the customer's address on every order row, store the address once in `customers` and reference it by `customer_id`.
4 · Prerequisites	L1.7, L4.1.
5 · Tools & installation	dbdiagram.io, draw.io, pgModeler or DBeaver ER view; SQL DDL.

6 · STEP-BY-STEP PROCEDURE

1. Interview the client and list nouns (entities) and verbs (relationships).
2. Choose primary keys; add natural unique keys (email, SKU) as UNIQUE constraints.
3. Normalise to 3NF; denormalise only with a measured reason.
4. Add NOT NULL, CHECK, UNIQUE and FOREIGN KEY constraints.
5. Add audit columns (`created_at`, `updated_at`) and soft-delete only where required.

7 · COMPLETE EXAMPLE

```sql
CREATE TABLE services (
  id           BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  name         VARCHAR(80)  NOT NULL UNIQUE,
  duration_min INTEGER      NOT NULL CHECK (duration_min BETWEEN 5 AND 480),
  price_minor  INTEGER      NOT NULL CHECK (price_minor >= 0),
  active       BOOLEAN      NOT NULL DEFAULT TRUE,
  created_at   TIMESTAMPTZ  NOT NULL DEFAULT now()
);
```

8 · EXPECTED OUTPUT

```output
`CREATE TABLE`. Inserting a negative price or a 0-minute service fails with a CHECK-constraint violation.
```

9 · Verification	Try to insert invalid rows - the database itself rejects them.
10 · Common errors	Money as `FLOAT`; no foreign keys "for speed"; one giant table; storing lists in a text column; EAV (entity-attribute-value) tables for everything.
11 · Security considerations	Store only data you need; classify personal data; plan encryption for sensitive columns and retention periods; never store card numbers (use the gateway's tokens).
12 · Documentation	ER diagram + data dictionary (table, column, type, meaning) in `docs/database-design.md`.
13 · Practical assignment	Design the schema for the booking system (Project 4) with at least 6 tables and a data dictionary.
14 · Assessment	Review a flawed schema and list 8 problems with fixes.

15 · Approval gate	Design review by Tech Lead - mandatory project gate before implementation.

▼ L4.3 · Relationships and Keys

Essential | Free | 5 h

1 · Definition	**Relationships** link tables: **one-to-many** (customer → orders), **many-to-many** via a **junction table** (students ↔ courses) and **one-to-one**. **Foreign keys** make the database enforce them; `ON DELETE` rules define what happens to children.
2 · Business purpose	Correct relationships keep data consistent without relying on every developer to remember a rule.
3 · Real-world example	`order_items(order_id, product_id, qty)` links orders to products; deleting an order removes its items (`ON DELETE CASCADE`), but deleting a product that was ever sold is blocked (`RESTRICT`).
4 · Prerequisites	L4.2.
5 · Tools & installation	psql, DBeaver, ER diagram tool.

6 · STEP-BY-STEP PROCEDURE

1. For each relationship decide cardinality and whether the child can exist without the parent.
2. Create foreign keys with an explicit `ON DELETE` rule.
3. Create a junction table with a composite primary key for many-to-many.
4. Index foreign-key columns used in joins.

7 · COMPLETE EXAMPLE

SQL

```
CREATE TABLE orders (
  id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  customer_id BIGINT NOT NULL REFERENCES customers(id) ON DELETE RESTRICT
);
CREATE TABLE order_items (
  order_id   BIGINT NOT NULL REFERENCES orders(id)   ON DELETE CASCADE,
  product_id BIGINT NOT NULL REFERENCES products(id) ON DELETE RESTRICT,
  qty        INTEGER NOT NULL CHECK (qty > 0),
  unit_price_minor INTEGER NOT NULL,
  PRIMARY KEY (order_id, product_id)
);
CREATE INDEX idx_orders_customer ON orders(customer_id);
```

8 · EXPECTED OUTPUT

OUTPUT

```
Deleting a customer with orders fails with a foreign-key error; deleting an order also
deletes its items.
```

9 · Verification	A script inserts orphan rows and attempts risky deletes - all are blocked or cascade as documented.
10 · Common errors	Missing index on the foreign key; CASCADE chains that delete too much; many-to-many modelled as a comma list.

11 · Security considerations	`ON DELETE CASCADE` can erase large volumes - restrict it to true ownership relations and require backups before bulk deletes.
12 · Documentation	Relationship table (parent, child, cardinality, delete rule) in `docs/database-design.md`.
13 · Practical assignment	Add junction tables and rules for tags, categories and product variants to the e-commerce schema.
14 · Assessment	Explain the effect of each `ON DELETE` option with a live demo.
15 · Approval gate	Mentor sign-off.

▼ L4.4 · SQL

Essential Free 14 h

1 · Definition	**SQL** (Structured Query Language) reads and changes relational data - `SELECT`, `INSERT`, `UPDATE`, `DELETE`, `JOIN`, `GROUP BY`, `HAVING`, subqueries and CTEs (`WITH`), and window functions.
2 · Business purpose	Every report, dashboard and data fix depends on SQL. Fluent SQL turns a client's vague question ("who are my best customers?") into an answer in minutes.
3 · Real-world example	Top customers by revenue in the last 90 days is one query joining customers, orders and order items.
4 · Prerequisites	L4.2, L4.3.
5 · Tools & installation	psql, DBeaver; sample dataset (pagila / the training schema).

6 · STEP-BY-STEP PROCEDURE

1. Practise SELECT with WHERE, ORDER BY, LIMIT.
2. Practise INNER and LEFT JOIN; explain the difference with row counts.
3. Aggregate with GROUP BY and HAVING.
4. Write a CTE and a window function (`RANK() OVER`).
5. Always use **parameters**, never string concatenation, from application code.

7 · COMPLETE EXAMPLE

```sql
WITH spend AS (
  SELECT c.id, c.name, SUM(oi.qty * oi.unit_price_minor) AS total_minor
  FROM customers c
  JOIN orders o       ON o.customer_id = c.id
  JOIN order_items oi ON oi.order_id   = o.id
  WHERE o.created_at >= now() - INTERVAL '90 days'
  GROUP BY c.id, c.name
)
SELECT name, total_minor, RANK() OVER (ORDER BY total_minor DESC) AS rnk
FROM spend
ORDER BY rnk
LIMIT 10;
```

8 · EXPECTED OUTPUT

OUTPUT

```
name    | total_minor | rnk
--------+-------------+-----
Asha    |      459000 |   1
Karthik |      312500 |   2
```

9 · Verification	Compare the query result to a manual total for one customer; run `EXPLAIN` and confirm no unexpected full scans on large tables.
10 · Common errors	`SELECT *` in production code; missing JOIN condition (cartesian product); `NOT IN` with NULLs; `UPDATE` or `DELETE` without `WHERE`.
11 · Security considerations	SQL injection is prevented only by parameterised queries; run `UPDATE` / `DELETE` first as `SELECT` to see the rows; use a transaction for data fixes.
12 · Documentation	Keep reusable reporting queries in `database/queries/` with a comment explaining intent.
13 · Practical assignment	Answer 20 business questions with SQL against the training database.
14 · Assessment	Timed SQL test (10 queries in 60 minutes).
15 · Approval gate	Score at least 80%.

Read: PostgreSQL tutorial

▼ L4.5 · Indexes and Query Performance

Essential Free 6 h

1 · Definition	An **index** is an extra data structure (usually a B-tree) that lets the database find rows without scanning the whole table. `EXPLAIN (ANALYZE)` shows the **query plan** and actual timings.
2 · Business purpose	Indexes are the cheapest performance win - and over-indexing slows writes and wastes storage. Slow queries are a top cause of timeouts under load.
3 · Real-world example	Searching orders by `customer_id` on a 5-million-row table drops from seconds to milliseconds after adding an index on that column.
4 · Prerequisites	L4.4.
5 · Tools & installation	`EXPLAIN (ANALYZE, BUFFERS)`, `pg_stat_statements`, pgAdmin/DBeaver.

6 · STEP-BY-STEP PROCEDURE

1. Generate a large test table (1 million rows) and run a filtered query.
2. Read the plan - note `Seq Scan` and its time.
3. Create an index on the filter column and re-run - note `Index Scan`.
4. Add a composite index for a common filter + sort and observe the plan.
5. Drop indexes that are never used.

7 · COMPLETE EXAMPLE

```sql
CREATE TABLE big_orders AS
SELECT g AS id, (random()*100000)::int AS customer_id, now() - (g || ' minutes')::interval AS
created_at
FROM generate_series(1, 1000000) g;

EXPLAIN (ANALYZE) SELECT * FROM big_orders WHERE customer_id = 4242;
CREATE INDEX idx_big_orders_customer ON big_orders (customer_id, created_at DESC);
ANALYZE big_orders;
EXPLAIN (ANALYZE) SELECT * FROM big_orders WHERE customer_id = 4242 ORDER BY created_at DESC
LIMIT 5;
```

8 · EXPECTED OUTPUT

```output
Before: Seq Scan on big_orders ... Execution Time: ~60-150 ms (hardware dependent)
After : Index Scan using idx_big_orders_customer ... Execution Time: < 1 ms
```

9 · Verification	The plan changes from Seq Scan to Index Scan and execution time drops by an order of magnitude.
10 · Common errors	Indexing every column; function on the indexed column in WHERE (`lower(email)` needs an expression index); stale statistics; leading wildcard `LIKE '%x'`.
11 · Security considerations	Indexes copy data - they inherit the sensitivity of the column; do not index unneeded personal data.
12 · Documentation	Record each non-obvious index and the query it serves in `docs/database-design.md`.
13 · Practical assignment	Tune three slow queries provided by the mentor and write before/after evidence.
14 · Assessment	Explain a given query plan line by line.
15 · Approval gate	Evidence reviewed by Tech Lead.

▼ L4.6 · Transactions and Concurrency

Essential | Free | 6 h

1 · Definition	A **transaction** groups statements so they all succeed or all fail (**ACID** - atomicity, consistency, isolation, durability). Isolation levels and row locks control what concurrent users see and prevent double-booking.
2 · Business purpose	Payments, stock and bookings must never end half-done or be sold twice. Transactions are how the database guarantees that.
3 · Real-world example	Booking the last 10:00 slot: two customers click at the same moment. A transaction with `SELECT ... FOR UPDATE` lets only one succeed.
4 · Prerequisites	L4.4.
5 · Tools & installation	psql with two terminals; the application's DB driver/ORM transaction API.

6 · STEP-BY-STEP PROCEDURE

1. Run `BEGIN; ... ROLLBACK;` and `COMMIT;` in psql and observe visibility from a second session.

2. Reproduce a double-booking race with two sessions.
3. Fix it with `FOR UPDATE` or a UNIQUE constraint on (slot, resource).
4. Keep transactions short; never wait for user input or an external API inside one.

7 · COMPLETE EXAMPLE

```sql
BEGIN;
SELECT id FROM slots WHERE id = 77 AND booked = FALSE FOR UPDATE;   -- locks the row
UPDATE slots SET booked = TRUE WHERE id = 77;
INSERT INTO bookings (slot_id, user_id) VALUES (77, 42);
COMMIT;
-- Safety net: ALTER TABLE bookings ADD CONSTRAINT uq_slot UNIQUE (slot_id);
```

8 · EXPECTED OUTPUT

```output
Two sessions run concurrently: one commits, the other either waits and finds `booked = TRUE`,
or fails with a unique violation - never both succeed.
```

9 · Verification	A concurrency test with 20 parallel requests creates exactly one booking.
10 · Common errors	Auto-commit per statement for multi-step operations; long transactions holding locks; deadlocks from inconsistent lock order; swallowing the error after a failed statement.
11 · Security considerations	Never leave a transaction open in a failing connection pool; log failed financial transactions for audit.
12 · Documentation	List which operations are transactional in `docs/architecture.md`.
13 · Practical assignment	Implement and test race-safe booking and stock decrement.
14 · Assessment	Explain READ COMMITTED vs REPEATABLE READ with an example each.
15 · Approval gate	Concurrency tests green.

▼ L4.7 · Migrations and Seed Data

Essential · Free · 6 h

1 · Definition	A **migration** is a versioned, reviewable script that changes the database structure from one state to the next (Alembic, Django migrations, Prisma Migrate, Flyway, Knex). **Seeds** load reference or demo data.
2 · Business purpose	Migrations make every environment identical and reproducible, and let us deploy schema changes safely and roll them back.
3 · Real-world example	Release 1.3 adds a `phone` column - the migration runs in staging first, then in production, and the same file is in Git history.
4 · Prerequisites	L4.2, L5.1 (Git).
5 · Tools & installation	Alembic (SQLAlchemy), Django migrations, Prisma Migrate or Flyway; `database/migrations/` and `database/seeds/` folders.

6 · STEP-BY-STEP PROCEDURE

1. Never edit the schema by hand in staging or production.

2. Generate a migration, read it, and test `upgrade` and `downgrade` on a copy of production-like data.
3. Use the expand-contract pattern for risky changes (add nullable column, backfill, switch code, then add NOT NULL).
4. Run migrations in CI and as a controlled deploy step.
5. Keep seeds idempotent (safe to run twice) and free of real personal data.

7 · COMPLETE EXAMPLE

```bash
alembic revision --autogenerate -m "add phone to customers"
alembic upgrade head          # apply
alembic downgrade -1          # prove the rollback works
alembic current               # show the version in this environment
```

8 · EXPECTED OUTPUT

```output
INFO  Running upgrade 7a1c... -> b92e..., add phone to customers
INFO  Running downgrade b92e... -> 7a1c..., add phone to customers
```

9 · Verification	A fresh database built only from migrations equals the one in staging (schema diff is empty).
10 · Common errors	Editing an already-applied migration; migrations that lock a big table for minutes; data changes mixed with schema changes; no downgrade plan.
11 · Security considerations	Migration credentials differ from application credentials; seeds contain no real customers or passwords.
12 · Documentation	Migration policy and the release runbook in `docs/deployment.md`.
13 · Practical assignment	Create five migrations for Project 4, including one using expand-contract.
14 · Assessment	Plan the safest way to rename a column on a live table.
15 · Approval gate	Tech Lead reviews migrations.

▼ L4.8 · Backup and Recovery

Essential · Free · 6 h

1 · Definition	A **backup** is a restorable copy of data. Key terms - **RPO** (how much data loss is acceptable) and **RTO** (how long recovery may take). Methods include logical dumps (`pg_dump`), physical/base backups with WAL archiving for **point-in-time recovery**, and managed-service snapshots.
2 · Business purpose	A backup that has never been restored is only a hope. Data loss from deletion, ransomware or a bad migration is an existential risk for the client.
3 · Real-world example	The client's RPO is 15 minutes and RTO is 1 hour. We use managed PITR snapshots plus a nightly dump copied to a separate account, and test a restore every quarter.
4 · Prerequisites	L4.1, L4.7.
5 · Tools & installation	`pg_dump` , `pg_restore` , `mysqldump` , cloud snapshots, object storage in a separate account/region, cron or scheduler.

6 · STEP-BY-STEP PROCEDURE

1. Agree RPO and RTO with the client in writing.
2. Automate backups; encrypt them and store a copy off the primary account/region.
3. Restore into a scratch database and run verification queries.
4. Record the restore time and compare it to the RTO.
5. Alert when a backup fails or is older than expected.

7 · COMPLETE EXAMPLE

```bash
pg_dump --format=custom --file=clinic_$(date +%F).dump "$DATABASE_URL"
createdb clinic_restore_test
pg_restore --dbname=clinic_restore_test --no-owner clinic_$(date +%F).dump
psql clinic_restore_test -c "SELECT count(*) FROM bookings;"
```

8 · EXPECTED OUTPUT

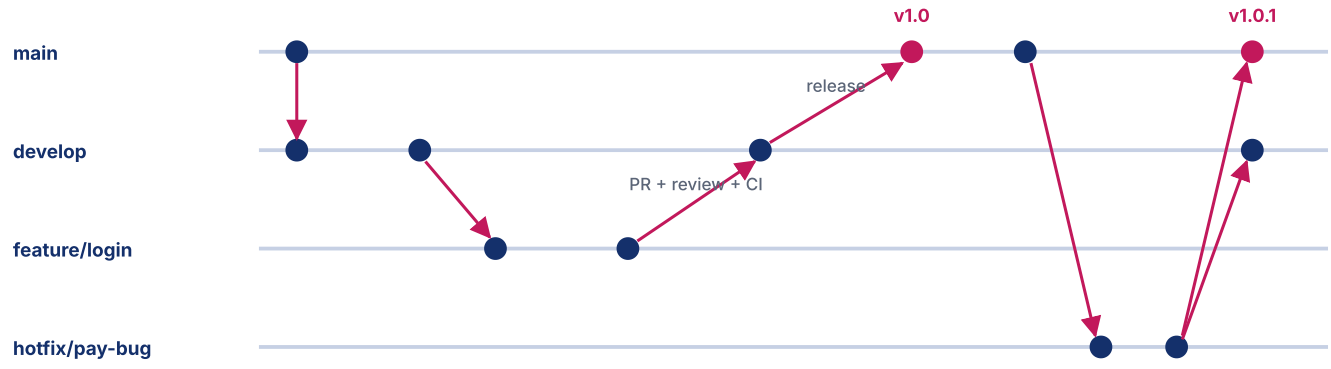
```output
A row count matching production at backup time, and a log line `restore completed in 4m12s`.
```

9 · Verification	A documented restore test exists for every project; the latest is under 90 days old.
10 · Common errors	Backups stored on the same server; never restoring; no encryption; forgetting uploaded files and configuration; assuming the host backs up automatically.
11 · Security considerations	Backups contain personal data - encrypt, restrict access, set retention and deletion rules; keep one immutable or offline copy.
12 · Documentation	Backup and restore runbook in `docs/deployment.md`; restore-test log in `qa/`.
13 · Practical assignment	Write and execute a backup + restore drill for the Project 4 database.
14 · Assessment	Given a "table dropped by mistake at 14:05" scenario, perform the recovery within the RTO.
15 · Approval gate	Level 4 completion review - modules signed; Project 4 CRUD application accepted with restore evidence.

Read: PostgreSQL - Backup and Restore

Level 5 - Professional Engineering (about 4 weeks)

Goal: work like a professional - version control, review, testing, containers, pipelines, cloud, observability and security.



Branch from develop → small PRs → squash-merge → tag releases on main → merge every hotfix back to develop

Figure 3.5 - Git branching strategy

Any red stage stops the pipeline — nothing reaches production on a failing check



Rollback = redeploy previous image tag (kept for at least 5 releases)

Figure 3.6 - Standard CI/CD pipeline

L5.1 · Git

Essential | Free | 8 h

1 · Definition	**Git** is a distributed version-control system. It records snapshots (**commits**) of the project, lets many people work in parallel on **branches**, and can restore any earlier state.
2 · Business purpose	Git is the safety net and the audit trail of every client project - who changed what, why, and how to undo it. No code is delivered without it.
3 · Real-world example	A developer breaks the checkout page at 5 pm. `git revert` restores the last good behaviour in a minute, and the history shows exactly which change caused it.
4 · Prerequisites	Level 2 started.
5 · Tools & installation	Git (free), VS Code Source Control, `git config` with company email, SSH key (ed25519).

6 · STEP-BY-STEP PROCEDURE

1. Install Git and set `user.name` and `user.email`.
2. Create a repository; make small commits with clear messages.
3. Create a feature branch, commit, and merge or rebase back.
4. Resolve a deliberate merge conflict.
5. Undo safely - `git restore`, `git revert`, and know why `git reset --hard` is dangerous.

7 · COMPLETE EXAMPLE

```bash
git config --global user.name "Your Name"
git config --global user.email "you@company.com"
git init demo && cd demo
echo "# Demo" > README.md && git add README.md && git commit -m "docs: add README"
git switch -c feature/contact-form
echo "form" > form.html && git add . && git commit -m "feat(form): add contact form markup"
git switch main && git merge --no-ff feature/contact-form
git log --oneline --graph
```

8 · EXPECTED OUTPUT

```output
*   a1b2c3d (HEAD -> main) Merge branch 'feature/contact-form'
|\
| * 9f8e7d6 feat(form): add contact form markup
|/
* 3c4d5e6 docs: add README
```

9 · Verification	The graph shows your branch and merge; `git status` is clean.
10 · Common errors	Committing secrets or `node_modules`; huge commits mixing unrelated changes; force-pushing shared branches; working directly on `main`.
11 · Security considerations	Never commit credentials (see `.gitignore` and secret scanning); if a secret is committed, **rotate it immediately** - deleting the commit is not enough.
12 · Documentation	`CONTRIBUTING.md` explains branch names, commit format and PR rules (Section 6).
13 · Practical assignment	Complete a 20-step Git lab - branch, merge, conflict, revert, tag, cherry-pick.
14 · Assessment	Live demo - recover a "lost" commit using `git reflog`.
15 · Approval gate	Mentor sign-off.

Read: Pro Git (free book) · GitHub Docs

▼ L5.2 · GitHub

Essential | Free | 6 h

1 · Definition	**GitHub** hosts Git repositories and adds collaboration - **pull requests**, **issues**, **projects**, **Actions** (CI/CD), branch protection, code owners, secret scanning and Dependabot. Plans and limits change - confirm on the GitHub pricing page.
2 · Business purpose	GitHub is where work is reviewed, tracked and automatically tested. The company organisation holds all client code; personal accounts are never used for client work.
3 · Real-world example	A pull request changes the pricing rule; two checks run (tests, lint), the Tech Lead reviews, and only then can it merge into `main`.
4 · Prerequisites	L5.1.
5 · Tools & installation	GitHub organisation account with 2FA, GitHub CLI `gh`, SSH keys, branch protection rules.

6 · STEP-BY-STEP PROCEDURE

1. Enable 2FA and add your SSH key.
2. Create a repository in the company organisation from the template.
3. Protect `main` - require pull request, 1 approval, passing checks, no force push.
4. Open an issue, branch from it, open a PR that links it (`Closes #12`).
5. Enable secret scanning and Dependabot alerts.

7 · COMPLETE EXAMPLE

```bash
ssh-keygen -t ed25519 -C "you@company.com"
gh auth login
gh repo create vel-org/client-project --private --template vel-org/web-project-template
git switch -c feat/12-booking-form
git push -u origin feat/12-booking-form
gh pr create --fill --reviewer techlead --body "Closes #12"
```

8 · EXPECTED OUTPUT

```output
A PR URL is printed; GitHub shows the checks running and the reviewer requested.
```

9 · Verification	Pushing directly to `main` is rejected; the PR cannot be merged until the checks pass.
10 · Common errors	Committing as a personal account; no branch protection; giant PRs; ignoring failed checks; using personal tokens in CI.
11 · Security considerations	Use fine-grained or app tokens; store secrets only in GitHub Secrets/Environments; require signed-off reviews for production workflows.
12 · Documentation	Repository `README.md`, `CODEOWNERS`, `.github/pull_request_template.md`.
13 · Practical assignment	Set up a complete repository with protection rules and a PR template for Project 3.
14 · Assessment	Review a sample PR with three planted problems.
15 · Approval gate	Tech Lead verifies repository settings against the checklist.

▼ L5.3 · VS Code

Essential · Free · 3 h

1 · Definition	**Visual Studio Code** is a free, extensible code editor with integrated terminal, debugger, Git and extensions. A shared **workspace configuration** keeps formatting and tools identical across the team.
2 · Business purpose	A consistent editor setup removes "works on my machine" formatting noise, speeds up debugging and enforces standards automatically on save.
3 · Real-world example	Prettier and ESLint run on save, so every pull request has the same style without review comments about spacing.
4 · Prerequisites	None.
5 · Tools & installation	VS Code; extensions - ESLint, Prettier, GitLens (optional), Docker, REST Client or Thunder Client, Playwright Test, Python, Error Lens.

6 · STEP-BY-STEP PROCEDURE

1. Install VS Code and the company extension pack (listed in `.vscode/extensions.json`).
2. Commit `.vscode/settings.json` with format-on-save.
3. Learn five shortcuts - command palette, quick open, multi-cursor, go to definition, rename symbol.
4. Debug a Node or Python program with a breakpoint through `launch.json`.

7 · COMPLETE EXAMPLE

JSON

```
{
  "editor.formatOnSave": true,
  "editor.defaultFormatter": "esbenp.prettier-vscode",
  "editor.codeActionsOnSave": { "source.fixAll.eslint": "explicit" },
  "files.trimTrailingWhitespace": true,
  "files.insertFinalNewline": true
}
```

8 · EXPECTED OUTPUT

OUTPUT

```
Saving a messy file reformats it and fixes lint issues automatically.
```

9 · Verification	Another team member opens the project and gets identical formatting with no manual setup.
10 · Common errors	Installing unknown extensions (supply-chain risk); committing personal settings; conflicting formatters.
11 · Security considerations	Install extensions only from verified publishers and the company list; enable Workspace Trust; keep secrets out of editor settings.
12 · Documentation	Editor setup steps in `README.md` ("Getting started").
13 · Practical assignment	Configure a workspace with debug configs for frontend and backend.
14 · Assessment	Perform five editing tasks within a time limit using shortcuts only.
15 · Approval gate	Mentor sign-off.

Read: VS Code documentation

▼ L5.4 · Code Reviews

Essential Free 4 h

1 · Definition	A **code review** is a structured check of a change by another engineer before it merges - for correctness, security, readability, tests and standards.
2 · Business purpose	Reviews catch defects when they are cheapest to fix, spread knowledge and protect the client from one person's mistake. Every production change needs a second person's review.
3 · Real-world example	The reviewer notices an unvalidated query parameter in a new endpoint and requests a fix before merge - avoiding a possible SQL injection.
4 · Prerequisites	L5.1, L5.2.
5 · Tools & installation	GitHub pull-request review, checklists, linters, CI results.

6 · STEP-BY-STEP PROCEDURE

1. Author - keep the PR small (under about 400 changed lines), explain what and why, attach screenshots or test evidence.
2. Reviewer - read the description, run the checks, review tests first, then code; comment on the code, not the person.
3. Classify comments - **blocker**, **suggestion**, **question**, **nit**.
4. Author responds to every comment; reviewer approves only when blockers are resolved.
5. Merge with squash and a conventional commit message.

7 · COMPLETE EXAMPLE

MARKDOWN

```
## What and why
Adds server-side date validation to booking creation (Closes #42).

## How tested
- [x] Unit tests added (12 cases)
- [x] Manual test in staging (screenshot attached)

## Checklist
- [x] No secrets - [x] Migrations reviewed - [x] Docs updated - [x] Accessibility checked
```

8 · EXPECTED OUTPUT

OUTPUT

```
A completed PR template and a review thread with labelled comments and an approval.
```

9 · Verification	Review checklist (Section 9.4) completed; CI green; no unresolved threads.
10 · Common errors	Rubber-stamp approvals; reviewing 2,000-line PRs; style debates that linters should settle; author merging own PR on production repositories.
11 · Security considerations	Reviewer checks authentication, authorisation, validation, secrets, logging of personal data and dependency changes.
12 · Documentation	Review checklist in `CONTRIBUTING.md`.
13 · Practical assignment	Review three PRs from the sample repository and write constructive comments.
14 · Assessment	Mentor rates review quality - found at least 5 of 7 planted issues.
15 · Approval gate	Mentor sign-off; reviewer rights granted after the gate.

▼ L5.5 · Testing

Essential | Free | 14 h

1 · Definition	**Testing** proves the software behaves as required. Layers - **unit** tests (one function), **integration** tests (several parts, e.g. API + database), **end-to-end (E2E)** tests (a real browser, e.g. Playwright), plus **API tests** (Postman/Newman) and manual exploratory testing.
2 · Business purpose	Automated tests let us change code fast without breaking the client's business, and provide evidence at acceptance.
3 · Real-world example	A Playwright test opens the site on a phone-sized viewport, books a slot and asserts the confirmation text. It runs on every pull request.

4 · Prerequisites	L2.3, L3.1.
5 · Tools & installation	Vitest/Jest, Pytest, Playwright (free), Postman + Newman (free tier), coverage tools (`coverage.py` , `c8`).

6 · STEP-BY-STEP PROCEDURE

1. Write unit tests for business rules first (test pyramid - many unit, fewer integration, few E2E).
2. Write an API integration test with a throw-away database.
3. Install Playwright and write one critical-path E2E test.
4. Run all tests in CI; fail the build on failure.
5. Track coverage as a guide, not a goal (aim for the important rules at 100%).

7 · COMPLETE EXAMPLE

```ts
// tests/booking.spec.ts  -  run with: npx playwright test
import { test, expect } from '@playwright/test';

test('visitor can request a booking on mobile', async ({ page }) => {
  await page.setViewportSize({ width: 390, height: 844 });
  await page.goto('/');
  await page.getByRole('link', { name: 'Book now' }).click();
  await page.getByLabel('Name').fill('Asha');
  await page.getByLabel('Email').fill('asha@example.com');
  await page.getByRole('button', { name: 'Confirm booking' }).click();
  await expect(page.getByRole('status')).toContainText('Booking confirmed');
});
```

8 · EXPECTED OUTPUT

```output
Running 1 test using 1 worker
  1 passed (3.2s)
(an HTML report is written to playwright-report/)
```

9 · Verification	The test fails when you break the feature on purpose, and passes when fixed (prove the test can fail).
10 · Common errors	Flaky tests using fixed sleeps; tests that depend on each other; testing implementation details; no test data cleanup; only testing the happy path.
11 · Security considerations	Never run tests against production data; use synthetic data; keep test credentials out of the repository.
12 · Documentation	Test plan and results in `qa/` (see QA checklist, Appendix A6).
13 · Practical assignment	Add unit, integration and two E2E tests to Project 3 with a CI run.
14 · Assessment	Write tests for a given buggy function so that the bug is detected.
15 · Approval gate	CI green with the required test layers.

Read: Playwright docs

L5.6 · Docker

Essential | Free | 10 h

1 · Definition	**Docker** packages an application and everything it needs into an **image** that runs the same everywhere as a **container**. **Docker Compose** runs several containers together (app, database, cache).
2 · Business purpose	Containers remove "works on my machine", make onboarding a one-command task and give repeatable deployments. Docker Desktop licensing for larger companies is paid - check Docker's current terms; alternatives include Podman and Colima.
3 · Real-world example	`docker compose up` starts the API, the PostgreSQL database and Mailpit identical to what staging runs.
4 · Prerequisites	L5.1, L3.1, L4.1.
5 · Tools & installation	Docker Engine / Docker Desktop, Docker Compose v2, a registry (GitHub Container Registry, Docker Hub, ECR).

6 · STEP-BY-STEP PROCEDURE

1. Write a Dockerfile - small base image, non-root user, dependencies cached before code.
2. Add `.dockerignore` (exclude `.git`, `node_modules`, `.env`).
3. Write `compose.yaml` with the app, `postgres` and a named volume.
4. Build, run and read logs; use health checks.
5. Scan the image for vulnerabilities (Trivy, Docker Scout).

7 · COMPLETE EXAMPLE

DOCKERFILE

```
# Dockerfile (FastAPI)
FROM python:3.12-slim
ENV PYTHONDONTWRITEBYTECODE=1 PYTHONUNBUFFERED=1
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY src/ ./src/
RUN useradd -m appuser
USER appuser
EXPOSE 8000
HEALTHCHECK CMD python -c "import urllib.request as u;
u.urlopen('http://localhost:8000/health')"
CMD ["uvicorn", "src.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

8 · EXPECTED OUTPUT

OUTPUT

```
$ docker compose up -d
[+] Running 3/3  api  db  mailpit
$ curl localhost:8000/health
{"status":"ok"}
```

9 · Verification	`docker compose ps` shows all services healthy; destroying and recreating containers keeps database data (volume).
10 · Common errors	Running as root; secrets baked into images; using `latest` tags; huge images; storing data inside the container filesystem.

11 · Security considerations	Pin versions, use minimal bases, non-root user, scan images, never mount the Docker socket, keep `.env` out of the image.
12 · Documentation	`deployment/docker/README.md` - build, run, environment variables, ports.
13 · Practical assignment	Containerise Project 4 (app + DB) and make it start with one command.
14 · Assessment	Explain image vs container vs volume and fix three Dockerfile problems.
15 · Approval gate	Tech Lead reviews Dockerfile and compose files.

Read: Docker docs

▼ L5.7 · CI/CD with GitHub Actions

Essential | Free | 10 h

1 · Definition	**Continuous Integration (CI)** automatically builds and tests every change. **Continuous Delivery/Deployment (CD)** automatically releases a tested build to staging, then production after approval. **GitHub Actions** runs workflows defined in `.github/workflows/*.yml`. Included free minutes differ by plan - check GitHub's current limits.
2 · Business purpose	Automation makes releases frequent, boring and safe; it enforces the quality gates for every client project.
3 · Real-world example	Merging to `main` builds the Docker image, runs tests and deploys to staging. A manual approval promotes the same image to production.
4 · Prerequisites	L5.2, L5.5, L5.6.
5 · Tools & installation	GitHub Actions, environments with required reviewers, GitHub Secrets, OIDC to cloud providers (no long-lived keys), a container registry.

6 · STEP-BY-STEP PROCEDURE

1. Create a CI workflow - install, lint, type-check, test, build.
2. Cache dependencies; run on `pull_request` and `push` to `main`.
3. Add a security job (dependency audit, secret scan, image scan).
4. Add a deploy job to staging, then a production job gated by environment approval.
5. Pin actions to a version and use least-privilege `permissions`.

7 · COMPLETE EXAMPLE

```yaml
# .github/workflows/ci.yml
name: CI
on:
  pull_request:
  push: { branches: [main] }
permissions: { contents: read }
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: 22, cache: npm }
      - run: npm ci
      - run: npm run lint
      - run: npx tsc --noEmit
      - run: npm test -- --run
      - run: npm run build
```

8 · EXPECTED OUTPUT

```output
A green check on the pull request; failures show the failing step log.
```

9 · Verification	Break a test on purpose - the PR is blocked; fix it - the PR turns green.
10 · Common errors	Secrets echoed in logs; unpinned third-party actions; deploying from forks; no rollback step; tests that pass locally but fail in CI due to hidden state.
11 · Security considerations	Least-privilege `permissions`, OIDC instead of stored cloud keys, environment protection for production, review changes to workflow files like production code.
12 · Documentation	Pipeline description and rollback procedure in `docs/deployment.md`.
13 · Practical assignment	Build a CI + staging CD pipeline for Project 4.
14 · Assessment	Explain each pipeline stage and show how a rollback is performed.
15 · Approval gate	Tech Lead approves pipeline and rollback test.

Read: GitHub Actions docs

▼ L5.8 · Cloud Deployment

Essential | Paid | 14 h

1 · Definition	**Cloud deployment** runs the application on infrastructure rented from AWS, Azure or Google Cloud (or platforms such as Vercel, Netlify, Cloudflare). Typical building blocks - compute (VM, container service, serverless), managed database, object storage, load balancer, DNS, CDN, secrets manager.
2 · Business purpose	Production needs repeatable, secure, monitored infrastructure with cost controls. The right level of cloud for the project avoids both fragile single servers and expensive over-engineering.

3 · Real-world example	A small business app runs as one container service + managed PostgreSQL + CDN; an enterprise SaaS uses a load-balanced container cluster, managed database with replicas and WAF.
4 · Prerequisites	L5.6, L5.7, L1.5.
5 · Tools & installation	AWS (App Runner, ECS, EC2, RDS, S3, CloudFront), Azure (App Service, Container Apps, Azure DB), GCP (Cloud Run, Cloud SQL), Cloudflare (DNS, CDN, WAF, Pages), Terraform (infrastructure as code). Always price with the provider's official calculator.

6 · STEP-BY-STEP PROCEDURE

1. Choose the provider and service model per the stack decision tree; record an ADR.
2. Create separate dev, staging and production environments (separate accounts/projects where possible).
3. Provision with infrastructure as code or a documented checklist; set budgets and cost alerts first.
4. Deploy through CI/CD; store secrets in the provider's secret manager.
5. Put the site behind HTTPS and a CDN/WAF; restrict database access to the private network.

7 · COMPLETE EXAMPLE

```bash
# Example: deploy a container to Google Cloud Run (illustrative - verify flags in current docs)
gcloud run deploy clinic-api \
  --image ghcr.io/vel-org/clinic-api:1.4.0 \
  --region asia-south1 \
  --set-secrets DATABASE_URL=clinic-db-url:latest \
  --allow-unauthenticated \
  --min-instances 0 --max-instances 3
```

8 · EXPECTED OUTPUT

```output
Service URL printed; revision `clinic-api-00007` receives 100% of traffic; rollback = route traffic to the previous revision.
```

9 · Verification	Smoke test passes (`/health`), HTTPS valid, logs visible, billing alert exists, rollback tested.
10 · Common errors	Public database; admin keys in the repository; one shared account for everything; no budget alert; manual console changes nobody records.
11 · Security considerations	Least-privilege IAM, MFA on root/admin, private networking, encryption at rest and in transit, WAF, regular patching, audit logging.
12 · Documentation	Architecture diagram, environment table, access list and cost estimate in `docs/deployment.md` .
13 · Practical assignment	Deploy Project 4 to a free/low-cost tier in one provider with HTTPS and a custom domain.
14 · Assessment	Whiteboard the production design for a medium client and justify cost vs reliability.
15 · Approval gate	Deployment checklist (Appendix A7) fully completed.

Read: AWS Documentation · Azure Documentation · Google Cloud Documentation · Cloudflare Docs

L5.9 · Logging

Essential | Free | 4 h

1 · Definition	**Logging** records events the application produces - requests, errors, business events. **Structured logs** (JSON with fixed fields) can be searched and aggregated; log levels (DEBUG, INFO, WARNING, ERROR) separate noise from signal.
2 · Business purpose	Logs are the first tool for diagnosing production problems and for audit trails, but they can also leak personal data if careless.
3 · Real-world example	A customer says "my payment failed". Searching the request id in the logs shows the gateway returned `card_declined` at 14:03.
4 · Prerequisites	L3.7.
5 · Tools & installation	Python `logging` / `structlog`, Node `pino`, Docker logs, cloud log services (CloudWatch, Azure Monitor, Cloud Logging), Loki/Grafana, ELK; Sentry for exceptions.

6 · STEP-BY-STEP PROCEDURE

1. Choose log levels and a JSON format with timestamp, level, message, request id, user id (not email).
2. Log at boundaries - request in/out, external calls, errors, important business events.
3. Never log passwords, tokens, full card numbers or national IDs.
4. Send logs to a central place with a retention period.
5. Create two saved searches - "all errors in 24 h" and "trace one request".

7 · COMPLETE EXAMPLE

PYTHON

```
import json, logging, sys

class JsonFormatter(logging.Formatter):
    def format(self, r: logging.LogRecord) -> str:
        return json.dumps({"ts": self.formatTime(r), "level": r.levelname, "msg":
r.getMessage(),
                           "logger": r.name, **getattr(r, "ctx", {})})

h = logging.StreamHandler(sys.stdout); h.setFormatter(JsonFormatter())
log = logging.getLogger("api"); log.addHandler(h); log.setLevel(logging.INFO)
log.info("booking created", extra={"ctx": {"rid": "3f9a2c1b", "booking_id": 981, "user_id":
42}})
```

8 · EXPECTED OUTPUT

OUTPUT

```
{"ts": "2026-11-04 10:32:11,408", "level": "INFO", "msg": "booking created", "logger": "api",
"rid": "3f9a2c1b", "booking_id": 981, "user_id": 42}
```

9 · Verification	A request id from the response header finds every related log line within seconds.
10 · Common errors	`print` debugging in production; logging whole request bodies; no retention policy; logs only on the server's disk; alert fatigue from noisy logs.
11 · Security considerations	Mask personal data; restrict log access; define retention (privacy law); treat logs as sensitive.

12 · Documentation	Log format, fields, levels and retention table in `docs/security.md`.
13 · Practical assignment	Add structured logging and one saved search to Project 4.
14 · Assessment	Diagnose a seeded incident from a log file within 15 minutes.
15 · Approval gate	Mentor sign-off.

▼ L5.10 · Monitoring and Alerting

Essential | Free | 6 h

1 · Definition	**Monitoring** measures the health of a system continuously. The key signals are **latency, traffic, errors and saturation**. **Uptime checks** test the site from outside; **alerts** notify a person when a threshold is crossed; an **SLO** is a target such as 99.9% availability.
2 · Business purpose	We should know about a failure before the client does. Monitoring is also what makes a maintenance contract credible and billable.
3 · Real-world example	An uptime check on `/health` every minute emails and messages the on-call engineer after two failures; a dashboard shows p95 latency and error rate.
4 · Prerequisites	L5.8, L5.9.
5 · Tools & installation	UptimeRobot/Better Stack/Checkly (free tiers - check limits), Grafana + Prometheus, cloud monitors, Sentry, Lighthouse CI.

6 · STEP-BY-STEP PROCEDURE

1. Add a `/health` endpoint that checks the database and critical dependencies.
2. Configure an external uptime check with SSL-expiry alert.
3. Create a dashboard - requests, p95 latency, 5xx rate, CPU/memory, disk, DB connections.
4. Define alerts that need action (not noise) and an escalation path.
5. Test one alert end to end.

7 · COMPLETE EXAMPLE

```python
@app.get("/health")
def health(db=Depends(get_db)):
    try:
        db.execute("SELECT 1")
    except Exception:
        raise HTTPException(status_code=503, detail="database unavailable")
    return {"status": "ok", "version": APP_VERSION}
```

8 · EXPECTED OUTPUT

```output
`200 {"status":"ok","version":"1.4.0"}` normally; `503` when the database is down, which triggers the alert.
```

9 · Verification	Stop the database in staging - an alert reaches the on-call channel within 5 minutes.
10 · Common errors	Alerts nobody owns; monitoring only CPU; health checks that always return 200; no certificate-expiry or backup-failure alerts.

11 · Security considerations	Keep `/health` free of sensitive details; restrict dashboards; monitor failed-login spikes and unusual traffic.
12 · Documentation	Monitoring plan - checks, thresholds, owners, escalation - in `docs/deployment.md`.
13 · Practical assignment	Build monitoring and alerts for the Project 4 deployment.
14 · Assessment	Explain the difference between an alert, a dashboard and a log, and when each is used.
15 · Approval gate	Alert fire-drill passed.

Read: web.dev - Measure performance

▼ L5.11 · Web Application Security

Essential | Free | 16 h

1 · Definition	**Web application security** protects confidentiality, integrity and availability. The reference is the **OWASP Top 10** (broken access control, cryptographic failures, injection, insecure design, misconfiguration, vulnerable components, authentication failures, integrity failures, logging failures, SSRF) and the OWASP **ASVS** checklist.
2 · Business purpose	A breach harms the client's customers and reputation and ends our relationship. Security is designed in, reviewed at every stage and verified before release - but we never claim a system is \"secure\" or \"hack-proof\".
3 · Real-world example	A pre-release review finds a customer-id parameter that returns other customers' invoices (IDOR); fixed with an ownership check and a regression test.
4 · Prerequisites	L3.3 to L3.8, L5.7.
5 · Tools & installation	OWASP ZAP (free), `npm audit` / `pip-audit`, Dependabot, Semgrep, Gitleaks, Trivy, Mozilla Observatory / securityheaders.com, browser DevTools.

6 · STEP-BY-STEP PROCEDURE

1. Threat-model the feature - assets, entry points, abuse cases.
2. Apply controls - validation, parameterised queries, output encoding, auth and access checks, CSRF protection, rate limiting.
3. Set security headers (CSP, HSTS, X-Content-Type-Options, frame protections, Referrer-Policy).
4. Scan - dependencies, secrets, containers and the running app (ZAP baseline).
5. Triage findings by severity; fix criticals before release; document accepted risks with the client.

7 · COMPLETE EXAMPLE

NGINX

```
# Nginx security headers (tune the CSP for each project)
add_header Strict-Transport-Security "max-age=31536000; includeSubDomains" always;
add_header X-Content-Type-Options "nosniff" always;
add_header Referrer-Policy "strict-origin-when-cross-origin" always;
add_header X-Frame-Options "DENY" always;
add_header Content-Security-Policy "default-src 'self'; img-src 'self' data:; frame-ancestors
'none'" always;
```

8 · EXPECTED OUTPUT

```
securityheaders.com shows A or better; ZAP baseline reports no high-severity findings.
```
OUTPUT

9 · Verification	A written security report lists scans run, findings, fixes and accepted risks, and is attached to the release.
10 · Common errors	Relying on the frontend for protection; disabling TLS verification "to make it work"; default admin passwords; verbose error pages; unpatched dependencies.
11 · Security considerations	This module is the control framework itself - apply the OWASP ASVS level agreed in the SOW; use the least privilege everywhere; plan for incident response and responsible disclosure.
12 · Documentation	`docs/security.md` - threat model, controls, scan results, incident contacts.
13 · Practical assignment	Run a full security review (manual + ZAP + dependency scan) on Project 4 and fix all high findings.
14 · Assessment	Attack-and-defend lab - exploit three seeded vulnerabilities, then patch them.
15 · Approval gate	Level 5 completion review - all modules signed; Security review signed by Tech Lead.

Read: OWASP · OWASP Top 10 · OWASP Cheat Sheet Series

Level assessment summary

Level	Written	Practical	Oral (Technical Lead)
1	20-question quiz, pass 80%	Whiteboard request path; DNS plan	10 minutes
2	30-question quiz	Timed UI task; component review	15 minutes
3	Security quiz	Attack-test of your API (five attacks)	20 minutes
4	SQL test (10 queries / 60 min)	Restore drill within RTO	15 minutes
5	OWASP and DevOps quiz	Pipeline + rollback demonstration; security review	30 minutes

PART A · FOUNDATIONS

4. Technology Stack and Tool Selection

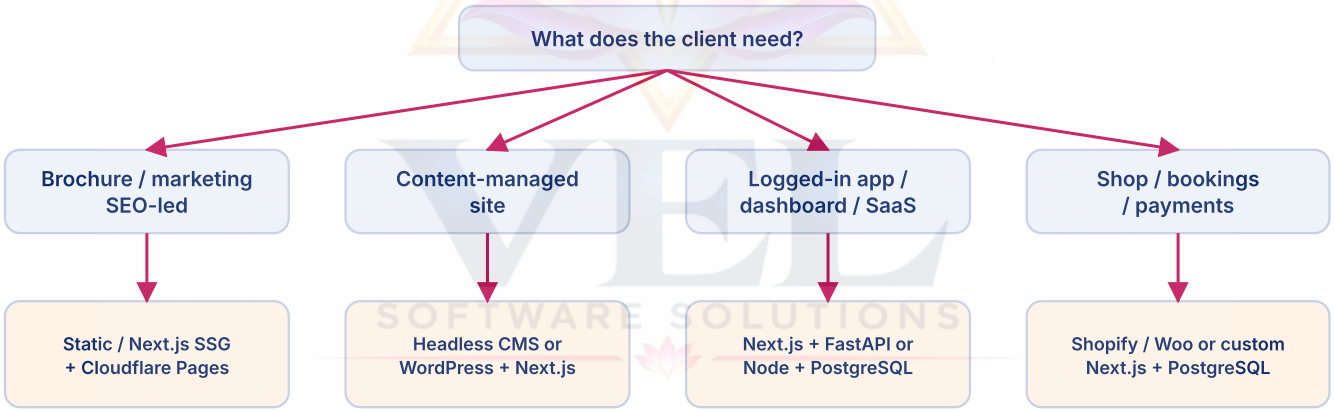
The department teaches and uses a broad toolbox, but **selects technology according to project requirements** instead of forcing one stack on every client. This section classifies each tool as **essential** or **optional**, and **free** or **paid**. Licence terms, free-tier limits and prices change - always check the official page on the day of use.

Tool classification

Tool / technology	Role	Essential or optional	Free or paid	Notes
HTML, CSS, JavaScript	Core web languages	**Essential**	Free (open standards)	MDN and the HTML Living Standard are the references
TypeScript	Typed JavaScript	**Essential** (all new projects)	Free (open source)	`strict` mode on
React	UI library	**Essential**	Free (open source)	Default for interactive UIs
Next.js	React framework (SSR/SSG, routing, API)	**Essential**	Free (open source); hosting may cost	Alternatives - Astro, Remix, Vite SPA, Nuxt, SvelteKit
Node.js	JavaScript runtime / backend	Essential	Free (open source)	LTS versions only for production
FastAPI	Python API framework	**Essential** (API-first products)	Free (open source)	Pydantic validation, OpenAPI docs
Django	Python full-stack framework	Optional (admin/ORM-heavy projects)	Free (open source)	Strong built-in admin and auth
PostgreSQL	Relational database	**Essential** (default)	Free (open source); managed hosting is paid	
MySQL / MariaDB	Relational database	Optional (WordPress/WooCommerce, legacy)	Free (community edition); managed hosting is paid	
Git	Version control	**Essential**	Free	
GitHub	Hosting, PRs, Actions	**Essential**	Free and paid plans - verify current limits	Organisation account with MFA
VS Code	Editor	**Essential**	Free	Shared extension pack
Docker	Containers	**Essential**	Engine free; Docker Desktop licence terms vary by company size - verify	Podman is an alternative

Tool / technology	Role	Essential or optional	Free or paid	Notes
Playwright	End-to-end testing	**Essential** (critical journeys)	Free (open source)	Runs in CI
Postman	API testing	Essential (or curl/Bruno/Insomnia)	Free tier + paid plans - verify	Export collections to Git
AWS / Azure / GCP	Cloud hosting	One **essential** per project	Pay-as-you-go; free tiers have limits - verify	Choose per client, region, budget
Cloudflare (or similar CDN)	DNS, CDN, WAF, caching	Optional but recommended	Free and paid plans - verify	Speed and DDoS protection
Terraform / OpenTofu	Infrastructure as code	Optional (medium/enterprise)	Free (open source)	
Sentry or similar	Error tracking	Recommended	Free tier + paid - verify	
Figma	UI/UX design	Essential for design work	Free tier + paid - verify	
Lighthouse, axe, ZAP	Quality and security scans	**Essential**	Free	

How to choose a stack



Pick the simplest stack that meets the requirement, then justify it in an Architecture Decision Record.

Figure 4.1 - Stack selection guide (start with the simplest option that meets the requirement)

Criterion	Questions to ask	Effect on the choice
Content or application?	Does the client mainly publish content, or do users log in and transact?	Content - static/CMS; application - Next.js + API + database
Who edits content?	Non-technical staff or developers?	Staff - headless CMS or WordPress; developers - Markdown/Git
Traffic and growth	Visitors per month now and in 12 months; peaks	Low - managed platform; high - CDN, caching, scaling
Data sensitivity	Personal, financial, health data?	Higher - private networking, encryption, audit, stricter hosting region

Criterion	Questions to ask	Effect on the choice
Integrations	Payment, ERP, CRM, SMS, WhatsApp	Prefer providers with good SDKs and webhooks
Team skills	Who maintains it after launch?	Prefer stacks the client's team or ours can maintain
Budget and running cost	Monthly cost the client can sustain	Managed services trade money for time
Time to market	Deadline	Buy (Shopify, WordPress, SaaS) before build
Compliance and region	Data residency, accessibility law, tax	Choose provider regions accordingly

Standard stacks (starting points, not rules)

Project type	Frontend	Backend	Database	Hosting pattern
Brochure / landing page	Astro or Next.js static	Serverless function for forms	-	CDN static hosting
Corporate site with CMS	Next.js	Headless CMS (or WordPress)	CMS-managed	Managed platform + CDN
E-commerce (standard)	Shopify / WooCommerce theme	Platform	Platform	SaaS or managed WordPress
E-commerce (custom)	Next.js	FastAPI or Node	PostgreSQL	Container service + managed DB + CDN
Customer portal / admin	Next.js + React	FastAPI/Django/Node	PostgreSQL	Container service + managed DB
SaaS	Next.js	FastAPI/Node + queue worker	PostgreSQL + cache	Containers, CI/CD, monitoring, backups
Booking system	Next.js	FastAPI/Node	PostgreSQL	Container service + e-mail/SMS provider

RECORD THE DECISION

Write an **Architecture Decision Record (ADR)** for every major choice (see L1.5 for the template): the context, at least two options, the decision, the consequences and the date any vendor facts were verified.

Licence and supply-chain rules

- Use only software whose licence allows commercial use; record each dependency's licence (`npm ls`, `pip-licenses`).
- Commit lock files (`package-lock.json`, `poetry.lock` / `requirements.txt` with hashes) and enable Dependabot.
- Do not install unknown packages or extensions; check download counts, maintainers and recent activity.

- Never paste client code or data into unapproved online tools or AI assistants. Use only company-approved tools and follow the client contract.

PART B · CLIENT DELIVERY

5. Client Project Lifecycle: Stages 01-27

Every client project - whether a one-page site or a SaaS platform - follows these **27 stages in order**. Small projects run stages quickly and combine documents; they never skip a gate. For each stage this section defines the **purpose, responsible employee, inputs, procedure, outputs, tools, risks, verification, documentation and approval authority**.

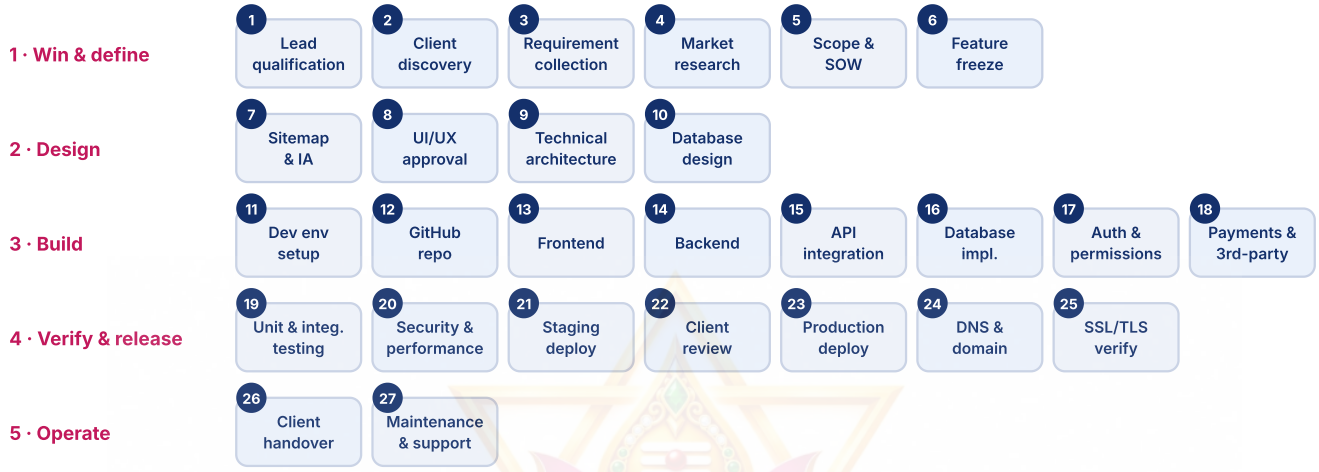


Figure 5.1 - The 27-stage client project lifecycle

Phases and gates

Phase	Stages	Main output	Key gate
1 - Win and define	1-6	Signed SOW, frozen scope	**Gate 05:** SOW approved - work may start
2 - Design	7-10	Sitemap, approved UI/UX, architecture, database design	**Gate 08:** design approved; **Gate 10:** DB design review
3 - Build	11-18	Working, integrated product in staging	Code review on every pull request
4 - Verify and release	19-25	Tested, secured, deployed, live with HTTPS	**Gate 22:** UAT sign-off before production
5 - Operate	26-27	Handover, maintenance, support	**Gate 26:** signed handover certificate

Role abbreviations used in the stage tables

Abbreviation	Role
PM	Project Manager
TL	Technical Lead
BD	Business Development executive
UX	UI/UX Designer
FE / BE / FS	Frontend / Backend / Full-stack developer

Abbreviation	Role
QA	QA engineer
DO	DevOps engineer

Approval authority matrix

Decision	Approves	Informed
Accept the lead	PM	BD, TL
Scope and SOW	Authorised commercial representative + client signatory	PM, TL
Design	Client sponsor	UX, TL, FE
Architecture and database design	TL	PM, DO, client IT
Merge to `main`	A reviewer who is **not** the author (TL for sensitive modules)	Team
Release to staging	TL	PM, QA
UAT acceptance	Client sponsor	PM, QA
Production deployment (go/no-go)	TL	PM, client
Handover	Client sponsor	PM, TL, DO
Change request	PM (impact) + client sponsor (cost/time)	TL

DO NOT START IMPLEMENTATION

without (1) an approved scope and SOW, (2) agreed commercial terms and (3) written access authorisation. Access is granted to named individuals with individual accounts - never shared admin credentials.

Change-request procedure (applies from Stage 6 onwards)

1. The client or a team member raises the change **in writing** (ticket or e-mail).
2. The PM and TL assess impact on cost, time, risk and other features.
3. The client approves the revised quote and timeline.
4. An SOW addendum is signed **before** work begins; the change log is updated.

Stage definitions

Stage 01 — Lead Qualification

Purpose	Decide quickly and honestly whether an enquiry is a good fit - budget, scope, timeline, decision-maker, risk - before spending delivery effort.
Responsible employee	Business Development executive (with Project Manager for technical fit).
Inputs	Enquiry (email, call, form, referral); company profile; rough idea and budget range.

Procedure	1. Log the lead in the CRM with source, contact, need and date. 2. Call within one working day; confirm who decides and who pays. 3. Score the lead (budget, urgency, clarity, authority, fit) 1-5 each; pass at 15 or more. 4. Screen for red flags - no budget, "copy this site exactly", unclear ownership of content, legal or ethical concerns. 5. Book a discovery call or politely decline with a referral.
Outputs	Lead record with score; go/no-go decision; discovery meeting invite.
Tools	CRM or spreadsheet, calendar, email templates.
Risks	Chasing unqualified leads; promising price or dates before understanding scope; copyright-infringing requests.
Verification	Second person reviews any lead scoring below 15 that is being pursued.
Documentation	Lead qualification sheet (Appendix A3, part 1).
Approval authority	Project Manager approves go/no-go.

GATE 01

Qualified lead recorded and discovery call scheduled.

Stage 02 — Client Discovery

Purpose	Understand the client's business, users, goals and constraints before proposing any solution.
Responsible employee	Project Manager (lead) with Technical Lead and UI/UX Designer attending.
Inputs	Qualified lead; existing website/system access (read-only); brand materials.
Procedure	1. Send the discovery questionnaire (Appendix A3) 2 days before the meeting. 2. Run a 60-90 minute structured session - business goals, target users, current process, pain points, success measures. 3. Ask about content (who writes it), branding, integrations, compliance and data sensitivity. 4. Record decisions, assumptions and open questions; never assume. 5. Send meeting minutes within 24 hours for the client to confirm.
Outputs	Completed questionnaire; confirmed meeting minutes; list of assumptions and open questions.
Tools	Discovery questionnaire, video call, shared notes, Miro/FigJam.
Risks	Hidden stakeholders; vague goals; "we will know it when we see it"; undisclosed legacy systems.
Verification	Client replies "confirmed" to the minutes; open questions have owners and dates.
Documentation	Discovery notes in `docs/requirements.md` (section "Discovery").
Approval authority	Project Manager; client sponsor confirms the minutes.

GATE 02

Client confirms the problem statement and goals in writing.

Stage 03 — Business and Technical Requirement Collection

Purpose	Turn needs into clear, testable requirements - functional, non-functional, content, integration and compliance.
Responsible employee	Business analyst or Project Manager; Technical Lead for non-functional requirements.
Inputs	Discovery minutes; existing assets; user stories; stakeholder list.
Procedure	1. Write user stories - "As a [role] I want [goal] so that [benefit]" - each with acceptance criteria. 2. Capture non-functional requirements - performance targets, availability, browsers/devices, accessibility level (WCAG 2.2 AA), SEO, languages, data retention, privacy. 3. List integrations (payment, email, SMS, CRM, analytics) and who owns each account. 4. Prioritise with MoSCoW (must, should, could, won't this time). 5. Review line by line with the client; mark each requirement as agreed.
Outputs	Requirements document with unique ids (REQ-001...), priorities and acceptance criteria.
Tools	Requirements template (Appendix A4), Jira/Linear/GitHub Projects, spreadsheet.
Risks	Ambiguous words ("fast", "user-friendly"); gold-plating; missing compliance rules; unspoken dependencies.
Verification	Every requirement is testable; each "must" has at least one acceptance test; Technical Lead confirms feasibility.
Documentation	`docs/requirements.md` (versioned in Git).
Approval authority	Technical Lead (feasibility) and Project Manager; client signs off.

Stage 04 — Competitor and Market Research

Purpose	Learn what good looks like in the client's market so design and features are informed, not guessed.
Responsible employee	Project Manager with UI/UX Designer.
Inputs	Client's competitors, target market, keywords, positioning.
Procedure	1. Identify 3-5 direct competitors and 2 best-in-class references. 2. Compare features, user journeys, pricing presentation, trust signals, speed and accessibility (Lighthouse, manual). 3. Check search visibility with free tools (Google Search Console later, keyword planners). 4. Note what to copy (patterns), what to avoid, and where the client can differentiate. 5. Never copy text, images or code from competitors (copyright).
Outputs	Competitor matrix, opportunity list, recommended differentiators.
Tools	Browser DevTools, Lighthouse, spreadsheet, public SEO tools.
Risks	Copying; analysis paralysis; confusing taste with evidence.
Verification	Findings cite specific pages/screenshots with date; claims are checkable.
Documentation	Research summary in `docs/requirements.md` (section "Market").

Approval authority	Project Manager.

Stage 05 — Scope and Statement of Work (SOW) Approval

Purpose	Create a binding agreement on what will be built, what will not, for how much and by when.
Responsible employee	Project Manager (author); Technical Lead (technical review); authorised commercial representative (price).
Inputs	Approved requirements; estimate (Section 8); risk list; legal templates.
Procedure	1. Use the SOW template (Appendix A5) - objectives, in-scope, out-of-scope, deliverables, assumptions, client responsibilities, timeline, milestones, acceptance criteria, price and payment terms, warranty/support, change control, IP and confidentiality. 2. Cost it with the costing workbook; include contingency; add a managed-service quote. 3. Technical Lead reviews feasibility and estimate; PM reviews commercial terms. 4. Send to client; address comments; obtain a signature (e-sign acceptable).
Outputs	Signed SOW; approved estimate; project baseline.
Tools	SOW template, costing workbook, e-signature tool, legal review where needed.
Risks	Undefined acceptance; scope creep; unlimited revisions; missing client responsibilities (content, access, approvals).
Verification	Checklist - every requirement mapped to an item; out-of-scope list present; payment milestones tied to deliverables.
Documentation	Signed SOW archived in the project folder; version recorded.
Approval authority	Authorised commercial representative and client signatory.

GATE 05

No work starts without an approved SOW, agreed commercial terms and written access authorisation.

Stage 06 — Input/Output and Feature Freeze

Purpose	Lock the inputs the client must supply and the outputs we will deliver, and freeze the feature list so estimates stay valid.
Responsible employee	Project Manager; Technical Lead.
Inputs	Signed SOW; requirement list.
Procedure	1. List client inputs - logo, brand colours, text, images, product data, third-party account access - with owners and due dates. 2. List outputs - screens, endpoints, reports, emails, documentation, training. 3. Agree the feature freeze date; after it, changes follow the change-request procedure only. 4. Record the baseline in the project tracker (all items, estimates, acceptance criteria).
Outputs	Input/output register; frozen feature list; change-control log opened.

Tools	Project board, register spreadsheet, change-request form.
Risks	Late content (the top cause of delay); "small additions"; hidden dependencies on third parties.
Verification	Client confirms the register; each item has an owner and date.
Documentation	Input/output register in `docs/requirements.md`; change log in `CHANGELOG.md` and tracker.
Approval authority	Project Manager and client sponsor.

GATE 06

Client signs the feature freeze.

Stage 07 — Sitemap and Information Architecture

Purpose	Organise content and navigation so users find things easily and search engines understand the site.
Responsible employee	UI/UX Designer with Project Manager.
Inputs	Requirements; content inventory; keyword research; competitor findings.
Procedure	1. Build a content inventory - every page, its owner and purpose. 2. Draw the sitemap (hierarchy) and key user flows (e.g. visitor → enquiry; customer → checkout). 3. Define navigation, URL structure (short, lowercase, readable) and page templates. 4. Plan SEO basics - one primary topic per page, titles, meta descriptions, redirects for old URLs. 5. Review with the client using a clickable outline.
Outputs	Sitemap diagram, user-flow diagrams, URL map, redirect list.
Tools	FigJam/Miro/Whimsical, spreadsheet, Screaming Frog (free tier) for existing sites.
Risks	Too many top-level menu items; orphan pages; breaking old URLs and losing rankings.
Verification	Card-sort or 5-user quick test on the outline; every page reachable in 3 clicks.
Documentation	Sitemap and URL map in `docs/requirements.md`.
Approval authority	Project Manager; client approves sitemap.

Stage 08 — UI/UX Design Approval

Purpose	Agree what the product looks like and how it behaves before any code is written.
Responsible employee	UI/UX Designer (author); client sponsor (approver).
Inputs	Sitemap; brand assets; competitor matrix; accessibility requirements.

Procedure	1. Create low-fidelity wireframes for key pages and flows; review with the client. 2. Build a design system - colours, typography, spacing, components, states. 3. Produce high-fidelity mock-ups for desktop and mobile (mobile-first). 4. Check contrast, tap-target size and focus order against WCAG 2.2 AA. 5. Build a clickable prototype; run a quick usability test; record client approval of final screens.
Outputs	Wireframes; design tokens; high-fidelity screens; prototype; developer hand-off notes.
Tools	Figma, accessibility checkers, Stark/contrast tools, Maze or manual user tests.
Risks	Endless revisions; designs that ignore real content; inaccessible colour choices; no mobile design.
Verification	Design review checklist; client approves in writing; Frontend Lead confirms feasibility.
Documentation	Figma link, version, approval email stored with the project.
Approval authority	Client sponsor (design); Technical Lead (feasibility).

GATE 08

Written design approval - changes after this follow change control.

Stage 09 — Technical Architecture

Purpose	Choose and document the stack, structure and infrastructure that will meet the requirements at the agreed cost and risk.
Responsible employee	Technical Lead (author); DevOps engineer; client's IT contact (review).
Inputs	Requirements; non-functional targets; budget; team skills; compliance needs.
Procedure	1. Compare at least two options (e.g. CMS vs custom, monolith vs services, managed vs self-hosted) against cost, security, scalability, maintainability. 2. Draw system, data-flow, deployment and security-boundary diagrams. 3. Define environments (dev, staging, production), hosting, CDN, email, storage, monitoring. 4. Record each major choice as an Architecture Decision Record (ADR). 5. Estimate hosting and third-party running costs from official pricing pages.
Outputs	Architecture document; ADRs; environment plan; running-cost estimate.
Tools	draw.io/Excalidraw, ADR template, vendor pricing calculators.
Risks	Over-engineering; resume-driven technology; hidden vendor lock-in; running costs not shared with the client.
Verification	Peer review by another senior engineer; cost figures traceable to vendor pages with date.
Documentation	`docs/architecture.md` and `docs/adr/`.
Approval authority	Technical Lead; client IT contact where applicable.

Stage 10 — Database Design

Purpose	Model the client's data correctly, securely and efficiently before building features on top of it.
Responsible employee	Backend developer or full-stack developer (author); Technical Lead (reviewer).
Inputs	Requirements; architecture; data-protection requirements.
Procedure	1. Identify entities, attributes and relationships; normalise to 3NF. 2. Define keys, constraints, indexes for expected queries and audit columns. 3. Classify personal and sensitive data; define retention and encryption needs. 4. Write the ER diagram and data dictionary; review with Technical Lead. 5. Plan migrations and seed data; plan backup (RPO/RTO).
Outputs	ER diagram; data dictionary; initial migration plan; backup plan.
Tools	dbdiagram.io/pgModeler, SQL DDL, migration tool.
Risks	Poor modelling that is expensive to change; money in floats; no constraints; collecting unnecessary personal data.
Verification	Walk through 10 real scenarios on the model; Technical Lead signs the review checklist.
Documentation	`docs/database-design.md` .
Approval authority	Technical Lead.

GATE 10

Database design review passed before implementation.

Stage 11 — Development Environment Setup

Purpose	Give every engineer an identical, secure, reproducible working environment.
Responsible employee	DevOps engineer or Technical Lead; each developer completes onboarding checks.
Inputs	Architecture; stack decisions; access list.
Procedure	1. Provision company-approved laptop with disk encryption and screen lock. 2. Install the toolchain from the README (Node/Python versions pinned, Docker, VS Code extension pack). 3. Create `.env` from `.env.example` using development-only values. 4. Run `docker compose up` and the test suite - both must pass on a fresh clone. 5. Give each person individual accounts (GitHub, cloud, tracker) with MFA.
Outputs	Working dev environment per developer; verified README setup steps.
Tools	Git, Docker, VS Code, password manager, MFA app.
Risks	Shared accounts; "works on my machine"; production credentials on laptops.
Verification	A new teammate follows the README and runs the app within 30 minutes.
Documentation	`README.md` getting-started section.

Approval authority	Technical Lead.

Stage 12 — GitHub Repository Creation

Purpose	Create the single source of truth for code and documentation with protection and automation in place from day one.
Responsible employee	Technical Lead or DevOps engineer.
Inputs	Project name; team list; standard repository template.
Procedure	1. Create a private repository in the company organisation from the template (folder structure, `.gitignore`, `.env.example`, PR template, CI workflow). 2. Protect `main` and `develop` - PR required, approvals, passing checks, no force push. 3. Add CODEOWNERS; enable secret scanning and Dependabot. 4. Create labels, milestones and the initial issues from the frozen requirements. 5. Add team members with least privilege.
Outputs	Configured repository; first commit; issue backlog.
Tools	GitHub, `gh` CLI, template repository.
Risks	Public repository by mistake; committed secrets; unprotected branches.
Verification	Repository settings checklist; a test push to `main` is rejected.
Documentation	`README.md`, `CONTRIBUTING.md`.
Approval authority	Technical Lead.

Stage 13 — Frontend Development

Purpose	Build the user interface exactly to the approved design, responsive, accessible and fast.
Responsible employee	Frontend developer (or full-stack developer); UI/UX Designer reviews.
Inputs	Approved design and tokens; API specification (or mock server); content.
Procedure	1. Implement design tokens, layout and shared components first. 2. Build pages from the design with mock data; then connect to the real API. 3. Handle loading, empty, error and success states for every data view. 4. Test at 360, 768, 1024 and 1440 px; keyboard-only; screen reader spot check. 5. Optimise images, fonts and bundle size; add SEO metadata. 6. Open small pull requests; demo each milestone in staging.
Outputs	Working UI in staging; component library; unit/component tests.
Tools	React/Next.js, TypeScript, Tailwind or CSS modules, Storybook (optional), Playwright, Lighthouse.
Risks	Design drift; inaccessible components; performance regressions; hard-coded content.
Verification	Design review against mock-ups; Lighthouse targets; accessibility checks; PR reviews.

Documentation	Component notes in `frontend/README.md`; Storybook where used.
Approval authority	UI/UX Designer (visual match); Technical Lead (code).

Stage 14 — Backend Development

Purpose	Implement the business rules, API and security controls that make the product work.
Responsible employee	Backend developer (or full-stack developer).
Inputs	Approved API specification; database design; business rules.
Procedure	1. Implement endpoints in the order of the critical user journeys. 2. Keep routes thin; put rules in a tested service layer. 3. Validate all input; return the standard error format; log with request ids. 4. Write unit and integration tests alongside each feature. 5. Add rate limiting, CORS rules and security headers.
Outputs	Working API in staging; automated tests; OpenAPI document.
Tools	FastAPI/Django/Node framework, Pytest/Jest, Postman, linters.
Risks	Logic duplicated in the frontend; missing authorisation checks; unbounded queries; secrets in code.
Verification	Test suite green in CI; peer code review; API contract tests against the spec.
Documentation	`docs/api-specification.md` (kept in sync with OpenAPI).
Approval authority	Technical Lead.

Stage 15 — API Integration

Purpose	Connect frontend, backend and the client's other systems through stable, documented, tested interfaces.
Responsible employee	Full-stack developer; Technical Lead.
Inputs	API specification; third-party API documentation and sandbox keys.
Procedure	1. Generate or write a typed API client; centralise base URL, auth and error handling. 2. Integrate screen by screen; remove mocks when the real endpoint passes tests. 3. Add timeouts, retries (only for safe calls) and circuit-breaking for third parties. 4. Test failure cases - 401, 403, 404, 422, 500, network down.
Outputs	Fully integrated flows; integration test results.
Tools	OpenAPI generators, fetch/axios, Postman/Newman, mock servers.
Risks	Contract drift; breaking changes; unhandled third-party downtime.
Verification	End-to-end test of each critical journey in staging.
Documentation	Integration notes and error handling table in `docs/api-specification.md`.

Approval authority	Technical Lead.

Stage 16 — Database Implementation

Purpose	Create and manage the real schema through versioned migrations with seed data and backups.
Responsible employee	Backend developer; DevOps engineer (backup/hosting).
Inputs	Approved database design; chosen database service.
Procedure	1. Write migrations; run them on a fresh database in CI. 2. Create least-privilege application and migration users. 3. Load seed/reference data (no real personal data). 4. Add indexes for measured queries; review plans with `EXPLAIN`. 5. Enable automated backups and run a restore test.
Outputs	Schema in dev and staging built only from migrations; backup/restore evidence.
Tools	Alembic/Django/Prisma migrations, psql, pgAdmin/DBeaver.
Risks	Manual schema edits; slow queries; missing backups; data loss during migration.
Verification	Schema diff between environments is empty; restore test passes within the RTO.
Documentation	`docs/database-design.md` updated; migration notes in `database/migrations/`.
Approval authority	Technical Lead.

Stage 17 — Authentication and Permissions

Purpose	Ensure only the right people can sign in and perform the right actions.
Responsible employee	Backend developer; Technical Lead reviews every change.
Inputs	Access matrix (roles x actions); security requirements; identity provider decision.
Procedure	1. Implement registration, login, logout, token/session handling and password reset (or integrate an identity provider). 2. Implement the access matrix (RBAC + ownership checks) server-side. 3. Add rate limiting, lockout/backoff, secure cookies and CSRF protection. 4. Add MFA for admin users where the SOW requires it. 5. Write tests for every cell of the access matrix and the five standard attacks.
Outputs	Authentication and authorisation implemented and tested; access matrix document.
Tools	Argon2/bcrypt, JWT or sessions, OAuth/OIDC provider, OWASP ZAP.
Risks	Broken access control (OWASP #1); weak password storage; session fixation; open registration abuse.
Verification	Security review checklist; negative tests; peer review by the Technical Lead.
Documentation	`docs/security.md` (flows, token lifetimes, access matrix).

Approval authority	Technical Lead.

Stage 18 — Payment and Third-Party Integrations

Purpose	Safely connect payment gateways and other external services so money, messages and data flow correctly.
Responsible employee	Backend developer; Technical Lead; Project Manager for client account setup.
Inputs	Client-owned gateway/provider accounts; sandbox keys; business rules for refunds, taxes and failures.
Procedure	1. Use the provider's hosted checkout or tokenisation so card data never touches our servers (reduces PCI scope). 2. Create orders server-side; verify payment by a **signed webhook**, not by the browser redirect alone. 3. Make webhook handling idempotent; store provider ids; handle failed, pending and refunded states. 4. Test in the sandbox - success, decline, timeout, duplicate webhook, refund. 5. Store provider API keys in the secret manager; rotate on staff changes.
Outputs	Working integrations; reconciliation report; sandbox test evidence.
Tools	Gateway SDKs (for example Razorpay, Stripe, PayPal - confirm availability, fees and compliance for the client's country on each official site), Postman, ngrok-style tunnels for webhooks in dev.
Risks	Double charging; unverified webhooks; storing card data; wrong tax or rounding; gateway downtime.
Verification	Test matrix executed and signed; finance contact reconciles test transactions.
Documentation	Integration runbook and flow diagram in `docs/architecture.md` .
Approval authority	Technical Lead and client finance contact.

Stage 19 — Unit and Integration Testing

Purpose	Prove with repeatable automated tests that each part and the connected parts behave as required.
Responsible employee	QA engineer with developers.
Inputs	Requirements with acceptance criteria; test data; staging build.
Procedure	1. Write a test plan mapping every requirement id to test cases. 2. Developers maintain unit tests; QA adds integration, API (Postman/Newman) and E2E (Playwright) suites. 3. Cover happy path, edge cases, invalid input and permission failures. 4. Run cross-browser (Chrome, Edge, Firefox, Safari) and device checks. 5. Log defects with steps, evidence and severity; retest after fixes.
Outputs	Test plan, automated suites, defect log, test report.
Tools	Pytest/Vitest, Playwright, Postman/Newman, BrowserStack or real devices.

Risks	Test gaps; flaky tests; testing only on developer machines.
Verification	100% of must-have requirements have a passing test; no open critical/high defects.
Documentation	`qa/test-plan.md`, `qa/test-report.md`.
Approval authority	QA engineer (results); Technical Lead (release readiness).

Stage 20 — Security and Performance Testing

Purpose	Find and fix vulnerabilities and slowness before the client's customers do.
Responsible employee	Technical Lead or security-trained engineer; DevOps for load tests.
Inputs	Staging environment; threat model; performance targets.
Procedure	1. Scan dependencies, secrets, containers and code (Dependabot/ `npm audit` / `pip-audit`, Gitleaks, Trivy, Semgrep). 2. Run OWASP ZAP baseline against staging; manually test top abuse cases (IDOR, XSS, injection, upload abuse). 3. Check headers, TLS configuration, cookies and CORS. 4. Run Lighthouse/WebPageTest; load-test critical endpoints (k6 or Locust) at the agreed traffic with p95 and error-rate targets. 5. Fix findings by severity; retest; document accepted risks.
Outputs	Security report, performance report, fixed-findings list.
Tools	OWASP ZAP, Gitleaks, Trivy, Semgrep, Lighthouse, k6/Locust.
Risks	Testing production by mistake; ignoring "medium" findings forever; unrealistic load profiles.
Verification	No open critical or high findings; performance targets met or exceptions agreed in writing.
Documentation	`docs/security.md` (results) and `qa/` reports. Never claim the system is "secure" - state what was tested.
Approval authority	Technical Lead; client acknowledges accepted risks.

Stage 21 — Staging Deployment

Purpose	Run the release candidate in a production-like environment for final verification.
Responsible employee	DevOps engineer.
Inputs	Release candidate tag; deployment runbook; staging environment.
Procedure	1. Deploy the exact artefact (Docker image) that will go to production via the pipeline. 2. Apply migrations; load realistic synthetic data (no real personal data). 3. Configure staging domain with HTTPS and basic protection (password or IP allow-list); block indexing. 4. Run smoke tests and the full E2E suite. 5. Rehearse rollback once.
Outputs	Staging URL; smoke-test results; rehearsed rollback.

Tools	GitHub Actions, Docker, cloud platform, Cloudflare.
Risks	Staging differs from production; staging indexed by search engines; real data copied.
Verification	Same image digest later promoted to production; configuration differences listed and justified.
Documentation	Release notes draft in `CHANGELOG.md`; deployment log.
Approval authority	Technical Lead.

Stage 22 — Client Review and User Acceptance Testing (UAT)

Purpose	Let the client verify the product against the agreed acceptance criteria.
Responsible employee	Project Manager (coordination); QA engineer (support).
Inputs	Staging URL; UAT script; acceptance criteria from the SOW.
Procedure	1. Hold a demo walkthrough; share the UAT script and test accounts. 2. Give a fixed review window (for example 5 working days) and one consolidated feedback round. 3. Triage feedback - defect (fix), change request (quote), or preference (discuss). 4. Fix defects, redeploy to staging and re-verify. 5. Obtain written UAT sign-off.
Outputs	UAT feedback log; fixed defects; signed UAT acceptance.
Tools	Staging site, shared feedback sheet or issue tracker, screen recordings.
Risks	Unstructured feedback; new scope disguised as bugs; unavailable client reviewers.
Verification	Each acceptance criterion marked pass or fail with evidence.
Documentation	UAT sign-off form in the project folder.
Approval authority	Client sponsor.

GATE 22

Written UAT acceptance before production deployment.

Stage 23 — Production Deployment

Purpose	Release the approved build to the live environment safely, with a rollback ready.
Responsible employee	DevOps engineer (execute); Technical Lead (go/no-go); Project Manager (client communication).
Inputs	UAT sign-off; deployment checklist (Appendix A7); backup; rollback plan.

Procedure	1. Agree a low-traffic window and notify the client. 2. Take a fresh backup and confirm it restores. 3. Promote the exact staging image through the pipeline with manual approval; run migrations. 4. Run smoke tests; watch logs, errors and latency for at least 30 minutes. 5. Record evidence - time, version, approver, results; roll back immediately if the criteria fail.
Outputs	Live system; deployment record; release tag.
Tools	CI/CD, cloud console (read-only after deploy), monitoring dashboards.
Risks	Unplanned downtime; failed migration; forgotten configuration; no rollback.
Verification	Post-deployment checklist complete; first backup of production taken.
Documentation	Deployment log and release notes.
Approval authority	Technical Lead (go/no-go); client sponsor (change window).

Stage 24 — DNS and Domain Configuration

Purpose	Point the client's domain at the live system correctly without breaking email.
Responsible employee	DevOps engineer; client domain owner (access).
Inputs	Domain ownership details; DNS plan; production endpoint.
Procedure	1. Document the current DNS records (screenshot/export) before any change. 2. Lower TTL to 300 seconds at least 24 hours before cut-over. 3. Add or change A/AAAA/CNAME records; keep MX, SPF, DKIM, DMARC unchanged unless planned. 4. Configure `www` and apex redirects; set canonical host. 5. Verify from multiple resolvers; raise TTL after stability.
Outputs	Live domain; DNS record table; email confirmed working.
Tools	Registrar/Cloudflare, `dig`, `nslookup`, mail test tools.
Risks	Email outage; wrong record; domain registered in an employee's name.
Verification	`dig` from at least two public resolvers returns the intended records; test email sent and received.
Documentation	DNS record table and registrar details in `docs/deployment.md`.
Approval authority	Technical Lead; client domain owner.

Stage 25 — SSL/TLS Verification

Purpose	Confirm all traffic is encrypted with a valid, auto-renewing certificate and a sound configuration.
Responsible employee	DevOps engineer.

Inputs	Live domain; certificate source (Let's Encrypt, cloud-managed, Cloudflare, paid certificate if required).
Procedure	1. Confirm HTTPS works for apex and `www`; HTTP redirects to HTTPS. 2. Check certificate chain, expiry date and automatic renewal. 3. Test with SSL Labs/Qualys server test and Mozilla Observatory; fix weak protocols. 4. Add HSTS only after confirming no HTTP-only dependencies. 5. Add certificate-expiry monitoring (alert at 21 and 7 days).
Outputs	TLS report; renewal evidence; monitoring alert.
Tools	SSL Labs, `openssl s_client`, browser padlock details, uptime monitor.
Risks	Expired certificate; mixed content; misconfigured redirect loop.
Verification	SSL Labs grade A or better; zero mixed-content warnings in DevTools.
Documentation	Certificate details and renewal method in `docs/deployment.md`.
Approval authority	Technical Lead.

Stage 26 — Client Handover

Purpose	Transfer ownership, knowledge and access so the client can run the system - or hand it to us on a managed plan.
Responsible employee	Project Manager; Technical Lead; DevOps engineer.
Inputs	Live system; documentation set; handover checklist (Appendix A8).
Procedure	1. Deliver the documentation pack - architecture, database, API, deployment, security, user guide, runbooks. 2. Transfer repository and cloud-account ownership as agreed; remove temporary access. 3. Share credentials only through the approved secret manager or expiring secure link. 4. Run a training session and record it; provide a quick-reference sheet. 5. Obtain the signed handover/acceptance certificate and confirm the warranty start date.
Outputs	Documentation pack; ownership transfer record; training recording; signed handover certificate.
Tools	Handover checklist, password/secret manager, video call recording.
Risks	Credentials in documents; missing runbooks; client cannot operate without us and does not know it.
Verification	Client performs three routine tasks (update content, create a user, restore a test backup) unaided.
Documentation	Complete `docs/` folder and `CHANGELOG.md` at release version.
Approval authority	Client sponsor (acceptance); Project Manager (closure).

GATE 26

Signed handover certificate.

Stage 27 — Maintenance and Support

Purpose	Keep the system secure, available and improving after launch under a clear service agreement.
Responsible employee	Support lead / DevOps engineer; Project Manager for account management.
Inputs	Support plan and SLA from the SOW; monitoring and ticket tools.
Procedure	1. Monitor uptime, errors, certificates and backups; respond to alerts by agreed severity. 2. Apply security patches and dependency updates monthly (critical issues in the agreed window) through staging first. 3. Test a restore quarterly; review cost and performance monthly. 4. Handle support tickets with first-response and resolution targets; classify bug vs change request. 5. Send a monthly report; review renewal and improvement roadmap quarterly.
Outputs	Monthly service report; patch log; restore-test log; renewal proposal.
Tools	Uptime monitor, ticket system, Dependabot, cloud billing alerts.
Risks	Unpriced "unlimited" support; unpatched components; lapsing domains/certificates.
Verification	SLA metrics measured and reported; renewals tracked 60 days ahead.
Documentation	Support log; runbooks updated after every incident (post-incident review).
Approval authority	Project Manager; client sponsor for plan changes and renewals.

VEL SOFTWARE SOLUTIONS

PART B · CLIENT DELIVERY

6. Project Structure and Repository Standards

Every project repository starts from the **company template** (also provided in this package as `project-template/`). The structure below is a real example for a **Next.js + FastAPI + PostgreSQL** project; adapt folder names to the chosen stack, but keep the same responsibilities. **Do not create empty folders for appearance** - every folder must have a purpose you can explain.

Standard project structure

```
web-development-project/
├── README.md
├── CHANGELOG.md
├── CONTRIBUTING.md
├── .gitignore
├── .env.example
├── compose.yaml
├── docs/
│   ├── requirements.md
│   ├── architecture.md
│   ├── database-design.md
│   ├── api-specification.md
│   ├── deployment.md
│   ├── security.md
│   ├── user-guide.md
│   └── adr/                (Architecture Decision Records)
├── frontend/
│   ├── public/
│   ├── src/
│   │   ├── components/
│   │   ├── pages/  (or app/ in Next.js App Router)
│   │   ├── layouts/
│   │   ├── hooks/
│   │   ├── services/
│   │   ├── styles/
│   │   └── assets/
│   └── tests/
├── backend/
│   ├── src/
│   │   ├── routes/
│   │   ├── controllers/
│   │   ├── services/
│   │   ├── models/
│   │   ├── middleware/
│   │   └── config/
│   └── tests/
├── database/
│   ├── migrations/
│   └── seeds/
├── deployment/
│   ├── docker/
│   └── scripts/
├── .github/
│   ├── workflows/
│   ├── CODEOWNERS
│   └── pull_request_template.md
└── qa/
```

What every folder and key file is for

Path	Purpose	Rules
`README.md`	Front door: what the project is, prerequisites, how to run, test, deploy, who to ask	A new joiner must run the app in 30 minutes using only this file

Path	Purpose	Rules
`CHANGELOG.md`	Human-readable history of releases (Keep a Changelog format, SemVer versions)	Updated in every release PR
`CONTRIBUTING.md`	Branching, commit, PR and review rules; definition of done	Mirrors this handbook
`.gitignore`	Files Git must never track - dependencies, build output, `.env`, keys, IDE files, OS files	Reviewed at repository creation
`.env.example`	Names and **fake/safe sample values** of every environment variable	Real values live only in the secret manager; update when a variable is added
`compose.yaml`	Local development stack - app, database, mail catcher	No production secrets
`docs/requirements.md`	Discovery notes, user stories (REQ-ids), acceptance criteria, input/output register	Source of truth for scope
`docs/architecture.md`	Diagrams, stack, environments, integrations, cost estimate	Updated with every architectural change
`docs/database-design.md`	ER diagram, data dictionary, indexes, retention, backup policy	Matches migrations
`docs/api-specification.md`	Endpoints, auth, request/response, error catalogue	Generated from OpenAPI where possible
`docs/deployment.md`	Environments, CI/CD, release, rollback, DNS/TLS, monitoring, runbooks	Used by DevOps and the client
`docs/security.md`	Threat model, controls, access matrix, scan results, incident contacts	Reviewed before every release
`docs/user-guide.md`	How the client's staff operate the system	Written in plain language with screenshots
`docs/adr/`	One short file per architecture decision	Never edited after acceptance - superseded instead
`frontend/public/`	Static files served as-is (favicon, robots.txt, images that need stable URLs)	No secrets, no private data
`frontend/src/components/`	Reusable UI building blocks (Button, Card, Form fields)	Small, typed, accessible, tested
`frontend/src/pages/` (or `app/`)	Routes/screens	One folder per route; thin - compose components
`frontend/src/layouts/`	Page shells (header, footer, dashboard frame)	
`frontend/src/hooks/`	Reusable stateful logic (`useDebounce`, `useAuth`)	
`frontend/src/services/`	API client and external calls	The only place that knows URLs
`frontend/src/styles/`	Design tokens, global CSS	Tokens mirror the Figma design system

Path	Purpose	Rules
`frontend/src/assets/`	Images/fonts imported by code	Optimised before commit
`frontend/tests/`	Component and E2E tests	Run in CI
`backend/src/routes/`	HTTP endpoints - parse, validate, call a service, return a response	No business logic
`backend/src/controllers/`	Optional adapter between routes and services (framework dependent)	
`backend/src/services/`	**Business logic** - pure, tested functions	Where rules live
`backend/src/models/`	Database models and schemas (ORM/Pydantic)	Match the migrations
`backend/src/middleware/`	Cross-cutting concerns - auth, request id, rate limit, error handling	
`backend/src/config/`	Settings loaded from environment variables	Fails fast when a variable is missing
`backend/tests/`	Unit and integration tests	Use a throw-away database
`database/migrations/`	Versioned schema changes	Never edit an applied migration
`database/seeds/`	Reference/demo data	No real personal data
`deployment/docker/`	Dockerfiles, nginx config, compose overrides	
`deployment/scripts/`	Release, backup, restore, smoke-test scripts	Documented and idempotent
`.github/workflows/`	CI/CD pipelines	Reviewed like production code
`.github/CODEOWNERS`	Auto-requests reviewers for sensitive paths	
`qa/`	Test plan, test cases, UAT script, reports, restore-test logs	Evidence for acceptance

Naming conventions

Item	Convention	Example
Repositories	`client-project` lowercase, hyphens	`sunrise-dental-portal`
Branches	`type/issue-short-description`	`feat/42-booking-form`
Folders and files (frontend)	Components `PascalCase.tsx` ; others `kebab-case`	`ProductCard.tsx` , `use-auth.ts`

Item	Convention	Example
Python files / functions	`snake_case`	`booking_service.py`, `create_booking()`
Python classes / TypeScript types	`PascalCase`	`BookingIn`, `OrderStatus`
Constants and env variables	`UPPER_SNAKE_CASE`	`DATABASE_URL`
Database tables / columns	`snake_case`, plural tables, singular columns	`bookings.starts_at`
API paths	Lowercase plural nouns, versioned	`/api/v1/bookings`
CSS classes	Short, descriptive; BEM or utility classes consistently	`.card__title`
Git tags	`vMAJOR.MINOR.PATCH`	`v1.4.0`

Environment separation

Environment	Purpose	Data	Access	Deploy
Local	Daily development	Synthetic seeds	The developer	Manual (`docker compose up`)
Development (shared)	Integration of merged work	Synthetic	Team	Auto from `develop`
Staging	Production-like verification, UAT	Synthetic or anonymised	Team + client reviewers	Auto from `main` /release candidate
Production	Real users	Real	Minimum named people	Pipeline with manual approval

Rules: separate cloud accounts or projects per environment where possible; separate databases, secrets, API keys and payment keys (test keys outside production); staging is protected and blocked from search engines; **production data is never copied to development**.

```
# .env.example  (committed - names and SAFE sample values only)
APP_ENV=local
DATABASE_URL=postgresql://app_user:change-me-dev@localhost:5432/app_dev
JWT_SECRET=generate-a-long-random-value-per-environment
SMTP_HOST=localhost
SMTP_PORT=1025
PAYMENT_KEY_ID=test_key_id_here
PAYMENT_KEY_SECRET=test_secret_here
ALLOWED_ORIGINS=http://localhost:3000
```

NEVER COMMIT

Credentials, API keys, private keys (`*.pem` , `id_ed25519`), production `.env` files, database dumps, customer data, screenshots showing tokens, `terraform.tfstate` with secrets. If a secret is committed: **(1)** revoke and rotate it immediately, **(2)** tell the Technical Lead, **(3)** only then clean the history. Deleting the file in a new commit does **not** remove it from history.

```
# .gitignore (essentials)
node_modules/
dist/
.next/
__pycache__/
.venv/
.env
.env.*
!.env.example
*.pem
*.key
*.sqlite3
*.dump
terraform.tfstate*
.DS_Store
.idea/
playwright-report/
coverage/
```

Branching strategy

Branch	Purpose	Rules
`main`	Always releasable; tagged for releases	Protected - PR + 1 approval + passing checks; no force push
`develop`	Integration of finished features (for multi-developer projects)	Protected
`feat/<id>-<name>`	One feature or fix	Short-lived (days, not weeks); branch from `develop`
`fix/<id>-<name>`	Bug fix	Same as above
`hotfix/<id>-<name>`	Urgent production fix	Branch from `main`; merge to `main` **and** `develop`; tag a patch release
`release/x.y.z`	Optional stabilisation branch for large releases	

Small projects may use trunk-based flow (`main` + short-lived feature branches). The Technical Lead chooses at Stage 12 and records it in `CONTRIBUTING.md`.

Commit standards (Conventional Commits)

Format: `type(scope): short imperative summary` - maximum about 72 characters; body explains **why**.

Type	Use
`feat`	New user-visible capability
`fix`	Bug fix
`docs`	Documentation only

Type	Use
`style`	Formatting, no logic change
`refactor`	Code change that is neither fix nor feature
`test`	Add or change tests
`perf`	Performance improvement
`build` / `ci`	Build system or pipeline
`chore`	Maintenance
`security`	Security fix (use a private channel for the details of unpatched vulnerabilities)

```
feat(booking): validate slot availability on the server

Prevents double booking when two users submit at the same time.
Adds a unique constraint on (slot_id) and a regression test.
Closes #42
```

Pull-request procedure

1. Branch from the latest `develop` / `main`; keep the change small (about 400 lines or fewer).
2. Run locally - format, lint, type-check, tests. Fix everything before pushing.
3. Open the PR using the template - **what and why, how tested, screenshots, checklist**; link the issue (`Closes #42`).
4. CI must be green. Request a reviewer who is **not** the author.
5. Reviewer applies the checklist (L5.4): correctness, tests, security, accessibility, performance, docs, no secrets.
6. Author answers every comment; reviewer approves; **squash-merge** with a conventional message.
7. Delete the branch. The merge to `main` triggers the pipeline to staging.

Definition of done

- [ ] Meets every acceptance criterion in the requirement (REQ-id)
- [ ] Unit/integration tests added and passing; critical journey covered by E2E where relevant
- [ ] Lint and type checks pass; no new warnings
- [ ] Reviewed and approved by another person
- [ ] Accessibility and responsive checks done
- [ ] No secrets, no personal data in code or logs
- [ ] Documentation and `CHANGELOG.md` updated
- [ ] Deployed to staging and smoke-tested

Versioning and releases

Use **Semantic Versioning**: `MAJOR.MINOR.PATCH` - MAJOR for breaking changes, MINOR for new backward-compatible features, PATCH for fixes. Release steps: update `CHANGELOG.md`, tag `vX.Y.Z`, build **once**, deploy the same artefact to staging and then production, and record the deployment evidence.

PART B · CLIENT DELIVERY

7. Professional Delivery Standards

These standards define what "professional" means in our deliverables. Each row has a **measurable target** and a **way to verify it**. A release is not accepted until the QA checklist (Appendix A6) shows every applicable standard as met or as a documented, client-approved exception.

7.1 Coding conventions

Standard	Requirement	Verified by
Language	TypeScript `strict` for frontend/Node; Python 3.11+ with type hints for FastAPI	`tsc --noEmit`, `mypy` / `pyright` in CI
Formatting	Prettier (JS/TS/CSS/MD), Black + Ruff (Python); format on save	CI format check
Linting	ESLint with accessibility plugin; Ruff; zero warnings in `main`	CI
Naming	Per Section 6 table; names describe intent (`calculateInvoiceTotal`, not `calc2`)	Review
Function size	One responsibility; aim below about 40 lines; avoid deep nesting (more than 3 levels)	Review
Comments	Explain **why**, not what; public functions have docstrings/JSDoc	Review
Dead code	No commented-out code or unused exports in `main`	Linter
Dependencies	Justify each; pinned via lock file; licence recorded; no abandoned packages	Dependabot, review
Secrets	Read from environment variables only; the app fails fast at startup if one is missing	Config tests, secret scan
Error handling	Never swallow exceptions; catch where you can act	Review

7.2 Reusable components

Standard	Requirement
Design system	Colours, type scale, spacing and radii live as design tokens (CSS variables), mirrored from Figma
Component API	Typed props with sensible defaults; no hidden global state; composable (children/slots)
States	Every interactive component has default, hover, focus-visible, active, disabled, loading and error states
Accessibility	Built on semantic HTML; keyboard operable; labelled; documented in the catalogue
Documentation	Name, purpose, props table and usage example (Storybook for larger projects)
Testing	Each shared component has at least a render test and an interaction test
Reuse rule	The second use of a pattern → extract a component; the third → it must already be one

7.3 Responsive behaviour

Standard	Target
Approach	Mobile-first CSS; `<meta name="viewport">` on every page
Breakpoints	360 (base), 768, 1024, 1440 px - test at each; also 320 and 1920
Layout	Flexbox/Grid, relative units; no horizontal scrolling at any width
Touch	Tap targets at least 44×44 px; spacing between targets; no hover-only actions
Typography	Body text at least 16 px; line length about 45-75 characters; fluid headings (`clamp()`)
Images	`srcset` / `sizes` , modern formats (WebP/AVIF), explicit width/height to prevent layout shift
Verification	DevTools device mode + at least one real phone and one tablet

7.4 Accessibility (WCAG 2.2 AA)

Standard	Requirement
Structure	Landmarks (`header` , `nav` , `main` , `footer`), one `h1` , logical heading order, `lang` attribute
Keyboard	Everything reachable and operable by keyboard; visible focus; no keyboard traps; skip-to-content link
Contrast	Text 4.5:1 (large text 3:1); UI components and focus indicators 3:1
Alternatives	Meaningful `alt` text; captions/transcripts for media; no information by colour alone
Forms	Visible labels, error messages linked with `aria-describedby` , clear instructions, correct `autocomplete`
Motion	Respect `prefers-reduced-motion` ; no auto-playing audio; no flashing content
Verification	axe/Lighthouse >= 90 with zero critical issues + manual keyboard test + one screen-reader pass

7.5 Performance

Metric (mobile, field/lab)	Target ("good" thresholds per web.dev)
Largest Contentful Paint (LCP)	2.5 s or less
Interaction to Next Paint (INP)	200 ms or less
Cumulative Layout Shift (CLS)	0.1 or less
Lighthouse Performance (mobile)	90 or more for content sites; 80 or more for complex apps
API latency	p95 under 500 ms for standard endpoints (agreed per project)
Page weight	Initial JavaScript under about 200 KB gzipped for content pages; images optimised

Metric (mobile, field/lab)	Target ("good" thresholds per web.dev)
Practices	Compress and lazy-load images, subset fonts, cache static assets with long TTL + content hashing, use a CDN, avoid blocking scripts, paginate lists, add DB indexes for measured queries
Verification	Lighthouse CI, WebPageTest, k6/Locust load test, DB `EXPLAIN` evidence

7.6 SEO (where relevant)

Standard	Requirement
Rendering	Content visible in HTML (SSR/SSG), not only after client JavaScript
Metadata	Unique `<title>` and meta description per page; canonical URL; Open Graph tags
Structure	Clean URLs, XML sitemap, `robots.txt`, correct redirects (301) for changed URLs
Content	One primary topic per page; descriptive headings and link text; alt text
Structured data	JSON-LD (Organization, LocalBusiness, Product, Article) validated in the Rich Results test
Technical	HTTPS, mobile-friendly, fast, no broken links; submit sitemap in Search Console
Honesty	No ranking guarantees - SEO results depend on competition and time

7.7 Secure authentication

Standard	Requirement
Password storage	Argon2id (or bcrypt) with per-user salt; never reversible encryption
Password policy	Minimum 10 characters, allow long passphrases, check against breached-password lists, no forced periodic rotation without cause
Sessions/tokens	httpOnly, Secure, SameSite cookies; access tokens 15 minutes or less; refresh rotation; logout invalidates
Brute force	Rate limit and backoff per account and per IP; generic error messages
MFA	Required for admin/staff accounts; offered to customers
Recovery	Single-use, expiring, random reset tokens; notify the user of changes
Authorisation	Server-side RBAC + ownership checks on every request; deny by default
Verification	Attack tests (credential stuffing, token tampering, IDOR, CSRF, replay) pass

7.8 Database security

Standard	Requirement
Network	Private network only; no public database port
Access	Separate least-privilege roles for application, migrations, read-only reporting

Standard	Requirement
Queries	Parameterised queries / ORM only - never string-built SQL
Encryption	TLS in transit; encryption at rest; field-level encryption for highly sensitive data
Data minimisation	Store only what is needed; define retention; pseudonymise where possible
Auditing	Log privileged access and changes to sensitive tables
Backups	Encrypted, off-site, restore-tested (Section 7.12)
Test data	Synthetic or anonymised data only outside production

7.9 API validation

Standard	Requirement
Schemas	Every request validated by a typed schema (Pydantic/Zod); unknown fields rejected where relevant
Limits	Maximum body size, string lengths, page sizes, upload sizes
Output	Response models whitelist fields; never return password hashes or internal ids you do not need
Idempotency	`Idempotency-Key` for payment-like POSTs; webhooks verified by signature and de-duplicated
Rate limiting	Per user/IP; stricter on login, reset and expensive endpoints
CORS	Allow-list of origins - never `*` with credentials
Contract	OpenAPI is the contract; breaking changes require a new version

7.10 Error handling

Standard	Requirement
Format	One JSON error shape: `{ "error": { "code", "message", "request_id" } }`
Status codes	400/422 validation, 401 unauthenticated, 403 forbidden, 404 missing, 409 conflict, 429 rate limited, 5xx server
User messages	Plain language, actionable, no technical details
Frontend	Loading, empty and error state for every data view; retry where safe; global error boundary
Resilience	Timeouts on all outbound calls; retries only for idempotent operations; graceful degradation

7.11 Logging and monitoring

Standard	Requirement
Format	Structured JSON with timestamp, level, message, request id, user id
Content	No passwords, tokens, full card or national-ID numbers; mask personal data
Levels	ERROR for action needed, WARNING for unexpected but handled, INFO for business events, DEBUG off in production

Standard	Requirement
Retention	Defined per project and privacy law (for example 30-90 days operational; longer for audit as agreed)
Monitoring	Uptime check, error tracking, latency/error-rate dashboard, certificate expiry, disk and database alerts, backup-failure alert

7.12 Backup and recovery

Standard	Requirement
Objectives	RPO and RTO agreed in the SOW
Method	Automated daily backups plus point-in-time recovery where the database supports it; uploaded files and configuration included
Location	At least one copy outside the primary account/region; encrypted
Testing	Restore test before go-live and at least quarterly, with timing recorded
Retention	Documented (for example 7 daily, 4 weekly, 12 monthly)

7.13 Browser and device compatibility

Standard	Requirement
Supported	Latest two versions of Chrome, Edge, Firefox and Safari (desktop and mobile); anything different is agreed in the SOW
Method	Feature support checked on MDN/Can I use; progressive enhancement; no browser sniffing
Verification	Playwright projects for Chromium, Firefox and WebKit + real-device spot checks

7.14 Documentation

Standard	Requirement
Required set	`README` , requirements, architecture, database design, API specification, deployment runbook, security notes, user guide, CHANGELOG
Quality	Accurate, dated, versioned in Git, written for the next person; diagrams as source (draw.io/Mermaid)
Update rule	A change is not "done" until its documentation is updated
Handover	The complete pack is delivered and walked through with the client (Stage 26)

PART C · BUSINESS AND PEOPLE

8. Commercial Costing and Pricing

This section teaches how to cost a web project, set the selling price, and price the monthly service - and gives worked examples for **small, medium and enterprise** projects. The **same numbers** are in the Excel workbook `VEL-Web-Project-Costing-Workbook.xlsx` (formulas live), so you can change an assumption and see every result update.

NO VENDOR PRICES ARE STATED IN THIS HANDBOOK

Vendor prices (hosting, cloud, domains, e-mail, payment fees, licences) change often and differ by region, plan and date, and this edition could not retrieve live vendor pricing pages. Every vendor and internal-rate figure below is a **clearly labelled placeholder in generic currency units (CU)**, shown only to demonstrate the method. **Before any quotation**, replace each placeholder with the figure from the vendor's **official pricing page or calculator** (links in 8.10), and record the region, plan, currency, date checked, usage assumptions and taxes in the *Vendor Pricing Log* sheet. A quotation containing an unverified price is not approved.

8.1 What goes into a project cost

#	Cost item	Type	How it is priced	Official source of truth
1	Discovery and planning	One-time, labour	Hours x role rate	Internal rate card
2	UI/UX design	One-time, labour	Hours x rate (screens, components, prototype, accessibility)	Internal rate card
3	Frontend labour	One-time, labour	Hours x rate (pages, components, states, responsive)	Internal rate card
4	Backend labour	One-time, labour	Hours x rate (endpoints, rules, auth, jobs)	Internal rate card
5	Database work	One-time, labour	Hours x rate (design, migrations, tuning)	Internal rate card
6	Integrations	One-time, labour	Hours x rate per integration (payment, e-mail, CRM, SMS, analytics)	Internal rate card + vendor docs
7	Testing / QA	One-time, labour	Hours x rate (plan, automation, UAT support)	Internal rate card
8	Security review	One-time, labour	Hours x rate + scan tools	Internal rate card
9	Project management	One-time, labour	About 10-15% of build hours	Internal rate card
10	Software licences (one-time)	One-time, third party	Vendor quote	Vendor pricing page
11	Contingency	One-time, reserve	10-20% of subtotal by risk	Policy

#	Cost item	Type	How it is priced	Official source of truth
12	Hosting / compute	Recurring	Calculator, per month	Cloud/vendor calculator
13	Domain and DNS	Recurring (yearly)	Registrar price / 12 for monthly view	Registrar page
14	SSL/TLS where chargeable	Recurring (yearly)	Often free (managed or Let's Encrypt); paid for EV/OV or special cases	CA / platform page
15	E-mail services	Recurring	Per message or per mailbox	Provider page
16	Storage and backups	Recurring	Per GB-month + requests	Cloud calculator
17	Bandwidth / CDN	Recurring	Per GB transferred	CDN/cloud calculator
18	Monitoring and logging	Recurring	Per host/check/GB	Vendor page
19	Maintenance	Recurring, labour	Hours/month x rate	Internal rate card
20	Support	Recurring, labour	Hours/month x rate or SLA tier	Internal rate card
21	Software licences (recurring)	Recurring	Per seat or per month	Vendor pricing page

8.2 Formulas

```
Labour cost            = sum(hours x internal cost rate per role)
Subtotal               = Labour + one-time third-party costs
Contingency            = Subtotal x contingency %
Total delivery cost    = Subtotal + Contingency
Selling price          = Total delivery cost / (1 - target gross margin %)
Gross profit           = Selling price - Total delivery cost
Gross margin %         = Gross profit / Selling price
Markup %               = Gross profit / Total delivery cost        (NOT the same as margin)
Variable costs         = one-time third-party costs + commissions + bought-in contractor cost
Contribution margin    = Selling price - Variable costs           (what the project adds toward
salaries and overhead)
Break-even (projects)  = Annual fixed costs / Contribution margin per project
Break-even (hours)     = hours at which gross profit reaches zero
Monthly operating cost = (vendor pass-through + service labour) x (1 + service contingency)
Annual operating cost  = Monthly operating cost x 12
Monthly service fee    = Monthly operating cost / (1 - service margin %)
MRR = sum of monthly fees      ARR = MRR x 12
Renewal fee            = max(current fee, next-year cost / (1 - service margin %))
Cost per user          = Monthly cloud cost / Active users
```

MARGIN IS NOT MARKUP

A 40% **margin** requires dividing cost by 0.60 (a 66.7% **markup**). Multiplying cost by 1.40 gives only a 28.6% margin. Always say which one you mean in quotes and reports.

8.3 Rate card and assumptions (all placeholders)

Role	Internal cost / hour (CU)
Project Manager	25
Technical Lead	32
UI/UX Designer	20
Frontend Developer	18
Backend Developer	20
Database Engineer	20
Full-stack Developer	20
QA Engineer	15
DevOps Engineer	22

Assumption (all placeholders)	Value
Contingency (small / medium / enterprise)	10.0% / 15.0% / 20.0%
Target gross margin (price = cost / (1 - margin))	40.0% / 40.0% / 35.0%
Price rounding	up to the next 100 CU
Referral/sales commission (variable)	5.0%
Company fixed costs per year (salaries + overhead)	360,000 CU
Monthly service contingency / target margin	5.0% / 45.0%
Renewal inflation - vendor / wages	5.0% / 6.0%

HOW TO SET THE REAL RATE CARD

Internal cost per hour = (annual salary + employer costs + share of overhead) / productive hours per year (typically 70-75% of paid hours). Finance owns the rate card; developers never quote from personal rates.

8.4 Effort estimate (hours) for the three example projects

Size	Example scope
Small	Corporate website - about 8 pages, CMS-lite, contact form, SEO basics
Medium	Booking / customer portal - login, roles, database, payment gateway, admin screens
Enterprise	Multi-tenant SaaS - roles, billing, dashboards, integrations, audit trail, high availability

Work item	Role	Small (h)	Medium (h)	Enterprise (h)
Discovery and planning	Project Manager	12	40	140

Work item	Role	Small (h)	Medium (h)	Enterprise (h)
Architecture and technical-lead review	Technical Lead	4	24	100
UI/UX design	UI/UX Designer	20	70	240
Frontend development	Frontend Developer	40	160	620
Backend development	Backend Developer	10	180	760
Database design and implementation	Database Engineer	0	30	150
Integrations (APIs, payment, email)	Full-stack Developer	6	40	180
Testing and QA	QA Engineer	12	60	260
Security review	Technical Lead	4	16	80
Deployment and DevOps	DevOps Engineer	8	24	120
Project management	Project Manager	16	56	220
Total hours		**132**	**700**	**2870**

8.5 Project cost and selling price

Cost line (CU)	Small	Medium	Enterprise
Discovery and planning	300	1,000	3,500
Architecture and technical-lead review	128	768	3,200
UI/UX design	400	1,400	4,800
Frontend development	720	2,880	11,160
Backend development	200	3,600	15,200
Database design and implementation	0	600	3,000
Integrations (APIs, payment, email)	120	800	3,600
Testing and QA	180	900	3,900
Security review	128	512	2,560
Deployment and DevOps	176	528	2,640
Project management	400	1,400	5,500
Labour subtotal	**2,752**	**14,388**	**59,060**
Third-party one-time (placeholder)	40	150	600
Subtotal	2,792	14,538	59,660
Contingency	279 (10.0%)	2,181 (15.0%)	11,932 (20.0%)
Total delivery cost	**3,071**	**16,719**	**71,592**
Target margin	40.0%	40.0%	35.0%

Cost line (CU)	Small	Medium	Enterprise
Selling price (cost / (1 - margin), rounded up)	**5,200**	**27,900**	**110,200**
Gross profit (price - cost incl. contingency)	2,129	11,181	38,608
Gross margin %	**40.9%**	**40.1%**	**35.0%**
Equivalent markup on cost	69.3%	66.9%	53.9%

Hand-calculated check - Small project

Every formula is checked by hand; compare this block with the table above.

```
Labour      = sum(hours x rate)           = 2,752 CU   (132 h, blended 20.85 CU/h)
Third-party = one-time licences/assets    = 40 CU
Subtotal    = 2,752 + 40                  = 2,792 CU
Contingency = 2,792 x 10.0%              = 279.2 CU
Total cost  = 2,792 + 279.2              = 3,071.2 CU
Price       = cost / (1 - 0.40)  = 3,071.2 / 0.60 = 5,118.7 -> rounded up to 5,200 CU
Gross profit= 5,200 - 3,071.2             = 2,128.8 CU
Gross margin= 2,128.8 / 5,200               = 40.9%
Markup      = 2,128.8 / 3,071.2               = 69.3%   (markup is NOT margin)
Contribution= price - (third-party + 5.0% commission) = 5,200 - 300 = 4,900 CU (94.2%)
Break-even  = fixed costs / contribution  = 360,000 / 4,900 = 73.5 small projects per year
```

8.6 Contribution margin and break-even

Measure	Small	Medium	Enterprise
Variable costs (third-party + commission)	300	1,545	6,110
Contribution margin (price - variable costs)	4,900	26,355	104,090
Contribution margin %	94.2%	94.5%	94.5%
Break-even: projects per year to cover fixed costs	73.5	13.7	3.5
Blended labour cost per hour	20.85	20.55	20.58
Break-even hours (gross profit reaches zero)	222 vs 132 planned	1176 vs 700 planned	4478 vs 2870 planned

WHY CONTRIBUTION MARGIN LOOKS VERY HIGH HERE

Our developers are salaried, so their pay is a **fixed** cost - it is paid whether or not this project exists. Contribution margin therefore excludes labour; it shows how much each project contributes toward salaries and overhead. If the work were bought from freelancers, their cost would be **variable** and the contribution margin would fall to about the gross margin. The break-even figure answers: *how many projects of this size per year cover our fixed costs?*

8.7 Monthly and annual operating cost, service fee and renewal

Operating cost line (CU / month)	Small	Medium	Enterprise
Hosting / compute	12.00	85.00	900.00
Domain and DNS (annual cost / 12)	1.25	1.25	2.50
SSL/TLS certificate (if chargeable)	0.00	0.00	0.00
Email service	4.00	15.00	80.00
Storage and backups	2.00	10.00	120.00
Bandwidth / CDN	3.00	20.00	250.00
Monitoring and logging	0.00	15.00	150.00
Software licences / SaaS	5.00	25.00	300.00
Vendor pass-through / month	**27.25**	**171.25**	**1,802.50**
Maintenance and patching (DevOps Engineer, 3/10/40 h)	66.00	220.00	880.00
Support (tickets) (Full-stack Developer, 2/8/30 h)	40.00	160.00	600.00
Account management (Project Manager, 0.5/2/8 h)	12.50	50.00	200.00
Service labour / month	118.50	430.00	1,680.00
Contingency 5%	7.29	30.06	174.12
Monthly operating cost	**153.04**	**631.31**	**3,656.62**
Annual operating cost (x 12)	**1,836**	**7,576**	**43,880**
Monthly service fee (cost / (1 - 45%), rounded up to 5)	280	1,150	6,650
Annual service fee (MRR x 12 = ARR)	3,360	13,800	79,800
Service gross margin	45.3%	45.1%	45.0%
Renewal fee / month (next-year cost, never below current)	**295**	**1,215**	**7,015**
Renewal fee / year	3,540	14,580	84,180

Renewal pricing rule. Renew at the next-year cost plus the service margin, using the vendors' published price changes (check at renewal time) and the company's wage plan. Give the client **60 days'** notice of the renewal price. Never reduce a service fee below the cost of delivering the agreed SLA. Offer a multi-year discount only if the margin stays at or above the policy minimum.

8.8 Summary of the three examples

Size	Typical scope	Delivery cost	Selling price	Gross margin	Service / month	Service / year
Small	Corporate website - about 8 pages, CMS-lite, contact form, SEO basics	3,071	5,200	40.9%	280	3,360

Size	Typical scope	Delivery cost	Selling price	Gross margin	Service / month	Service / year
Medium	Booking / customer portal - login, roles, database, payment gateway, admin screens	16,719	27,900	40.1%	1,150	13,800
Enterprise	Multi-tenant SaaS - roles, billing, dashboards, integrations, audit, high availability	71,592	110,200	35.0%	6,650	79,800

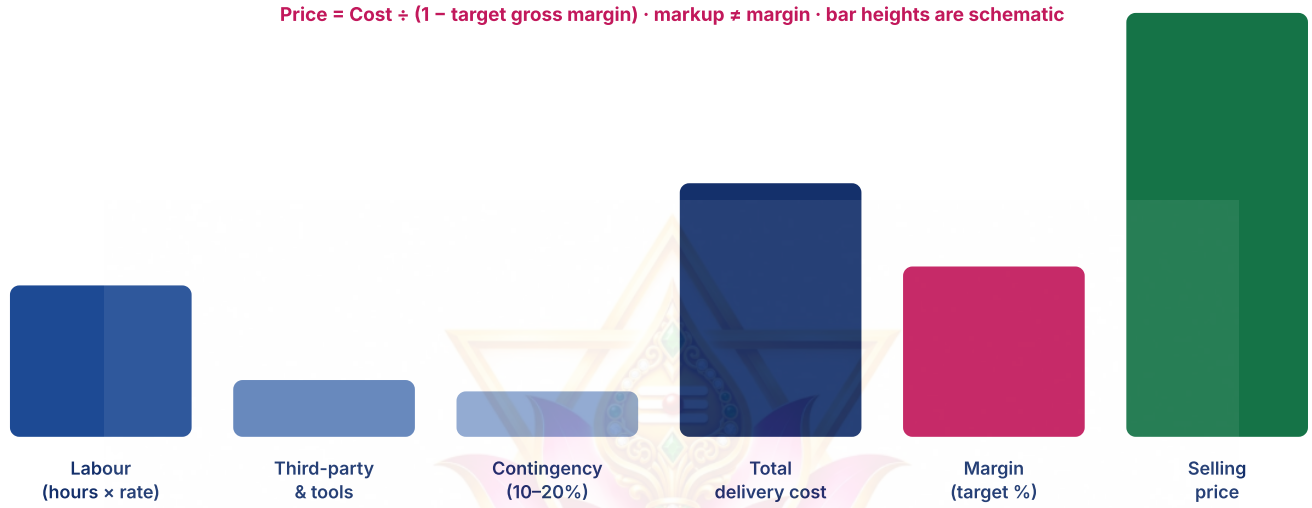


Figure 8.1 - From cost to price (schematic)

8.9 Scale scenario template (cloud cost per user)

For products with usage-based costs, estimate at four scales using the provider calculator. Fill the table from the **official calculator** - never from memory - and record region, date and assumptions.

Active users	Compute	Database	Storage	Network / CDN	Monitoring	Total / month	Cost per active user
100	*fill from calculator*						
1,000							
10,000							
100,000							

Questions to answer for each row: does the architecture change at this scale (cache, read replica, queue, autoscaling)? What is the new risk? Does the price per user still support our margin?

8.10 Official pricing sources to verify before quoting

Cost	Official page or calculator (check the date and region)
AWS	`https://calculator.aws/` and `https://aws.amazon.com/pricing/`
Microsoft Azure	`https://azure.microsoft.com/pricing/calculator/`

Cost	Official page or calculator (check the date and region)
Google Cloud	`https://cloud.google.com/products/calculator`
Cloudflare	`https://www.cloudflare.com/plans/`
Vercel / Netlify	`https://vercel.com/pricing` - `https://www.netlify.com/pricing/`
GitHub	`https://github.com/pricing`
Docker	`https://www.docker.com/pricing/`
Domains / registrar	The chosen registrar's price list (renewal price differs from first-year price)
E-mail providers	The chosen provider's pricing page
Payment gateways	The gateway's official fee page for the client's country and payment methods
Taxes	Local tax authority guidance (for example GST/VAT); quote tax separately

Verification rule. Each price in a quote has a row in the Vendor Pricing Log with: vendor, item, plan, region, currency, unit price, usage assumption, URL, date checked, checked by. A reviewer re-opens at least 20% of the links before the quote is approved. Prices older than 30 days are re-checked.

8.11 Expense classification (put it in every contract)

Type	Meaning
Customer-billed cloud	Client's own cloud account, paid directly to the provider (preferred)
Company-managed cloud	Our account; re-billed with a stated management fee
Pass-through third party	Licences and subscriptions billed at cost
Service fee	Our labour and expertise

Prefer the **client owning their cloud, domain and repository accounts**, with us holding limited, individual access. Define additional-usage rates and caps. Avoid "unlimited" support or usage unless it is priced in.

8.12 Commercial rules

- Quote **in writing** with scope, assumptions, exclusions, validity (for example 30 days), taxes, payment schedule and change-request rates.
- Typical payment milestones: advance on SOW signature; on design approval; on staging delivery; on go-live; retention until the end of the warranty. Milestones are tied to **accepted deliverables**.
- A quote includes **contingency**; it is not a hidden discount pool. Unused contingency is profit; overruns are managed through change control.
- Do not discount the price without removing scope. Discount only with approval from the authorised commercial representative.
- Estimates for fixed-price work must be reviewed by the Technical Lead; the project manager owns the budget tracking.
- Track actual hours against the estimate weekly; report variance above 10% to the PM and TL at once.

PART C · BUSINESS AND PEOPLE

9. Roles, Procedures and Escalation

9.1 Roles and responsibilities

Role	Core responsibilities	Authority and limits	How success is measured
Junior Frontend Developer	Implement UI components and pages from approved designs; write component tests; fix UI bugs; follow accessibility and responsive standards; open small PRs	Works from assigned tickets; no architecture changes; no production access; every PR reviewed	Design match, accessibility score, review rework rate, tickets delivered vs estimate
Junior Backend Developer	Implement endpoints, validation, service-layer rules and tests from the API spec; write migrations under review; fix defects	No schema or auth changes without TL review; no production access; no secrets in code	Test coverage of rules, defect rate, spec conformity
Full-stack Developer	Deliver features end to end - UI, API, database; integrate APIs; own a module; mentor juniors	Can propose design changes; merges only after review; limited staging access	Feature lead time, quality at UAT, documentation completeness
UI/UX Designer	Research, sitemap, wireframes, design system, hi-fi screens, prototypes, accessibility and hand-off; review the built UI	Owns design decisions within the approved brief; client approves	Client approval cycles, usability findings, design-implementation match
QA Engineer	Test plan from requirements; manual and automated tests (Playwright, Postman); defect management; UAT support; release-readiness report	Can block a release by raising a critical defect to the TL	Defect escape rate, automation coverage of critical journeys
DevOps Engineer	Environments, CI/CD, hosting, infrastructure as code, monitoring, backups, DNS/TLS, patching, cost alerts	Production changes only through the pipeline or a documented change; least privilege	Deployment success rate, mean time to restore, backup-restore test results
Technical Lead	Architecture and ADRs, standards, code review of sensitive areas, security sign-off, estimation review, mentoring, go/no-go for releases	Approves technical decisions and releases; escalates budget/scope to PM	Quality at release, security findings, team growth, estimate accuracy
Project Manager	Scope, SOW, schedule, budget, risks, change control, client communication, status reports, UAT and handover coordination	Owns client relationship and plan; cannot change technical decisions alone	On-time and on-budget delivery, client satisfaction, margin vs plan

CAREER LADDER

Junior → Developer → Senior → Lead. Promotion requires demonstrated gates: independent delivery, quality of reviews given, mentoring, documentation, and security awareness - not tenure alone.

9.2 Daily work procedure

Time	Activity	Output
Start of day	Read messages; check assigned tickets and CI status; plan the day (3 priorities)	Daily plan in the tracker
Stand-up (15 min)	Each person: yesterday, today, blockers - no problem-solving in the meeting	Blockers list with owners
Focus blocks	Build, test, review - phone and notifications off	Commits, PRs
Mid-day	Review at least one teammate's PR; respond to review comments	Review given
End of day	Push work-in-progress to your branch; update tickets and timesheet; note next steps	Daily log (3 lines)

Working agreements: commit at least daily; keep PRs small; never leave a branch un-pushed overnight; respond to review requests within one working day; ask early (stuck rule in Section 2).

9.3 Task assignment

Stage	Rule
Ticket quality ("ready")	Has a clear title, user story or problem, acceptance criteria, design/API links, estimate, priority, owner, requirement id
Assignment	Technical Lead / PM assigns by skill and learning need; a junior receives work with a named reviewer
Sizing	Estimate in hours or points; any ticket over 2 days is split
Work-in-progress limit	Maximum 2 active tickets per person
Workflow	Backlog → Ready → In progress → In review → In staging → Accepted → Done
Blocked	Mark as blocked with the reason and owner; mention in stand-up; escalate per 9.8
Done	Meets the definition of done (Section 6)

9.4 Code review (reviewer checklist)

- [ ] Understands the purpose (ticket, requirement id); change is small and focused
- [ ] CI green; tests added and meaningful; edge cases covered
- [ ] Authentication and authorisation checks present on the server
- [ ] Input validated; output encoded; SQL parameterised; no injection or XSS path
- [ ] No secrets, tokens or personal data in code, logs, fixtures or screenshots
- [ ] Error handling and logging consistent with standards
- [ ] Accessibility and responsive behaviour checked (UI changes)
- [ ] Performance: no N+1 queries, no unbounded lists, images optimised
- [ ] Migrations safe (locks, rollback), backward compatible
- [ ] Documentation and CHANGELOG updated
- [ ] Comments labelled **blocker**, **suggestion**, **question**, **nit**; tone respectful

9.5 Timesheets

Rule	Detail
Frequency	Daily entry; weekly submission by Friday 18:00
Granularity	By project and task (ticket id); 15-minute increments
Categories	Billable (project work), Non-billable (training, internal, sales support), Leave
Honesty	Record actual time - estimates are improved only by accurate actuals
Approval	PM approves weekly; discrepancies above 10% against the plan are explained
Use	Invoicing (time-and-materials), margin tracking, estimate calibration. Never used to measure keystrokes or punish honest reporting

9.6 Leave and handover

Before leave of more than one day, complete and share a **handover note** with your mentor/TL:

```
HANDOVER NOTE
Name / dates away:
Project / tickets in progress (links, state, next step):
Branches and PRs open (links, review state):
Environments touched (what is deployed where, anything fragile):
Pending client communication / deadlines:
Where the notes and credentials access are (never the credentials themselves):
Who to contact for what while I am away:
Known risks:
```

Rules: push all work; no one should need your laptop; production access is never given to "cover" a colleague - the TL arranges authorised cover.

9.7 Documentation rules

- Documentation lives in the repository (`docs/`) and in the shared knowledge base; **no knowledge only in chat or in one person's head**.
- Every module owner updates docs in the same PR as the code.
- Decisions are recorded as ADRs; incidents as post-incident reviews; meetings as short minutes with action owners.
- Diagrams are stored as editable source files.

9.8 Escalation



Figure 9.1 - Escalation ladder (start at L1; move up when the trigger is met)

Severity	Definition	Example	Initial response (internal target - set per SLA)	Escalate to
P1 Critical	Production down, data loss or security breach	Site returns 500 for all users; leaked credentials	Immediately; page the on-call and TL	TL + PM at once; Director and client sponsor within 1 hour
P2 High	Major feature broken, no workaround	Payments failing for some users	Within 1 hour	TL, PM
P3 Medium	Feature degraded, workaround exists	Search slow	Same working day	Mentor / TL
P4 Low	Cosmetic or request	Typo, minor layout issue	Within 2 working days	Ticket owner

How to escalate: state *what happened, impact, since when, what you tried, what you need*. Do not wait for certainty. Keep a timeline of actions during incidents.

SECURITY INCIDENTS

If you suspect a leaked secret, a compromised account or a data breach: **(1)** stop - do not delete evidence, **(2)** tell the TL within one hour, **(3)** revoke/rotate exposed credentials with the TL's direction, **(4)** the PM and Director decide client and legal notifications. Never discuss an incident in public channels or social media.

9.9 Communication and conduct

- Client communication goes through the PM or the agreed channel; developers do not agree to scope or price changes in chat.
- Write short, specific messages; summarise decisions in the ticket; confirm agreements in writing.
- Be honest about status and risk - bad news early is cheaper than bad news late.
- Respect confidentiality: no client details on public forums, AI tools or personal accounts without approval.
- Credit sources; never present copied code or text as your own; respect open-source licences.

PART D · PRACTICE AND SOURCES

10. Practical Employee Projects and Certification

Seven practical projects take the trainee from a single page to a complete simulated client engagement. Each project has a **brief, acceptance criteria, skills practised, deliverables and a reviewer**. Projects are built in the training organisation on GitHub, follow the repository standards (Section 6) and are accepted only after code review and demonstration.

#	Project	Level	Duration	Reviewer
1	Responsive business website	2	1 week	Mentor
2	Multi-page corporate website	2	2 weeks	Mentor + UX
3	Authentication-enabled web application	3	2 weeks	Technical Lead
4	CRUD application with database	3-4	2 weeks	Technical Lead
5	E-commerce prototype	5	2 weeks	Technical Lead
6	SaaS admin dashboard	5	2 weeks	Technical Lead + QA
7	Complete simulated client project	1-5	3 weeks	PM + Technical Lead + QA

Project 1 - Responsive business website

Item	Detail
Brief	A 3-5 section one-page site for a fictional local business (clinic, bakery, gym) with hero, services, about, contact form and footer
Stack	HTML5, CSS3 (Flexbox/Grid), a little JavaScript
Must include	Semantic structure; mobile-first responsive layout; accessible navigation and form; optimised images; SEO metadata; Lighthouse >= 90 (Performance, Accessibility, Best Practices, SEO)
Deliverables	GitHub repo with README, deployed page on free static hosting, Lighthouse report
Acceptance	Passes W3C validator; works at 360/768/1024/1440 px; keyboard-only usable; no console errors

Project 2 - Multi-page corporate website

Item	Detail
Brief	A 6-8 page corporate site (Home, About, Services, Service detail, Blog/News, Contact) with a design from the UX Designer
Stack	Next.js + TypeScript; content in Markdown/MDX or a headless CMS
Must include	Shared layout and component library; SEO (metadata, sitemap, `robots.txt`, structured data); image optimisation; a working contact form with server-side validation and e-mail; 404 page
Deliverables	Repo with docs, Storybook or component catalogue, Playwright smoke test, deployed preview, performance report
Acceptance	Matches the design within tolerance; LCP <= 2.5 s on mobile profile; accessibility checklist passed

Project 3 - Authentication-enabled web application

Item	Detail
Brief	A member area - register, verify e-mail, log in, log out, reset password, profile, roles (member/admin)
Stack	Next.js + FastAPI (or Node) + PostgreSQL
Must include	Argon2id hashing; httpOnly cookies or short-lived JWT + refresh; rate limiting; RBAC with ownership checks; validation; error format; request ids; tests for the access matrix
Deliverables	Repo, API specification, security notes (`docs/security.md`), test report
Acceptance	Technical Lead's five attack tests fail (credential stuffing, token tampering, IDOR, CSRF, brute force); no secrets in Git history

Project 4 - CRUD application with a database

Item	Detail
Brief	A bookings or inventory app with at least six related tables - list, search, filter, paginate, create, edit, delete, export CSV, upload a document
Stack	Next.js + API + PostgreSQL + migrations
Must include	ER diagram and data dictionary; migrations and seeds; indexes backed by `EXPLAIN`; transactions for multi-step operations; file upload hardening; Docker Compose; CI pipeline; backup and restore drill
Deliverables	Repo, `docs/database-design.md`, restore-test log, CI run
Acceptance	Fresh clone runs with one command; migrations build an identical schema; concurrency test passes; restore completes within the stated RTO

Project 5 - E-commerce prototype

Item	Detail
Brief	Catalogue, product page, cart, checkout, order history, admin product management; payment in **sandbox/test mode only**
Stack	Next.js + API + PostgreSQL + payment provider sandbox
Must include	Server-side price and total calculation; tax and discount rules with unit tests; order state machine; webhook verification and idempotency; order e-mails; inventory handling with transactions
Deliverables	Repo, API spec, payment flow diagram, test matrix (success, decline, duplicate webhook, refund)
Acceptance	No price or total can be changed from the browser; duplicate webhooks do not double-process; Playwright test completes a purchase

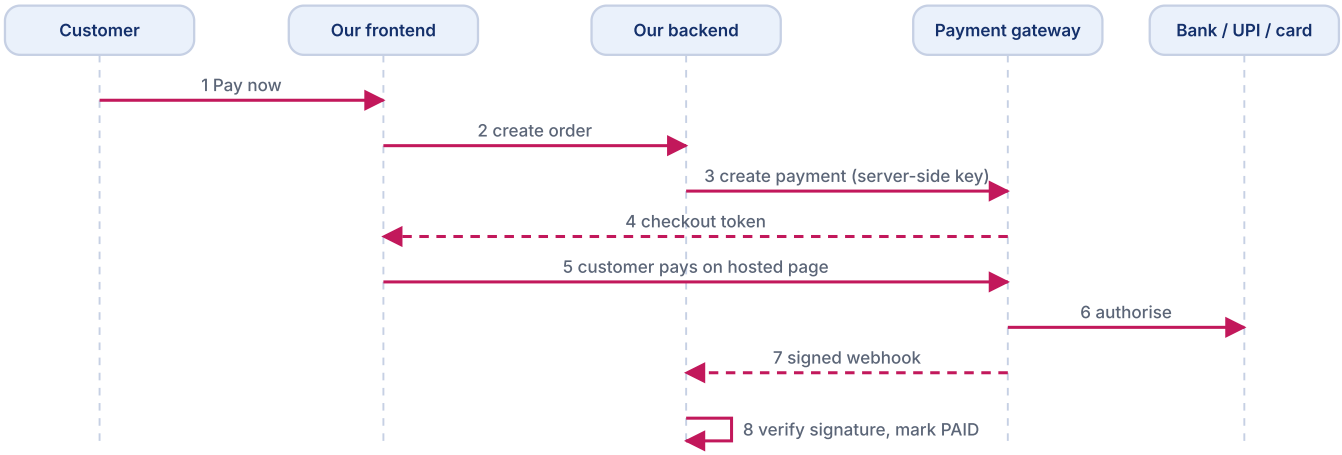


Figure 10.1 - Payment flow used in Project 5 (server-side order, signed webhook)

Project 6 - SaaS admin dashboard

Item	Detail
Brief	A multi-tenant dashboard - organisations, users and roles, invitations, subscription plan display, usage charts, audit log
Stack	Next.js + API + PostgreSQL; a charts library
Must include	Tenant isolation on every query; role-based UI and API; audit trail; responsive data tables with server pagination; accessibility for tables and charts; monitoring and logging; CI/CD to staging
Deliverables	Repo, architecture doc with tenant model, security tests for cross-tenant access, dashboards for logs/uptime
Acceptance	A user from tenant A can never read tenant B data (tests prove it); p95 API latency within target under a load test

Project 7 - Complete simulated client project

The capstone runs the **full 27-stage lifecycle** with a trainer acting as the client.

Item	Detail
Brief	The trainer supplies a one-paragraph enquiry for a realistic client (for example a dental clinic wanting a website with online booking and payment)
Team	3-4 trainees in roles (PM, UX, full-stack, QA/DevOps); roles rotate within the capstone
Required artefacts	Qualified-lead sheet, questionnaire answers, requirements, competitor matrix, costed SOW, feature-freeze register, sitemap, Figma design, architecture + ADRs, DB design, repository, working application, test report, security report, staging deployment, UAT sign-off, production-like deployment with domain and HTTPS, handover pack, monitoring and a first monthly report
Assessment	Trainer-client acceptance; Technical Lead code and security review; PM review of documents and costing; 30-minute oral defence by each trainee
Pass mark	Every gate evidenced; no critical security finding; costing reconciles; the client can operate the system from the handover pack

CERTIFICATION

On passing Project 7 and all level gates, the Technical Lead and Director issue the **Delivery Readiness Certificate**. The employee may then join supervised client projects. Certification is reviewed annually.

Project rubric (used by reviewers)

Dimension	Weight	Excellent means
Functionality	20%	All acceptance criteria met; edge cases handled
Code quality	20%	Clean, typed, tested, small PRs, consistent standards
Security	20%	Controls implemented and verified; no secrets; attack tests pass
UX and accessibility	15%	Responsive, accessible, polished states
Engineering practice	15%	Git history, CI, review, Docker, docs
Documentation and communication	10%	Clear README, ADRs, honest status, good demo

A project is accepted at 70% overall with no dimension below 50% and **no failure in Security**.

VEL SOFTWARE SOLUTIONS

PART D · PRACTICE AND SOURCES

11. Official Learning Sources

Learn from **official documentation first**. Blog posts and videos are supplements; when they disagree with the official documentation, the documentation wins. Links point to the documentation roots; sites reorganise their pages, so use each site's search if a deep link moves. Check version numbers - documentation for the wrong version causes real bugs.

Required official sources

Source	Link	Best used for	Relevant modules
MDN Web Docs	developer.mozilla.org	HTML, CSS, JavaScript, HTTP, web APIs, accessibility - the primary reference	L1.1-L1.4, L1.9, L2.1-L2.7
HTML Living Standard	html.spec.whatwg.org	Exact element and attribute behaviour (reference, not tutorial)	L2.1
TypeScript documentation	typescriptlang.org/docs	Types, generics, `tsconfig`, handbook	L2.4, L2.8
React documentation	react.dev/learn	Components, state, effects, thinking in React	L2.8
Next.js documentation	nextjs.org/docs	Routing, rendering, data fetching, deployment	L2.9
Node.js documentation	nodejs.org/docs	Runtime APIs, modules, `http`, streams, test runner	L1.2, L3.1
FastAPI	fastapi.tiangolo.com	Python APIs, validation, dependencies, security, testing	L3.1-L3.9
Django documentation	docs.djangoproject.com	ORM, admin, forms, authentication, migrations	L3.1, L4.7
PostgreSQL documentation	postgresql.org/docs	SQL, indexes, transactions, backup, performance	L4.1-L4.8
GitHub Docs	docs.github.com	Repositories, PRs, Actions, security features	L5.1, L5.2, L5.7
Docker Docs	docs.docker.com	Dockerfile, Compose, networking, security	L5.6
OWASP	owasp.org	Top 10, ASVS, cheat sheets, API security	L3.3-L3.8, L5.11
web.dev	web.dev	Performance, Core Web Vitals, accessibility, PWA, learning paths	L1.9, L2.6, L2.7, L5.10

Recommended additional official sources

Source	Link	Used for
Pro Git book	git-scm.com/book	Git in depth (L5.1)
Playwright	playwright.dev	End-to-end testing (L5.5)

Source	Link	Used for
WCAG 2.2 quick reference	w3.org/WAI/WCAG22/quickref	Accessibility success criteria (L2.6)
MySQL reference manual	dev.mysql.com/doc	MySQL/MariaDB (L4.1)
OWASP Cheat Sheet Series	cheatsheetseries.owasp.org	Practical secure-coding guidance
OWASP ASVS	owasp.org/www-project-application-security-verification-standard	Security verification requirements
Pydantic	docs.pydantic.dev	Validation models (L3.6)
Cloudflare Developers	developers.cloudflare.com	DNS, CDN, WAF, Pages (L1.4, L5.8)
AWS documentation	docs.aws.amazon.com	AWS services (L5.8)
Microsoft Learn - Azure	learn.microsoft.com/azure	Azure services (L5.8)
Google Cloud documentation	cloud.google.com/docs	GCP services (L5.8)
VS Code documentation	code.visualstudio.com/docs	Editor (L5.3)
Chrome DevTools	developer.chrome.com/docs/devtools	Debugging and performance (L2.7)
Astro docs	docs.astro.build	Content-site alternative (L2.9)
Postman Learning Center	learning.postman.com	API testing (L1.8, L5.5)

Reading plan by level

Level	Read first	Then
1	MDN "Learn web development" and HTTP overview; web.dev "Learn Responsive Design"	OWASP Top 10 introduction
2	MDN HTML, CSS, JavaScript guides; HTML Living Standard (reference); web.dev Accessibility and Performance paths	TypeScript handbook; react.dev/learn (complete the tutorial); Next.js "Learn" course
3	FastAPI tutorial (all sections: query, body, dependencies, security, testing) or Django tutorial; Node.js docs	OWASP API Security Top 10, Authentication and Session cheat sheets
4	PostgreSQL tutorial, SQL commands, indexes, transactions, backup	`EXPLAIN` guide; migrations tool docs
5	Pro Git ch. 1-3, 7; GitHub Actions docs; Docker "Get started"	Playwright intro; cloud provider "getting started"; OWASP ASVS

HOW TO READ DOCUMENTATION

1) Read the overview and concepts page. 2) Run the smallest example. 3) Change one thing and observe. 4) Read the reference for the exact option you used. 5) Write three lines of notes with the link in your training repository. Always note the **version** you used.

Reliability rules for sources

- Prefer the official site; check the "last updated" date and the version selector.
- For anything involving **prices, limits, licences, security advisories or compliance**, use only the official page dated within the last 30 days.
- Do not rely on AI-generated answers, forums or videos as evidence for a client decision - verify against official documentation or run a test.
- Record source links in ADRs and in `docs/architecture.md`.

VEL SOFTWARE SOLUTIONS

PART E · TOOLKIT

A1. Employee Onboarding Checklist

Use one copy per new employee. The mentor and the employee tick items together; the Technical Lead signs each gate. File the completed checklist in the HR/learning record.

Employee	Mentor	Start date

Section 1 - Before day 1 (HR, Technical Lead)

Item	Owner	Date	Done
Employment documents, NDA and policies prepared	HR		☐
Laptop prepared - disk encryption on, screen lock, auto-updates, company management profile	DevOps		☐
Company e-mail, GitHub organisation invitation, project tracker, chat, password manager accounts created	DevOps		☐
Mentor assigned and introduced	Technical Lead		☐
Training repository created and welcome pack shared	Mentor		☐

Section 2 - Day 1 to 2: access and security (Gate 1)

Item	Owner	Date	Done
Welcome, company overview, department structure explained	Director / TL		☐
NDA, acceptable-use, information-security, data-protection and code-of-conduct policies read and signed	Employee		☐
Password manager set up; unique strong master password; recovery kit stored safely	Employee		☐
MFA enabled on e-mail, GitHub, cloud, password manager	Employee		☐
SSH key (ed25519, passphrase) created; public key added to GitHub	Employee		☐
Security basics briefing - phishing, secrets, device hygiene, incident reporting (report within 1 hour)	TL		☐
Security basics quiz passed (80%)	Employee		☐

Section 3 - Day 2 to 5: tools and environment

Item	Owner	Date	Done
Git installed and configured (`user.name` , `user.email` company address)	Employee		☐
VS Code installed with the company extension pack and shared settings	Employee		☐

Item	Owner	Date	Done
Node.js LTS, Python 3.11+, Docker installed per README; versions recorded	Employee		☐
Cloned the training repository and ran the sample app and tests	Employee		☐
First branch, commit and pull request created, reviewed and merged	Employee / Mentor		☐
Handbook Sections 1-6 read; 27 lifecycle stages explained to mentor in own words	Employee		☐
Tracker, chat channels, meeting rhythm and timesheet process explained	PM		☐
30/60/90-day goals written and signed	Employee / Mentor		☐

Section 4 - Learning gates

Gate	Criteria	Target week	Signed by	Date
1 - Access and security	Accounts + MFA; policies; security quiz 80%	1	Technical Lead	
2 - Foundations	Levels 1-2 signed; Projects 1-2 accepted	4	Technical Lead	
3 - Backend and data	Levels 3-4 signed; Projects 3-4 accepted	8	Technical Lead	
4 - Professional practice	Level 5 signed; Projects 5-6 accepted; first code review given	11	Technical Lead	
5 - Delivery readiness	Project 7 accepted; oral assessment passed	13	Technical Lead + Director	

Section 5 - Access levels granted

Access	Granted after	Granted by	Date
Training repository (write via PR)	Gate 1	TL	
Client repositories (write via PR)	Gate 2	TL	
Staging environments	Gate 3	DevOps / TL	
Production (named projects only, via pipeline)	Gate 5	TL	
Client communication	Gate 5 + PM approval	PM	

Sign-off

Role	Name	Signature	Date
Employee			
Mentor			

Role	Name	Signature	Date
Technical Lead			

VEL SOFTWARE SOLUTIONS

PART E · TOOLKIT

A2. 90-Day Training Plan

A 90-day plan (13 weeks). Daily rhythm: **2 h reading + 4 h hands-on + 30 min notes**, 15-minute daily mentor check-in, 45-minute weekly review. Adjust pace with the mentor - gates matter more than dates.

Week	Level / module focus	Hands-on deliverable	Gate / review
1	Onboarding; L1.1-L1.3 (web architecture, browser/server, HTTP)	Dev environment; first PR; request-path diagram	Gate 1
2	L1.4-L1.9 (DNS, hosting, front/back, DB concepts, APIs, responsive)	DNS plan; hosting ADR; API collection; Level 1 quiz	Level 1 sign-off
3	L2.1-L2.2 (HTML5, CSS3); L2.7 DevTools	**Project 1** - responsive business site (start)	Mid-point review
4	L2.5-L2.6 (Flexbox/Grid, accessibility); L2.3 JavaScript (start)	Project 1 completed and deployed; accessibility audit	Project 1 accepted
5	L2.3-L2.4 (JavaScript, TypeScript)	Task-list app in TypeScript	Weekly review
6	L2.8-L2.9 (React, Next.js)	**Project 2** - multi-page corporate site	Project 2 accepted - **Gate 2**
7	L3.1-L3.2 (FastAPI/Django/Node, REST design); L4.1-L4.2 (Postgres, design)	API specification + ER design for Project 3/4	Spec review
8	L3.3-L3.5 (authentication, authorisation, business logic)	**Project 3** - auth application	Attack tests
9	L3.6-L3.9 (validation, errors, uploads, e-mail); L4.3-L4.5 (relations, SQL, indexes)	Project 3 accepted; SQL test	Project 3 accepted - **Gate 3** (with L4)
10	L4.6-L4.8 (transactions, migrations, backup); L5.1-L5.4 (Git, GitHub, VS Code, review)	**Project 4** - CRUD app with restore drill	Project 4 accepted
11	L5.5-L5.7 (testing, Docker, CI/CD)	**Project 5** e-commerce prototype (start) with CI	Pipeline demo
12	L5.8-L5.11 (cloud, logging, monitoring, security)	Project 5 deployed; **Project 6** SaaS dashboard	Security review - **Gate 4**
13	Capstone - full lifecycle with a trainer-client	**Project 7** simulated client project; oral defence	**Gate 5 - Delivery Readiness Certificate**

Weekly review template

```
WEEKLY REVIEW  -  Employee: ____  Week: __  Mentor: ____
Completed modules (ids + evidence links):
Demonstrated to mentor (what):
Assessment results:
Blockers and help needed:
Next week plan (3 priorities):
Confidence 1-5 per topic:
Mentor feedback:
Gate status:
```

30/60/90-day goals (example)

Day	Goal	Measure
30	Build and deploy a responsive, accessible site; explain the full request path	Project 1 + Level 1 quiz
60	Build a secure API with auth and a relational database	Projects 3-4 accepted
90	Deliver a simulated client project through the lifecycle	Project 7; Delivery Readiness Certificate

If you fall behind

Tell your mentor in the first week of slipping. Options: reduce breadth (keep the essential modules), add mentor pairing sessions, extend the schedule - but **never** skip a gate or sign work you cannot explain.

PART E · TOOLKIT

A3. Project Discovery Questionnaire

Send this questionnaire to the client two days before the discovery meeting. Complete it **with** the client during the meeting; record answers, assumptions and open questions. Unknown answers are recorded as "TBD - owner - due date", never guessed.

Project	Client	Date

Part 1 - Lead qualification (internal, Stage 1)

Criterion	Question	Score 1-5
Budget	Is there an approved budget range?	
Authority	Who decides and who pays? Are they in the meeting?	
Need	Is the problem clear and valuable to the business?	
Timeline	Is there a real deadline and a reason for it?	
Fit	Is it within our services and skills? Any ethical/legal red flags?	
Total (pass at 15 or more)		

Part 2 - Business

#	Question	Answer
1	What does the business do? Who are the customers?	
2	Why a new site/system now? What problem does it solve?	
3	What does success look like in 6 and 12 months (numbers)?	
4	Who are the target users? Devices and locations? Languages?	
5	Who are the main competitors? Which sites do you admire and why?	
6	What is unique about your offer?	
7	Any brand guidelines, logo files, colours, tone of voice?	
8	Who will provide text, photos, product data? By when?	

Part 3 - Functional

#	Question	Answer
9	Pages / sections needed (list)?	
10	User accounts? Roles (visitor, customer, staff, admin)?	
11	Key user journeys (enquiry, booking, purchase, download...)?	

#	Question	Answer
12	Forms and what happens after submission (e-mail, CRM, database)?	
13	Content management - who updates what, how often?	
14	E-commerce - products, variants, stock, shipping, tax, coupons, invoices?	
15	Booking - services, durations, staff, calendar, reminders, cancellations?	
16	Payments - methods, currencies, refunds, which gateway? Who owns the account?	
17	Reports, exports, dashboards needed?	
18	Search, filters, multilingual, notifications, chat?	

Part 4 - Technical

#	Question	Answer
19	Existing website/system? URL, platform, hosting, access, what must be kept?	
20	Domain name - who owns it, where is DNS, e-mail provider in use?	
21	Third-party systems to integrate (ERP, CRM, accounting, SMS/WhatsApp, analytics)? Do APIs exist?	
22	Expected traffic now and in 12 months; peak events (sales, campaigns)?	
23	Browsers/devices that must be supported?	
24	Hosting preferences or restrictions (country, provider, client's cloud account)?	
25	Existing data to migrate (volume, quality, format)?	
26	Who will maintain the system after launch?	

Part 5 - Non-functional, security and compliance

#	Question	Answer
27	Data collected - personal, financial, health, children's data?	
28	Applicable laws or standards (privacy, e-commerce, accessibility, industry rules)?	
29	Required availability (uptime) and tolerated downtime?	
30	Acceptable data loss (RPO) and recovery time (RTO)?	
31	Accessibility requirement (WCAG 2.2 AA?)	
32	SEO requirements - current rankings, redirects needed, target keywords?	
33	Performance expectations?	
34	Cookie consent, privacy policy, terms - who prepares them?	

Part 6 - Commercial and project

#	Question	Answer
35	Budget range and payment preferences?	
36	Launch date and reason for it; any hard dependencies (event, campaign)?	
37	Who is the single decision-maker and who gives day-to-day feedback?	
38	How fast can the client review and respond (days)?	
39	Support and maintenance expectations after launch?	
40	Anything we have not asked that we should know?	

Output of the meeting

Item	Owner	Due
Meeting minutes sent for confirmation (within 24 h)	PM	
Assumptions and open questions list	PM	
Problem statement confirmed in writing (Gate 02)	Client	
Requirements document started (Appendix A4)	PM / TL	

PART E · TOOLKIT

A4. Requirements Template

Copy to `docs/requirements.md` in the project repository. Every requirement has a unique id, a priority and **testable** acceptance criteria. Version this document in Git; changes after the feature freeze follow change control.

1. Document control

Project		Client	
Version		Date	
Author		Approved by (client)	

2. Business context

- **Problem statement** (confirmed at Gate 02):
- **Goals and success measures** (numbers, dates):
- **Stakeholders** (name, role, decision rights):
- **Target users / personas:**
- **Assumptions and constraints:**
- **Out of scope (explicit):**

3. Functional requirements

ID	Priority (M/S/C/W)	Requirement (user story)	Acceptance criteria (testable)	Status
REQ-001	M	As a *visitor* I want to *request a call-back* so that *the clinic contacts me*	Form validates name, phone, e-mail; success message shown; staff e-mail received within 1 minute; data stored; spam protected	Agreed
REQ-002	M			
REQ-003	S			

M = must, S = should, C = could, W = will not this time (MoSCoW).

4. Non-functional requirements

ID	Category	Requirement	Measure / verification
NFR-001	Performance	Mobile LCP at 2.5 s or less on the key pages	Lighthouse CI / field data
NFR-002	Accessibility	WCAG 2.2 AA	axe + manual keyboard test

ID	Category	Requirement	Measure / verification
NFR-003	Security	OWASP ASVS level agreed in the SOW; no critical/high open findings at release	Security report
NFR-004	Availability	Target uptime agreed in SLA (no guarantee outside the SLA)	Monitoring report
NFR-005	Compatibility	Latest 2 versions of Chrome, Edge, Firefox, Safari	Cross-browser test
NFR-006	Backup	RPO / RTO as agreed; restore tested	Restore-test log
NFR-007	SEO	Titles, meta, sitemap, redirects	SEO checklist
NFR-008	Privacy	Personal data minimised; retention defined; consent where needed	Data inventory

5. Integrations

System	Purpose	Direction	Auth	Owner of account	Sandbox available	Notes

6. Data and content

Item	Source	Owner	Due	Status
Logo / brand files	Client			
Page text	Client			
Product / service data	Client			
Photos	Client / stock			

7. Roles and permissions (access matrix draft)

Action	Visitor	Customer	Staff	Admin
View public pages	yes	yes	yes	yes
Manage own profile	no	yes	yes	yes
Manage all bookings	no	no	yes	yes
Manage users and settings	no	no	no	yes

8. Sitemap and user flows

(Insert diagrams or links.)

9. Risks and dependencies

ID	Risk / dependency	Probability	Impact	Mitigation	Owner
R-01	Client content delivered late	M	H	Dates in SOW; placeholder content approved	PM

10. Approval

Role	Name	Signature	Date
Client sponsor			
Project Manager			
Technical Lead			

VEL SOFTWARE SOLUTIONS

PART E · TOOLKIT

A5. Statement of Work Template

Statement of Work (SOW) between **VEL Software Solutions** ("Supplier") and the **Client**. This template is a working form for the Project Manager and **must be reviewed by an authorised commercial representative and, where required, legal counsel before signature**. It is not legal advice.

1. Parties and reference

Item	Detail
Client (legal name, address, tax id)	
Supplier	VEL Software Solutions
SOW number / version / date	
Project name	
Client sponsor / Supplier project manager	

2. Background and objectives

Short description of the problem, the business goals and the measurable success criteria (from the confirmed requirements).

3. Scope of work

#	Deliverable	Description	Requirement ids	Acceptance criteria
1	Discovery and plan	Workshops, requirements, sitemap, project plan		Signed requirements
2	UI/UX design	Wireframes, design system, hi-fi screens for desktop and mobile, prototype		Written client design approval
3	Frontend			
4	Backend and API			
5	Database			
6	Integrations			
7	Testing and security review	Test report, security report		No open critical/high defects
8	Deployment	Staging and production, DNS, TLS		Live URL, HTTPS valid
9	Handover and training	Documentation pack, training session		Signed handover certificate

4. Out of scope

List explicitly (for example: content writing, photography, translation beyond the languages listed, third-party licence fees, hardware, legal documents, SEO ranking guarantees, features not in section 3).

5. Assumptions

- The client provides content, brand files, and access within the dates in section 8.
- One single decision-maker consolidates feedback; review windows are `[5]` working days per round.
- Third-party services (payment, e-mail, hosting) are in the client's name unless stated.
- Estimates assume the technologies in the architecture document; a change of technology is a change request.

6. Client responsibilities

Timely content and feedback; access to systems; domain and account ownership; legal texts (privacy policy, terms); payment of fees; named contacts; participation in UAT.

7. Supplier responsibilities

Deliver the scope with professional skill; follow the agreed lifecycle and quality standards; protect client data; report progress weekly; raise risks early.

8. Timeline and milestones

Milestone	Planned date	Dependency	Payment trigger
SOW signed / kick-off			Advance
Design approved		Client feedback	Milestone 2
Staging delivered			Milestone 3
UAT accepted			
Go-live			Milestone 4
End of warranty			Retention

9. Commercial terms

Item	Detail
Price model	Fixed price / time-and-materials with cap / milestone (delete)
Project price	`[amount and currency, excluding taxes]`
Taxes	Added at the applicable rate
Third-party costs	Client-billed / pass-through at cost (list, with source and date of each price)
Payment schedule	`[percentages tied to milestones]` ; payment due in `[15]` days
Quote validity	`[30]` days

Item	Detail
Change requests	Written; quoted at `[rate card]`; signed addendum before work
Expenses	Pre-approved only

10. Acceptance

Each deliverable is accepted when it meets its criteria. The client has `[5]` working days to accept or list defects against the criteria; silence is not acceptance unless stated here. Defects are corrected at no charge; new requirements are change requests.

11. Warranty and support

`[60/90]` days warranty for defects against the accepted scope from go-live. Support and maintenance after warranty are provided only under a separate **Managed Service** schedule (monthly fee, hours included, response targets, exclusions). No uptime or security guarantee is given beyond the service schedule; **we state the tests performed, not that the system is "secure"**.

12. Intellectual property and licences

Define ownership of deliverables on full payment; supplier retains pre-existing tools and generic components; third-party and open-source components remain under their own licences (listed in the handover pack).

13. Confidentiality and data protection

Both parties keep the other's information confidential. Personal data is processed only to deliver the services and protected as described in `docs/security.md`. A data-processing agreement is attached where required.

14. Security and access

Access is granted to named individuals with individual accounts and MFA; no shared admin credentials; access is removed at handover unless a managed plan applies.

15. Change control

Per the Project Manager procedure: written request → impact assessment → client approval of revised cost/time → signed addendum.

16. Termination and dispute resolution

`[Per company legal template - reviewed by counsel.]`

17. Signatures

For the Client	For the Supplier
Name:	Name:
Title:	Title:
Signature:	Signature:
Date:	Date:

BEFORE SIGNING

Every price has a verified source and date. Scope matches the requirement ids. Out-of-scope list present. Milestones tied to accepted deliverables. Contingency included. Technical Lead reviewed feasibility. Legal review done. No "unlimited" promises.

VEL SOFTWARE SOLUTIONS

PART E · TOOLKIT

A6. QA Checklist

Complete in staging before UAT (Stage 19-21) and re-run before production. Mark **Pass**, **Fail**, **N/A** (with reason). Attach evidence (screenshots, reports, logs) in `qa/`. Release only when there are **no open critical/high defects** and every N/A is justified.

Project	Build / tag	Tester	Date

1. Functional

#	Check	Pass	Fail	N/A	Evidence
1.1	Every "must" requirement has a passing test (traceability matrix)	☐	☐	☐	
1.2	All forms validate on client and server; errors are clear	☐	☐	☐	
1.3	Happy paths and edge cases for each critical journey (E2E suite green)	☐	☐	☐	
1.4	E-mails sent with correct content, links and sender; unsubscribe where required	☐	☐	☐	
1.5	Payment sandbox: success, decline, timeout, duplicate webhook, refund	☐	☐	☐	
1.6	Uploads: size and type limits, malicious files rejected	☐	☐	☐	
1.7	Search, filters, pagination, sorting correct	☐	☐	☐	
1.8	404, 403, 500 pages friendly; no stack traces	☐	☐	☐	

2. Security

#	Check	Pass	Fail	N/A	Evidence
2.1	Access matrix tests pass; no IDOR (other users' records unreachable)	☐	☐	☐	
2.2	Passwords hashed with Argon2id/bcrypt; rate limiting and lockout work	☐	☐	☐	
2.3	Cookies HttpOnly, Secure, SameSite; CSRF protection where cookies are used	☐	☐	☐	
2.4	Input validation and output encoding; no SQL injection / XSS in tests and ZAP baseline	☐	☐	☐	
2.5	Security headers present (HSTS, CSP, nosniff, frame protection, referrer policy)	☐	☐	☐	
2.6	Dependency, secret, container scans: no unresolved critical/high	☐	☐	☐	
2.7	No secrets in repository history, logs, client bundle	☐	☐	☐	
2.8	CORS allow-list correct; admin routes protected; debug off	☐	☐	☐	

3. Accessibility (WCAG 2.2 AA)

#	Check	Pass	Fail	N/A	Evidence
3.1	Keyboard-only operation of all journeys; visible focus; no traps	☐	☐	☐	
3.2	Headings and landmarks logical; `lang` set; skip link	☐	☐	☐	
3.3	Contrast 4.5:1 text, 3:1 UI; no colour-only meaning	☐	☐	☐	
3.4	Images have alt; forms have labels and linked errors	☐	☐	☐	
3.5	axe/Lighthouse: no critical issues; screen-reader pass on key flows	☐	☐	☐	

4. Responsive and browser

#	Check	Pass	Fail	N/A	Evidence
4.1	360 / 768 / 1024 / 1440 px - no horizontal scroll, readable text, usable controls	☐	☐	☐	
4.2	Latest 2 versions: Chrome, Edge, Firefox, Safari (desktop + mobile)	☐	☐	☐	
4.3	At least one real iOS and one real Android device	☐	☐	☐	

5. Performance and SEO

#	Check	Pass	Fail	N/A	Evidence
5.1	Mobile LCP <= 2.5 s, INP <= 200 ms, CLS <= 0.1 on key pages	☐	☐	☐	
5.2	Lighthouse targets met (see Section 7.5)	☐	☐	☐	
5.3	Load test at agreed traffic: p95 and error rate within target	☐	☐	☐	
5.4	Titles, meta, canonical, sitemap, robots, redirects, structured data valid	☐	☐	☐	
5.5	Staging blocked from indexing; production allowed	☐	☐	☐	

6. Data and reliability

#	Check	Pass	Fail	N/A	Evidence
6.1	Schema built only from migrations; staging = production schema	☐	☐	☐	
6.2	Backup exists and **restore tested** within RTO	☐	☐	☐	
6.3	Logs structured, no sensitive data; request ids present	☐	☐	☐	
6.4	Health endpoint and uptime monitor; alert fire-drill passed	☐	☐	☐	
6.5	Rollback rehearsed	☐	☐	☐	

7. Documentation

#	Check	Pass	Fail	N/A	Evidence
7.1	README, architecture, database, API, deployment, security, user guide current	☐	☐	☐	
7.2	CHANGELOG and release notes updated	☐	☐	☐	

Defect log

ID	Severity (C/H/M/L)	Steps to reproduce	Expected / actual	Evidence	Owner	Status
QA-001						

Release recommendation

Result	Name	Signature	Date
QA engineer: Ready / Not ready			
Technical Lead: Go / No-go			

PART E · TOOLKIT

A7. Deployment Checklist

Use for **every** staging and production deployment. The DevOps engineer executes; the Technical Lead gives go/no-go; the PM informs the client. Keep the completed checklist as deployment evidence.

Project		Version / tag		Environment	
Date / window		Executed by		Approved by	

A. Pre-deployment (T minus 1 day)

Item	Done
Client approved the change plan and the maintenance window (UAT signed for production)	☐
Release candidate tagged; CHANGELOG and release notes complete	☐
CI green on the exact commit; the same image digest was tested in staging	☐
Infrastructure matches the approved IaC/plan (no manual drift)	☐
Secrets present in the secret store for this environment; none in the repository	☐
Database migrations reviewed; backward compatible; rollback or forward-fix plan written	☐
Fresh backup taken and restore verified	☐
Monitoring, alerts, on-call contacts and cost/budget alerts active	☐
Rollback plan tested in staging; previous image tag retained	☐
DNS plan ready; TTL lowered to 300 s at least 24 h before cut-over (if DNS changes)	☐
Admin ports closed; database private; least-privilege IAM confirmed	☐
Communication sent to client and team (time, expected impact, contact)	☐

B. Deployment

Step	Time	Done
Announce start in the team channel; open the deployment log		☐
Enable maintenance mode if needed		☐
Run the pipeline/promotion with approval; apply migrations		☐
Verify the running version and configuration		☐
Smoke tests: home, login, key journey, API `/health`, payment sandbox/live-safe check		☐
Disable maintenance mode		☐

C. Post-deployment (first 60 minutes)

Item	Done
Logs show no error spike; p95 latency and 5xx rate within normal range	☐
Monitoring shows green; uptime check passing	☐
First production backup completed	☐
HTTPS valid (padlock, chain, HSTS as planned); HTTP redirects to HTTPS; no mixed content	☐
DNS records verified from two public resolvers; e-mail send/receive tested	☐
Search engines: production indexable, staging blocked; sitemap submitted	☐
Analytics and error tracking receiving events	☐
Client smoke check and confirmation received	☐

D. Evidence record

Field	Value
Deployed version / image digest	
Start / end time	
Approver (name)	
Test results (link)	
Issues encountered and actions	
Rollback executed? (Y/N, reason)	

ROLL BACK IMMEDIATELY IF

Login or payment fails; error rate is clearly above baseline; data corruption is suspected; the smoke tests fail and the cause is not found within the agreed 15 minutes. Then report to the TL and PM and write a post-incident review.

PART E · TOOLKIT

A8. Client Handover Checklist

Complete at Stage 26. The goal: the client can **operate** the system (or knowingly hand operation to us under a managed plan). Credentials are never written in this document.

Project	Client	Handover date

1. Deliverables handed over

Item	Location / link	Delivered	Accepted
Source code repository (transfer or access as agreed)		☐	☐
Documentation pack: requirements, architecture, database design, API specification, deployment, security, user guide		☐	☐
Architecture diagrams and infrastructure inventory		☐	☐
Environment list (URLs, purpose, versions)		☐	☐
CI/CD documentation and rollback procedure		☐	☐
Access and permissions register (who has what, least privilege)		☐	☐
Backup and recovery runbook with the latest restore-test log		☐	☐
Monitoring dashboards, alert rules and contact list		☐	☐
Incident runbooks and troubleshooting guide		☐	☐
Test reports: QA, security, performance		☐	☐
Third-party services list: provider, plan, owner, renewal date, cost		☐	☐
Licence list of third-party and open-source components		☐	☐
Domain, DNS and certificate details; renewal dates		☐	☐
Cost breakdown of running costs and the maintenance schedule		☐	☐
Training recording and quick-reference sheets		☐	☐

2. Ownership transfer

Asset	Owner after handover	Transferred / access removed	Date
GitHub repository / organisation		☐	
Cloud account / project / subscription		☐	
Domain registrar and DNS		☐	
Payment gateway and e-mail provider accounts		☐	

Asset	Owner after handover	Transferred / access removed	Date
Analytics, search console, error tracking		☐	
CMS / admin accounts		☐	

CREDENTIALS

Transfer through the approved secrets manager or an expiring secure link - never in e-mail bodies, chat or documents. Rotate every secret that was known to departing staff. Remove temporary supplier access unless a managed plan applies.

3. Client competence check (performed by the client, observed by us)

Task	Done unaided
Update content / publish a page	☐
Create, edit and disable a user	☐
Run a report or export	☐
Find a log entry / error and report it correctly	☐
Trigger a backup and restore into a test environment (or witness one)	☐
Know how to raise a support request and what the severities mean	☐

4. Warranty, support and next steps

Item	Detail
Warranty period (start - end)	
Support plan / SLA reference	
Contacts and escalation path	
Planned maintenance schedule (patching, restore tests, reviews)	
Upcoming renewals (domain, certificates, licences)	
Recommended improvements (backlog)	

5. Acceptance certificate

We confirm that the project _____ has been delivered, reviewed and accepted according to the SOW dated _____, that the handover items above were provided, and that the warranty period begins on _____.

For the Client	For the Supplier
Name / title:	Project Manager:
Signature / date:	Technical Lead:

For the Client	For the Supplier
	Signature / date:

PART E · TOOLKIT

A9. Project Costing Spreadsheet Specification

This is the specification for the **Project Costing Workbook**. A working implementation is supplied as `VEL-Web-Project-Costing-Workbook.xlsx` (built to this specification, with live formulas). Use the workbook for every quotation; keep one copy per client version in the project folder.

1. Principles

- **Inputs are blue cells; formulas are black.** Inputs live only on `Assumptions`, `Estimate` hours, `Operating` vendor lines and `VendorLog`.
- **No vendor price is typed without a `VendorLog` row** (source URL, date checked, region, currency, plan, assumptions, checked-by).
- Currency is stated once on `Assumptions`; all other sheets inherit it. Taxes are excluded from every figure and added in the quotation.
- Margin and markup are separate, labelled outputs. Prices round **up** to the configured step.
- The workbook is **read-only for finished quotes** (save as PDF + archive); changes create a new version number.

2. Sheet specification

Sheet	Purpose	Key contents
`README`	Instructions, colour legend, warnings, version	Placeholder warning; how to use; change log
`Assumptions`	Global inputs	Currency; rate card (role, cost/hour); contingency % per size; target margin % per size; commission %; annual fixed costs; price rounding; service contingency and margin; renewal inflation (vendor, wages)
`Estimate`	Project cost and price	Work items x role x hours (Small/Medium/Enterprise), cost per line, labour subtotal, third-party one-time, subtotal, contingency, total cost, selling price, gross profit, gross margin %, markup %, variable cost, contribution margin, break-even projects, break-even hours
`Operating`	Recurring cost and service price	Vendor pass-through lines, service labour (hours x rate), service contingency, monthly operating cost, annual operating cost, monthly fee, MRR, ARR, service margin, renewal fee
`Scenarios`	Usage-based cost at 100 / 1,000 / 10,000 / 100,000 users	Compute, database, storage, network, monitoring (inputs from calculator), total, cost per user, margin check
`VendorLog`	Evidence for every external price	Vendor, item, plan, region, currency, unit price, quantity assumption, URL, date checked, checked by, reviewer
`Summary`	One-page result for the quotation	Price, GM %, CM, break-even, monthly fee, annual fee, renewal

3. Formulas (Excel notation)

Output	Formula
Line cost	`=Hours * VLOOKUP(Role, RateCard, 2, FALSE)`
Labour	`=SUM(line costs)`
Subtotal	`=Labour + ThirdPartyOneTime`
Contingency	`=Subtotal * ContingencyPct`
Total delivery cost	`=Subtotal + Contingency`
Selling price	`=CEILING(TotalCost / (1 - TargetMargin), RoundTo)`
Gross profit	`=Price - TotalCost`
Gross margin %	`=GrossProfit / Price`
Markup %	`=GrossProfit / TotalCost`
Variable cost	`=ThirdPartyOneTime + Commission * Price`
Contribution margin	`=Price - VariableCost`
Break-even projects / year	`=AnnualFixed / ContributionMargin`
Blended rate	`=Labour / TotalHours`
Break-even hours	`=(Price - ThirdParty - Contingency - Commission*Price) / BlendedRate`
Monthly operating cost	`=(VendorTotal + ServiceLabour) * (1 + ServiceContingency)`
Annual operating cost	`=MonthlyCost * 12`
Monthly service fee	`=CEILING(MonthlyCost / (1 - ServiceMargin), 5)`
MRR / ARR	`=SUM(monthly fees)` / `=MRR * 12`
Next-year cost	`=(VendorTotal*(1+VendorInflation) + ServiceLabour*(1+WageInflation)) * (1 + ServiceContingency)`
Renewal fee / month	`=MAX(CurrentFee, CEILING(NextYearCost / (1 - ServiceMargin), 5))`
Cost per active user	`=TotalMonthlyCost / ActiveUsers`

4. Validation and controls

Control	Rule
Data validation	Margins and contingency between 0 and 0.9; hours >= 0; role chosen from the rate card list
Conditional formatting	Gross margin below policy minimum → red; vendor price older than 30 days → amber; missing `VendorLog` URL → red
Protection	Formula cells locked; inputs unlocked
Cross-check	`Summary` totals equal the sum of detail sheets (a check cell shows OK/ERROR)

Control	Rule
Review	Technical Lead reviews hours; PM reviews commercial terms; a second person re-opens at least 20% of vendor links
Versioning	File name `client-project-costing-vN.xlsx`; changes logged in `README`

5. Quotation output checklist

- [ ] Scope and assumptions match the SOW
- [ ] Every vendor price traces to a `VendorLog` row dated within 30 days
- [ ] Contingency included; margin stated as **margin**, not markup
- [ ] Taxes excluded and stated; currency stated
- [ ] Monthly service fee, annual fee and renewal rule included
- [ ] Break-even and gross-margin check passed policy minimum
- [ ] Reviewed by Technical Lead and PM; approved by the authorised commercial representative

PART E · TOOLKIT

A10. Glossary

Term	Meaning
A record / AAAA	DNS record mapping a name to an IPv4 / IPv6 address
ADR	Architecture Decision Record - short document capturing a technical decision and its reasons
API	Application Programming Interface - a defined way for programs to talk to each other
ARIA	Accessible Rich Internet Applications - attributes that add accessibility meaning when HTML alone is not enough
ARR / MRR	Annual / Monthly Recurring Revenue
Argon2id	A modern, slow, memory-hard password-hashing algorithm
Authentication / Authorisation	Proving who you are / deciding what you may do
Backup / restore	A copy of data / the act of bringing it back; only a tested restore proves a backup
Branch	An independent line of development in Git
CDN	Content Delivery Network - caches content close to users
CI / CD	Continuous Integration (build and test every change) / Continuous Delivery or Deployment
CMS	Content Management System
Contingency	Reserve added to an estimate for uncertainty
Contribution margin	Price minus variable costs - what a project adds toward fixed costs and profit
CORS	Cross-Origin Resource Sharing - browser rule controlling which sites may call an API
CRUD	Create, Read, Update, Delete
CSP	Content Security Policy - header that restricts what a page may load or run
CSRF	Cross-Site Request Forgery - tricking a browser into sending an unwanted request
DNS	Domain Name System - turns names into IP addresses
Docker / container	Tool and format for packaging an app with its dependencies so it runs the same everywhere
ER diagram	Entity-Relationship diagram of database tables and relations
Gate	A mandatory approval point; work does not continue until it is signed
Gross margin	(Price - direct cost) / price
HSTS	HTTP Strict Transport Security - tells browsers to use only HTTPS
HTTP / HTTPS	The web protocol / HTTP over TLS encryption
IaC	Infrastructure as Code (Terraform/OpenTofu)

Term	Meaning
Idempotent	Repeating the operation has the same effect as doing it once
IDOR	Insecure Direct Object Reference - reading another user's record by changing an id
INP / LCP / CLS	Core Web Vitals - interaction responsiveness, loading speed, visual stability
JWT	JSON Web Token - signed token carrying claims
Markup	(Price - cost) / cost; not the same as margin
Migration	Versioned script that changes the database schema
MFA	Multi-Factor Authentication
OWASP	Open Worldwide Application Security Project - security standards and guidance
PR	Pull Request - a proposed change reviewed before merging
RBAC	Role-Based Access Control
REST	Architectural style for APIs using resources and HTTP methods
RPO / RTO	Recovery Point Objective (acceptable data loss) / Recovery Time Objective (acceptable downtime)
SEO	Search Engine Optimisation
SLA / SLO	Service Level Agreement / Objective - agreed service targets
SOW	Statement of Work - the agreed scope, price and terms
SQL injection	Attack that inserts SQL through unvalidated input; prevented by parameterised queries
SSR / SSG	Server-Side Rendering / Static Site Generation
TLS	Transport Layer Security - encryption used by HTTPS
UAT	User Acceptance Testing - the client verifies the product against acceptance criteria
WCAG	Web Content Accessibility Guidelines (target: 2.2 level AA)
XSS	Cross-Site Scripting - injecting script into a page viewed by other users